

The AppContainer

We've seen the steps required to create processes back in [Chapter 3](#); we've also seen some of the extra steps required to create UWP processes. The initiation of creation is performed by the `DCOMLaunch` service, because UWP packages support a set of protocols, one of which is the Launch protocol. The resulting process gets to run inside an AppContainer. Here are several characteristics of packaged processes running inside an AppContainer:

- The process token integrity level is set to Low, which automatically restricts access to many objects and limits access to certain APIs or functionality for the process, as discussed earlier in this chapter.
- UWP processes are always created inside a job (one job per UWP app). This job manages the UWP process and any background processes that execute on its behalf (through nested jobs). The jobs allow the Process State Manager (PSM) to suspend or resume the app or background processing in a single stroke.
- The token for UWP processes has an AppContainer SID, which represents a distinct identity based on the SHA-2 hash of the UWP package name. As you'll see, this SID is used by the system and other applications to explicitly allow access to files and other kernel objects. This SID is part of the `APPLICATION PACKAGE AUTHORITY` instead of the `NT AUTHORITY` you've mostly seen so far in this chapter. Thus, it begins with S-1-15-2 in its string format, corresponding to `SECURITY_APP_PACKAGE_BASE_RID (15)` and `SECURITY_APP_PACKAGE_BASE_RID (2)`. Because a SHA-2 hash is 32 bytes, there are a total of eight RIDs (recall that a RID is the size of a 4-byte `ULONG`) in the remainder of the SID.
- The token may contain a set of capabilities, each represented with a SID. These capabilities are declared in the application manifest and shown on the app's page in the store. Stored in the capability section of the manifest, they are converted to SID format using rules we'll see shortly, and belong to the same SID authority as in the previous bullet, but using the well-known `SECURITY_CAPABILITY_BASE_RID (3)` instead. Various com-

ponents in the Windows Runtime, user-mode device-access classes, and kernel can look for capabilities to allow or deny certain operations.

■ The token may only contain the following privileges:

`SeChangeNotifyPrivilege` , `SeIncrease-WorkingSetPrivilege` ,
`SeShutdownPrivilege` , `SeTimeZonePrivilege` , and `SeUndockPrivilege` .

These are the default set of privileges associated with standard user accounts. Additionally, the `AppContainerPrivilegesEnabledExt` function part of the `ms-win-ntos-ksecurity` API Set contract extension can be present on certain devices to further restrict which privileges are enabled by default.

■ The token will contain up to four security attributes (see the section on attribute-based access control earlier in this chapter) that identify this token as being associated with a UWP packaged application. These attributes are added by the `DcomLaunch` service as indicated earlier, which is responsible for the activation of UWP applications. They are as follows:

- **WIN://PKG** This identifies this token as belonging to a UWP packaged application. It contains an integer value with the application’s origin as well as some flags. See [Table 7-13](#) and [Table 7-14](#) for these values.

Origin	Meaning
Unknown (0)	The package origin is unknown.
Unsigned (1)	The package is unsigned.
Inbox (2)	The package is associated with a built-in (inbox) Windows application.
Store (3)	The package is associated with a UWP application downloaded from the store. This origin is validated by checking if the DACL of the file associated with the main UWP application’s executable contains a trust ACE.
Developer Unsigned (4)	The package is associated with an unsigned developer key.
Developer Signed (5)	The package is associated with a signed developer key.
Line-of-Business (6)	The package is associated with a side-loaded line-of-business (LOB) application.

TABLE 7-13 Package origins

Flag	Meaning
PSM_ACTIVATION_TOKEN_PACKAGED_APPLICATION (0x1)	This indicates that the AppContainer UWP application is stored in AppX packaged format. This is the default.
PSM_ACTIVATION_TOKEN_SHARED_ENTITY (0x2)	This indicates that this token is being used for multiple executables all part of the same AppX packaged UWP application.
PSM_ACTIVATION_TOKEN_FULL_TRUST (0x4)	This indicates that this AppContainer token is being used to host a Project Centennial (Windows Bridge for Desktop) converted Win32 application.
PSM_ACTIVATION_TOKEN_NATIVE_SERVICE (0x8)	This indicates that this AppContainer token is being used to host a packaged service created by the Service Control Manager (SCM)'s Resource Manager. See Chapter 9 in Part 2 for more information on services.
PSM_ACTIVATION_TOKEN_DEVELOPMENT_APP (0x10)	This indicates that this is an internal development application. Not used on retail systems.
BREAKAWAY_INHIBITED (0x20)	The package cannot create a process that is not itself packaged as well. This is set by using the PROC_THREAD_ATTRIBUTE_DESKTOP_APP_POLICY process-creation attribute. (See Chapter 3 for more information.)

TABLE 7-14 Package flags

- **WIN://SYSAPPID** This contains the application identifiers (called *package monikers* or *string names*) as an array of Unicode string values.
- **WIN://PKGHOSTID** This identifies the UWP package host ID for packages that have an explicit host through an integer value.
- **WIN://BGKD** This is only used for background hosts (such as the generic background task host BackgroundTaskHost.exe) that can store packaged UWP services running as COM providers. The attribute's name stands for *background* and contains an integer value that stores its explicit host ID.

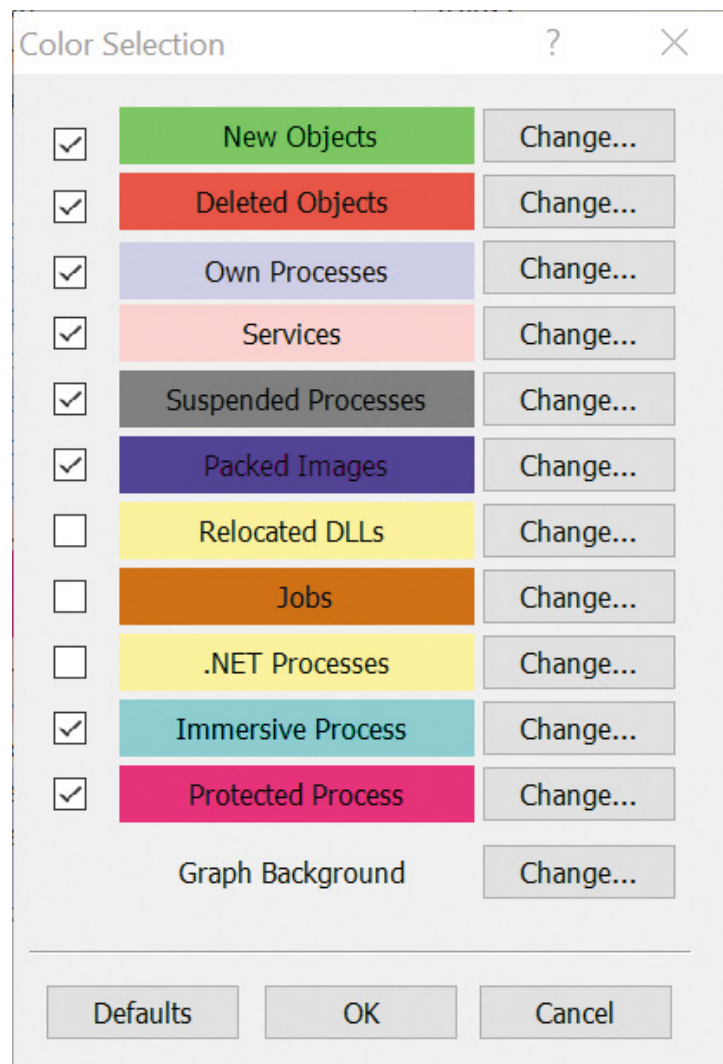
The **TOKEN_LOWBOX** (0x4000) flag will be set in the **token's Flags** member, which can be queried with various Windows and kernel APIs (such as **GetTokenInformation**). This allows components to identity and operate differently under the presence of an AppContainer token.



A second type of AppContainer exists: a child AppContainer. This is used when a UWP AppContainer (or parent AppContainer) wishes to create its own nested AppContainer to further lock down the security of the application. Instead of eight RIDs, a child AppContainer has four additional RIDs (the first eight match the parents') to uniquely identify it.

EXPERIMENT: VIEWING UWP PROCESS INFORMATION

There are several ways to look at UWP processes, some more obvious than others. Process Explorer can highlight processes that use the Windows Runtime in color (cyan by default). To see this in action, open Process Explorer, open the **Options** menu, and choose **Configure Colors**. Then make sure the **Immersive Processes** check box is selected.



Immersive process is the original term used to describe WinRT (now UWP) apps in Windows 8. (They were mostly full screen, and therefore

“immersive.”) This distinction is available by calling the

`IsImmersiveProcess` API.

Run `Calc.exe` and switch to Process Explorer. You should see several processes highlighted in cyan, including `Calculator.exe`. Now minimize the Calculator app and notice that the cyan highlight has turned gray. This is because Calculator has been suspended. Restore Calculator’s window, and it’s back to cyan.

You should have similar experiences with other apps—for example, Cortana (`SearchUI.exe`). Click or tap the **Cortana** icon on the taskbar and then close it. You should see the gray to cyan and back to gray transition. Or, click or tap the **Start** button. `ShellExperienceHost.exe` highlights in a similar fashion.

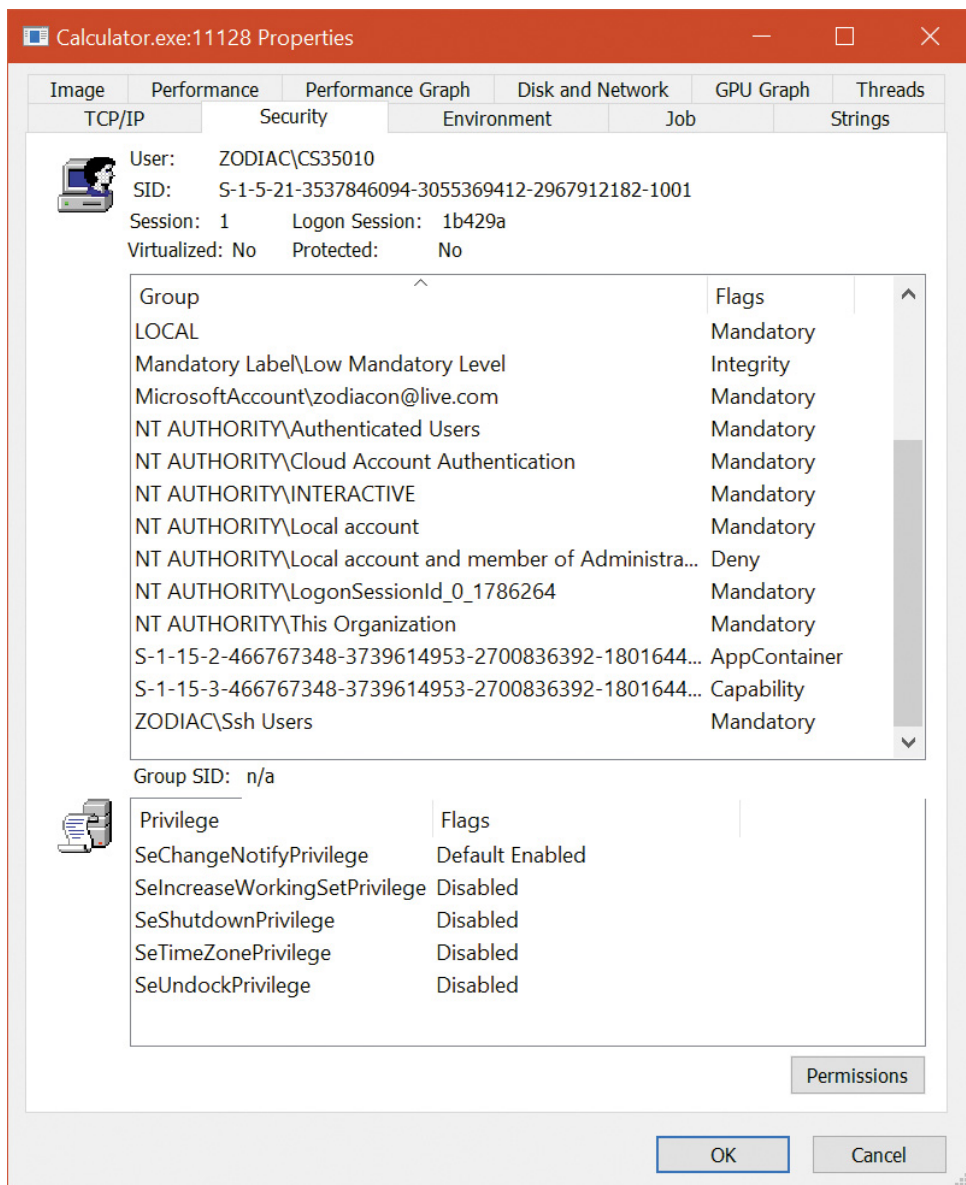
The presence of some cyan-highlighted processes might surprise you, such as `Explorer.exe`, `TaskMgr.Exe`, and `RuntimeBroker.exe`. These are not really apps, but use Windows Runtime APIs, and so are classified as immersive. (The role of `RuntimeBroker` will be discussed shortly.)

Finally, make sure the Integrity column is visible in Process Explorer and sort by that column. You’ll find processes such as `Calculator.exe` and `SearchUI.exe` with `AppContainer` integrity level. Notice that `Explorer` and `TaskMgr` are not there, clearly showing they are not UWP processes, and so live under different rules.

Process	PID	CPU	Integrity	Private Bytes	Working Set	Description
Interrupts	n/a	0.74		0 K	0 K	Hardware Interrupt
Calculator.exe	11128		AppContainer	13,420 K	36,712 K	
Microsoft.Msn.Weather.exe	3040	Suspended	AppContainer	45,600 K	15,764 K	Weather
SearchUI.exe	3728	Suspended	AppContainer	43,508 K	102,564 K	Search and Cortana
ShellExperienceHost.exe	1872	Suspended	AppContainer	79,872 K	130,636 K	Windows Shell Expi
SkypeHost.exe	9624	Suspended	AppContainer	33,084 K	26,844 K	Microsoft Skype Pr
System Idle Process	0	92.57		0 K	4 K	
Twitter.Windows.exe	2084	Suspended	AppContainer	33,348 K	8,576 K	Twitter.Windows
Video.UI.exe	8772	Suspended	AppContainer	19,900 K	37,444 K	Video Application
firefox.exe	11856	0.31	Low	571,804 K	615,372 K	Firefox
ApntEx.exe	1060		Medium	1,844 K	5,960 K	Alps Pointing-devic
Assist.exe	9934	0.01	Medium	2,644 K	12,776 K	Alps Pointing-devic

EXPERIMENT: VIEWING AN APPCONTAINER TOKEN

You can look at the properties of an `AppContainer` hosted process with several tools. In Process Explorer, the Security tab shows the capabilities associated with the token. Here’s the Security tab for `Calculator.exe`:



Notice two interesting pieces of information: the AppContainer SID, shown in the Flags column as AppContainer, and a single capability, right underneath the AppContainer SID. Except for the base RID (`SECURITY_APP_PACKAGE_BASE_RID` versus `SECURITY_CAPABILITY_BASE_RID`), the remaining eight RIDs are identical, and both refer to the package name in SHA-2 format as discussed. This shows you that there will always be one implicit capability, the capability of being the package itself, which really means Calculator requires no capabilities at all. The upcoming capabilities section covers a much more complex example.

EXPERIMENT: VIEWING APPCONTAINER TOKEN ATTRIBUTES

You can obtain similar information on the command line by using the `AccessChk` Sysinternals tool while also adding a full list of all of the token's attributes. For example, running `AccessChk` with the `-p -f`

switches followed by the process ID for SearchUI.exe, which hosts Cortana, shows the following:

[Click here to view code image](#)

```
C:\>accesschk -p -f 3728
```

Accesschk v6.10 - Reports effective permissions for securable objects

Copyright (C) 2006-2016 Mark Russinovich

Sysinternals - www.sysinternals.com

[7416] SearchUI.exe

RW DESKTOP-DD6KTPM\aigne

RW NT AUTHORITY\SYSTEM

RW Package\S-1-15-2-1861897761-1695161497-2927542615-642690995-

327840285-2659745135-2630312742

Token security:

RW DESKTOP-DD6KTPM\aigne

RW NT AUTHORITY\SYSTEM

RW DESKTOP-DD6KTPM\aigne-S-1-5-5-0-459087

RW Package\S-1-15-2-1861897761-1695161497-2927542615-642690995-

327840285-2659745135-2630312742

R BUILTIN\Administrators

Token contents:

User:

DESKTOP-DD6KTPM\aigne

AppContainer:

Package

\S-1-15-2-1861897761-1695161497-2927542615-642690995-327840285-

2659745135-2630312742

Groups:

Mandatory Label\Low Mandatory Level INTEGRITY

Everyone MANDATORY

NT AUTHORITY\Local account and member of Administrators group

DENY

...

Security Attributes:

WIN://PKGHOSTID

TOKEN_SECURITY_ATTRIBUTE_TYPE_UINT64

[0] 1794402976530433

WIN://SYSAPPID

TOKEN_SECURITY_ATTRIBUTE_TYPE_STRING

```

[0]
Microsoft.Windows.Cortana_1.8.3.14986_neutral_neutral_cw5n1h2txyewy
[1] CortanaUI
[2] Microsoft.Windows.Cortana_cw5n1h2txyewy
WIN://PKG
TOKEN_SECURITY_ATTRIBUTE_TYPE_UINT64
[0] 131073
TSA://ProcUnique
[TOKEN_SECURITY_ATTRIBUTE_NON_INHERITABLE]
[TOKEN_SECURITY_ATTRIBUTE_COMPARE_IGNORE]
TOKEN_SECURITY_ATTRIBUTE_TYPE_UINT64
[0] 204
[1] 24566825

```

First is the package host ID, converted to hex: 0x66000000000001. Because all package host IDs begin with 0x66, this means Cortana is using the first available host identifier: 1. Next are the system application IDs, which contain three strings: the strong package moniker, the friendly application name, and the simplified package name. Finally, you have the package claim, which is 0x20001 in hex. Based on the [Table 7-13](#) and [Table 7-14](#) fields you saw, this indicates an origin of `Inbox` (2) and flags set to `PSM_ACTIVATION_TOKEN_PACKAGED_APPLICATION`, confirming that Cortana is part of an AppX package.

AppContainer security environment

One of the biggest side-effects caused by the presence of an AppContainer SID and related flags is that the access check algorithm you saw in the “[Access checks](#)” section earlier in this chapter is modified to essentially ignore all regular user and group SIDs that the token may contain, essentially treating them as deny-only SIDs. This means that even though Calculator may be launched by a user John Doe belonging to the Users and Everyone groups, it will fail any access checks that grant access to John Doe’s SID, the Users group SID, or the Everyone group SID. In fact, the only SIDs that are checked during the discretionary access check algorithm will be that of the AppContainer SID, followed by the capability access check algorithm, which will look at any capability SIDs part of the token.

Taking things even further than merely treating the discretionary SIDs as deny-only, AppContainer tokens effect one further critical security change

to the access check algorithm: a NULL DACL, typically treated as an allow-anyone situation due to the lack of any information (recall that this is different from an empty DACL, which is a deny-everyone situation due to explicit allow rules), is ignored and treated as a deny situation. To make matters simple, the only types of securable objects that an AppContainer can access are those that explicitly have an allow ACE for its AppContainer SID or for one of its capabilities. Even unsecured (NULL DACL) objects are out of the game.

This situation causes compatibility problems. Without access to even the most basic file system, registry, and object manager resources, how can an application even function? Windows takes this into account by preparing a custom execution environment, or “jail” if you will, specifically for each AppContainer. These jails are as follows:



So far we’ve implied that each UWP packaged application corresponds to one AppContainer token. However, this doesn’t necessarily imply that only a single executable file can be associated with an AppContainer. UWP packages can contain multiple executable files, which all belong to the same AppContainer. This allows them to share the same SID and capabilities and exchange data between each other, such as a micro-service back-end executable and a foreground front-end executable.

■ The AppContainer SID’s string representation is used to create a subdirectory in the object manager’s namespace under `\Sessions\x\AppDataContainerNamedObjects`. This becomes the private directory of named kernel objects. This specific subdirectory object is then ACLed with the AppContainer SID associated with the AppContainer that has an allow-all access mask. This is in contrast to desktop apps, which all use the `\Sessions\x\BaseNamedObjects` subdirectory (within the same session `x`). We’ll discuss the implications of that shortly, as well as the requirement for the token to now store handles.

■ The token will contain a LowBox number, which is a unique identifier into an array of LowBox Number Entry structures that the kernel stores in the `g_SessionLowboxArray` global variable. Each of these maps to a `SEP_LOWBOX_NUMBER_ENTRY` structure that, most importantly, contains an

atom table unique to this AppContainer, because the Windows Subsystem Kernel Mode Driver (Win32k.sys) does not allow AppContainers access to the global atom table.

- The file system contains a directory in %LOCALAPPDATA% called Packages. Inside it are the package monikers (the string version of the AppContainer SID—that is, the package name) of all the installed UWP applications. Each of these application directories contains application-specific directories, such as TempState, RoamingState, Settings, LocalCache, and others, which are all ACLed with the specific AppContainer SID corresponding to the application, set to an allow-all access mask.

- Within the Settings directory is a Settings.dat file, which is a registry hive file that is loaded as an application hive. (You will learn more about application hives in Chapter 9 in Part 2.) The hive acts as the local registry for the application, where WinRT APIs store the various persistent state of the application. Once again, the ACL on the registry keys explicitly grants allow-all access to the associated AppContainer SID.

These four jails allow AppContainers to securely, and locally, store their file system, registry, and atom table without requiring access to sensitive user and system areas on the system. That being said, what about the ability to access, at least in read-only mode, critical system files (such as Ntdll.dll and Kernel32.dll) or registry keys (such as the ones these libraries will need), or even named objects (such as the \RPC Control\DNSResolver ALPC port used for DNS lookups)? It would not make sense, on each UWP application or uninstallation, to re-ACL entire directories, registry keys, and object namespaces to add or remove various SIDs.

To solve this problem, the security subsystem understands a specific group SID called `ALL APPLICATION PACKAGES`, which automatically binds itself to any AppContainer token. Many critical system locations, such as %SystemRoot%\System32 and HKLM\Software\Microsoft\Windows\CurrentVersion, will have this SID as part of their DACL, typically with a read or read-and-execute access mask. Certain objects in the object manager namespace will have this as well, such as the DNSResolver ALPC port in the \RPC Control object manager directory. Other examples include certain COM objects, which grant the execute right. Although not officially documented, third-party developers, as they create non-UWP applications, can also allow interactions with UWP applications by also applying this SID to their own resources.

Unfortunately, because UWP applications can technically load almost any Win32 DLL as part of their WinRT needs (because WinRT is built on top of Win32, as you saw), and because it's hard to predict what an individual UWP application might need, many system resources have the `ALL APPLICATION PACKAGES` SID associated with their DACL as a precaution. This now means there is no way for a UWP developer, for example, to prevent DNS lookups from their application. This greater-than-needed access is also helpful for exploit writers, which could leverage it to escape from the AppContainer sandbox. Newer versions of Windows 10, starting with version 1607 (Anniversary Update), contain an additional element of security to combat this risk: Restricted AppContainers.

By using the

`PROC_THREAD_ATTRIBUTE_ALL_APPLICATION_PACKAGES_POLICY` process attribute and setting it to

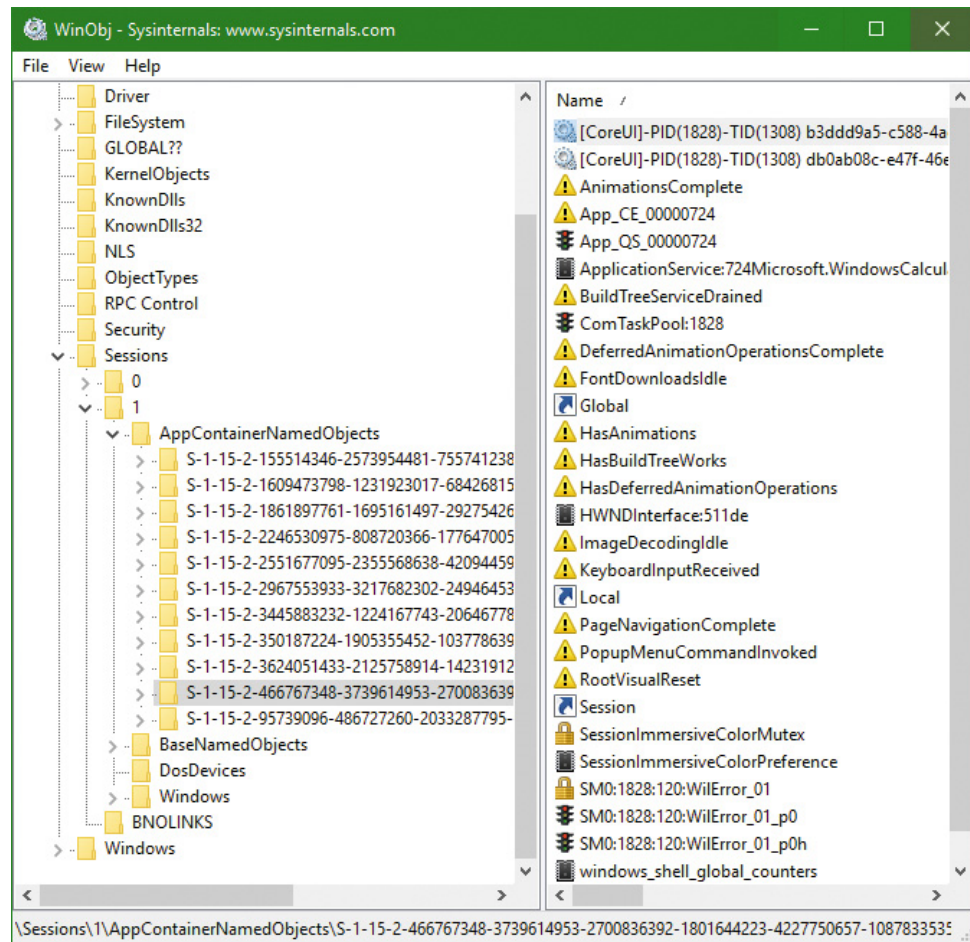
`PROCESS_CREATION_ALL_APPLICATION_PACKAGES_OPT_OUT` during process creation (see [Chapter 3](#) for more information on process attributes), the token will not be associated with any ACEs that specify the `ALL APPLICATION PACKAGES` SID, cutting off access to many system resources that would otherwise be accessible. Such tokens can be identified by the presence of a fourth token attribute named `WIN://NOALLAPPPKG` with an integer value set to `1`.

Of course, this takes us back to the same problem: How would such an application even be able to load `Ntdll.dll`, which is key to any process initialization? Windows 10 version 1607 introduces a new group, called `ALL RESTRICTED APPLICATION PACKAGES`, which takes care of this problem. For example, the `System32` directory now also contains this SID, also set to allow read and execute permissions, because loading DLLs in this directory is key even to the most sandboxed process. However, the `DNSResolver` ALPC port does not, so such an AppContainer would lose access to DNS.

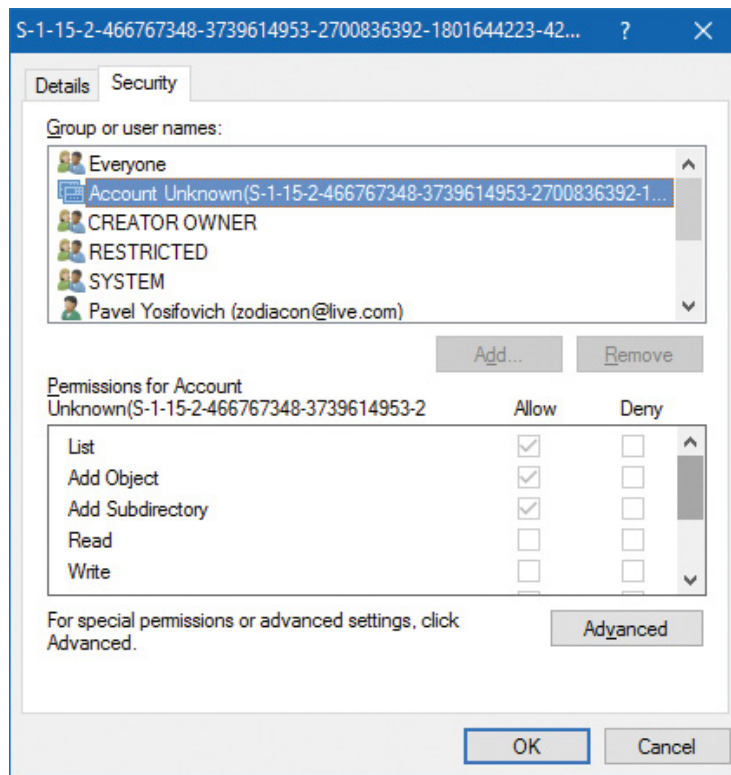
EXPERIMENT: VIEWING APPCONTAINER SECURITY ATTRIBUTES

In this experiment, we'll look at the security attributes of some of the directories mentioned in the previous section.

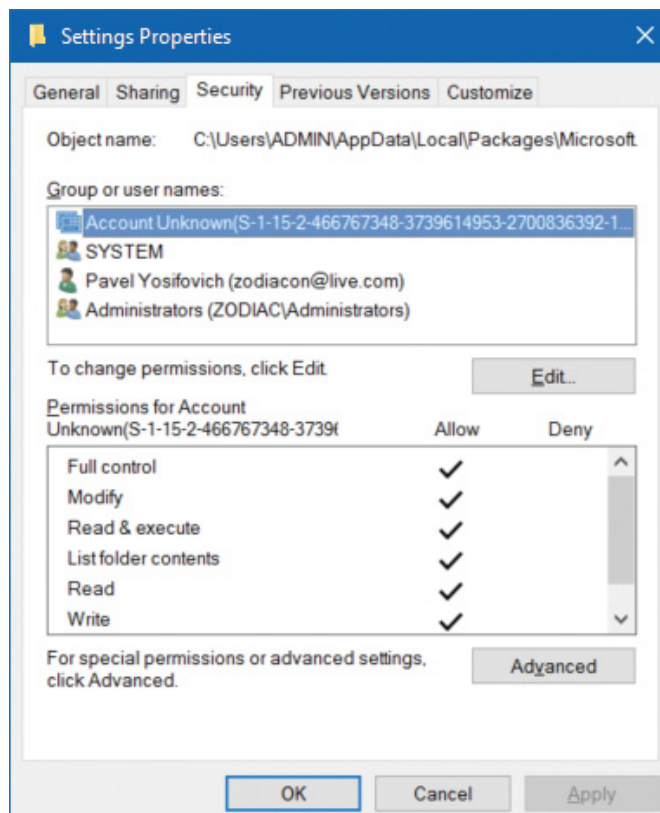
1. Make sure Calculator is running.
2. Open WinObj elevated from Sysinternals and navigate to the object directory corresponding to Calculator's AppContainer SID. (You saw it in a previous experiment.)



3. Right-click the directory, select **Properties**, and click the **Security** tab. You should see something like the following screenshot. Calculator's AppContainer SID has permission to list, add object, and add subdirectory (among others scrolled out of view), which simply means Calculator can create kernel objects under this directory.

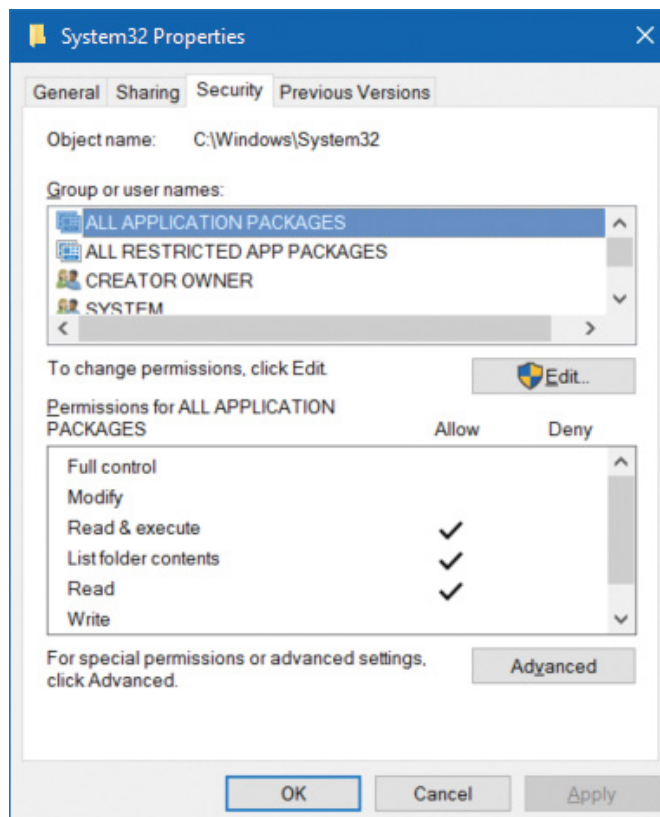


4. Open Calculator's local folder by navigating to %LOCALAPPDATA%\Packages\Microsoft.WindowsCalculator_8wekyb3d8bbwe. Then right-click the **Settings** subdirectory, select **Properties**, and click the **Security** tab. You should see Calculator's AppContainer SID having full permissions for the folder:



5. In Explorer, open the %SystemRoot% directory (for example, C:\Windows), right-click the **System32** directory, select **Properties**, and click the **Security** tab. You should see the read and execute permissions

for all application packages and all restricted application packages (if using Windows 10 version 1607 or later):



As an alternative, you can use the AccessChk Sysinternals command-line tool to view the same information.

EXPERIMENT: VIEWING THE APPCONTAINER ATOM TABLE

An atom table is a hash table of integers to strings that's used by the windowing system for various identification purposes, such as Window Class registration (`RegisterClassEx`) and custom Windows messages. The AppContainer private atom table can be viewed with the kernel debugger:

1. Run Calculator, open WinDbg, and start local kernel debugging.
2. Find the Calculator process:

[Click here to view code image](#)

```
lkd> !process 0 1 calculator.exe
```

```
PROCESS ffff828cc9ed1080
```

```
    SessionId: 1 Cid: 4bd8  Peb: d040bbc000 ParentCid: 03a4
```

```
DeepFreeze
```

```
    DirBase: 5fcaa000 ObjectTable: ffff950ad9fa2800 HandleCount:
```

<Data Not Accessible>

Image: Calculator.exe

VadRoot ffff828cd2c9b6a0 Vads 168 Clone 0 Private 2938. Modified
3332.

Locked 0.

DeviceMap ffff950aad2cd2f0

Token **ffff950adb313060**

...

3. Use the token value with the following expressions:

[Click here to view code image](#)

```
lkd> r? @$t1 = @$t0->NumberOfBuckets
```

```
lkd> r? @$t0 = (nt!_RTL_ATOM_TABLE*)
```

```
((nt!_token*)0xffff950adb313060)-
```

```
>LowboxNumberEntry->AtomTable
```

```
lkd> .for (r @$t3 = 0; @$t3 < @$t1; r @$t3 = @$t3 + 1) { ?? (wchar_t*)@$t0-
```

```
>Buckets[ @$t3]->Name }
```

```
wchar_t * 0xffff950a'ac39b78a
```

```
"Protocols"
```

```
wchar_t * 0xffff950a'ac17b7aa
```

```
"Topics"
```

```
wchar_t * 0xffff950a'b2fd282a
```

```
"TaskbarDPI_Deskband"
```

```
wchar_t * 0xffff950a'b3e2b47a
```

```
"Static"
```

```
wchar_t * 0xffff950a'b3c9458a
```

```
"SysTreeView32"
```

```
wchar_t * 0xffff950a'ac34143a
```

```
"UxSubclassInfo"
```

```
wchar_t * 0xffff950a'ac5520fa
```

```
"StdShowItem"
```

```
wchar_t * 0xffff950a'abc6762a
```

```
"SysSetRedraw"
```

```
wchar_t * 0xffff950a'b4a5340a
```

```
"UIA_WindowVisibilityOverridden"
```

```
wchar_t * 0xffff950a'ab2c536a
```

```
"True"
```

...

```
wchar_t * 0xffff950a'b492c3ea
```

```
"tooltips_class"
```

```
wchar_t * 0xffff950a'ac23f46a
"Save"
wchar_t * 0xffff950a'ac29568a
"MSDraw"
wchar_t * 0xffff950a'ac54f32a
"StdNewDocument"
wchar_t * 0xffff950a'b546127a
"{FB2E3E59-B442-4B5B-9128-2319BF8DE3B0}"
wchar_t * 0xffff950a'ac2e6f4a
"Status"
wchar_t * 0xffff950a'ad9426da
"ThemePropScrollBarCtl"
wchar_t * 0xffff950a'b3edf5ba
"Edit"
wchar_t * 0xffff950a'ab02e32a
"System"
wchar_t * 0xffff950a'b3e6c53a
"MDIClient"
wchar_t * 0xffff950a'ac17a6ca
"StdDocumentName"
wchar_t * 0xffff950a'ac6cbeea
"StdExit"
wchar_t * 0xffff950a'b033c70a
"{C56C5799-4BB3-7FAE-7FAD-4DB2F6A53EFF}"
wchar_t * 0xffff950a'ab0360fa
"MicrosoftTabletPenServiceProperty"
wchar_t * 0xffff950a'ac2f8fea
"OLEsystem"
```

AppContainer capabilities

As you've just seen, UWP applications have very restricted access rights. So how, for example, is the Microsoft Edge application able to parse the local file system and open PDF files in the user's Documents folder? Similarly, how can the Music application play MP3 files from the Music directory? Whether done directly through kernel access checks or by brokers (which you'll see in the next section), the key lies in capability SIDs. Let's see where these come from, how they are created, and when they are used.

First, UWP developers begin by creating an application manifest that specifies many details of their application, such as the package name, logo, resources, supported devices, and more. One of the key elements for capability management is the list of capabilities in the manifest. For example, let's take a look at Cortana's application manifest, located in %SystemRoot%\SystemApps\Microsoft.Windows.Cortana_cw5n1h2txywey\AppxManifest.xml:

[Click here to view code image](#)

```
<Capabilities>
  <wincap:Capability Name="packageContents"/>
  <!-- Needed for resolving MRT strings -->
  <wincap:Capability Name="cortanaSettings"/>
  <wincap:Capability Name="cloudStore"/>
  <wincap:Capability Name="visualElementsSystem"/>
  <wincap:Capability Name="perceptionSystem"/>
  <Capability Name="internetClient"/>
  <Capability Name="internetClientServer"/>
  <Capability Name="privateNetworkClientServer"/>
  <uap:Capability Name="enterpriseAuthentication"/>
  <uap:Capability Name="musicLibrary"/>
  <uap:Capability Name="phoneCall"/>
  <uap:Capability Name="picturesLibrary"/>
  <uap:Capability Name="sharedUserCertificates"/>
  <rescap:Capability Name="locationHistory"/>
  <rescap:Capability Name="userDataSystem"/>
  <rescap:Capability Name="contactsSystem"/>
  <rescap:Capability Name="phoneCallHistorySystem"/>
  <rescap:Capability Name="appointmentsSystem"/>
  <rescap:Capability Name="chatSystem"/>
  <rescap:Capability Name="smsSend"/>
  <rescap:Capability Name="emailSystem"/>
  <rescap:Capability Name="packageQuery"/>
  <rescap:Capability Name="slapiQueryLicenseValue"/>
  <rescap:Capability Name="secondaryAuthenticationFactor"/>
  <DeviceCapability Name="microphone"/>
  <DeviceCapability Name="location"/>
  <DeviceCapability Name="wiFiControl"/>
</Capabilities>
```

You'll see many types of entries in this list. For example, the `Capability` entries contain the well-known SIDs associated with the original capability set that was implemented in Windows 8. These begin with `SECURITY_CAPABILITY_`—for example, `SECURITY_CAPABILITY_INTERNET_CLIENT`, which is part of the capability

RID under the `APPLICATION PACKAGE AUTHORITY`. This gives us a SID of S-1-15-3-1 in string format.

Other entries are prefixed with `uap`, `rescap`, and `wincap`. One of these (`rescap`) refers to restricted capabilities. These are capabilities that require special onboarding from Microsoft and custom approvals before being allowed on the store. In Cortana's case, these include capabilities such as accessing SMS text messages, emails, contacts, location, and user data. Windows capabilities, on the other hand, refer to capabilities that are reserved for Windows and system applications. No store application can use these. Finally, UAP capabilities refer to standard capabilities that anyone can request on the store. (Recall that UAP is the older name for UWP.)

Unlike the first set of capabilities, which map to hard-coded RIDs, these capabilities are implemented in a different fashion. This ensures a list of well-known RIDs doesn't have to be constantly maintained. Instead, with this mode, capabilities can be fully custom and updated on the fly. To do this, they simply take the capability string, convert it to full upper-case format, and take a SHA-2 hash of the resulting string, much like AppContainer package SIDs are the SHA-2 hash of the package moniker. Again, since SHA-2 hashes are 32 bytes, this results in 8 RIDs for each capability, following the well-known `SECURITY_CAPABILITY_BASE_RID (3)`.

Finally, you'll notice a few `DeviceCapability` entries. These refer to device classes that the UWP application will need to access, and can be identified either through well-known strings such as the ones you see above or directly by a GUID that identifies the device class. Rather than using one of the two methods of SID creation already described, this one uses yet a third! For these types of capabilities, the GUID is converted into a binary format and then mapped out into four RIDs (because a GUID is 16 bytes). On the other hand, if a well-known name was specified instead, it must first be converted to a GUID. This is done by looking at the `HKLM\Software\Microsoft\Windows\CurrentVersion\DeviceAccess\CapabilityMappings` registry key, which contains a list of registry keys associated with device capabilities and a list of GUIDs that map to these capabilities. The GUIDs are then converted to a SID as you've just seen.



For an up-to-date list of supported capabilities, see <https://msdn.microsoft.com/en-us/windows/uwp/packaging/app-capability-declarations>.

As part of encoding all of these capabilities into the token, two additional rules are applied:

- As you may have seen in the earlier experiment, each AppContainer token contains its own package SID encoded as a capability. This can be used by the capability system to specifically lock down access to a particular app through a common security check instead of obtaining and validating the package SID separately.
- Each capability is re-encoded as a group SID through the use of the `SECURITY_CAPABILITY_APP_RID (1024)` RID as an additional sub-authority preceding the regular eight-capability hash RIDs.

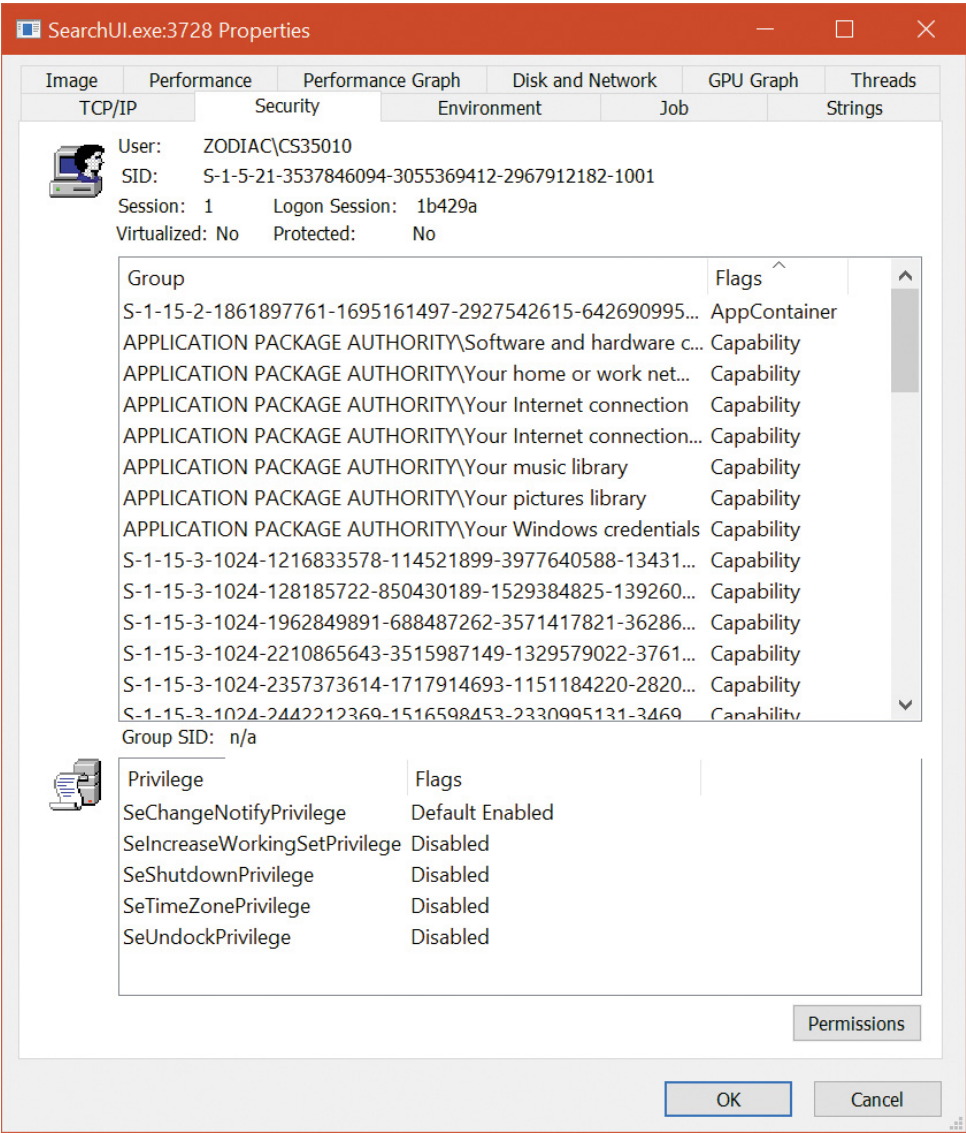
After the capabilities are encoded into the token, various components of the system will read them to determine whether an operation being performed by an AppContainer should be permitted. You'll note most of the APIs are undocumented, as communication and interoperability with UWP applications is not officially supported and best left to broker services, inbox drivers, or kernel components. For example, the kernel and drivers can use the `RtlCapabilityCheck` API to authenticate access to certain hardware interfaces or APIs.

As an example, the Power Manager checks for the `ID_CAP_SCREENOFF` capability before allowing a request to shut off the screen from an AppContainer. The Bluetooth port driver checks for the `bluetoothDiagnostics` capability, while the application identity driver checks for Enterprise Data Protection (EDP) support through the `enterpriseDataPolicy` capability. In user mode, the documented `CheckTokenCapability` API can be used, although it must know the capability SID instead of providing the name (the undocumented `RtlDeriveCapabilitySidFromName` can generate this, however). Another option is the undocumented `CapabilityCheck` API, which does accept a string.

Finally, many RPC services leverage the `RpcClientCapabilityCheck` API, which is a helper function that takes care of retrieving the token and requires only the capability string. This function is very commonly used by many of the WinRT-enlightened services and brokers, which utilize RPC to communicate with UWP client applications.

EXPERIMENT: VIEWING APPCONTAINER CAPABILITIES

To clearly demonstrate all these various capability combinations and their population in the token, let’s look at the capabilities for a complex app such as Cortana. You’ve already seen its manifest, so you can use that output to compare with the UI. First, looking at the Security tab for SearchUI.exe shows the following (sorted by the Flags column):



Clearly, Cortana has obtained many capabilities—all the ones in its manifest. Some are those that were originally in Windows 8 and are known to functions like `IsWellKnownSid`, for which Process Explorer shows a friendly name. Other capabilities are just shown using their SID, as they represent either hashes or GUIDs, as discussed.

To get the details of the package from which the UWP process was created, you can use the `UWPList` tool provided with the downloadable resources for this book. It can show all immersive processes on the system or a single process based on its ID:

[Click here to view code image](#)

C:\WindowsInternals>UwpList.exe 3728

List UWP Processes - version 1.1 (C)2016 by Pavel Yosifovich

Building capabilities map... done.

Process ID: 3728

Image name:

C:\Windows\SystemApps\Microsoft.Windows.Cortana_cw5n1h2txyewy\SearchUI.exe

Package name: Microsoft.Windows.Cortana

Publisher: CN=Microsoft Windows, O=Microsoft Corporation,

L=Redmond, S=Washington, C=US

Published ID: cw5n1h2txyewy

Architecture: Neutral

Version: 1.7.0.14393

AppContainer SID: S-1-15-2-1861897761-1695161497-2927542615-

642690995-327840285-

2659745135-2630312742

Lowbox Number: 3

Capabilities: 32

cortanaSettings (S-1-15-3-1024-1216833578-114521899-3977640588-

1343180512-

2505059295-473916851-3379430393-3088591068) (ENABLED)

visualElementsSystem (S-1-15-3-1024-3299255270-1847605585-

2201808924-710406709-

3613095291-873286183-3101090833-2655911836) (ENABLED)

perceptionSystem (S-1-15-3-1024-34359262-2669769421-2130994847-

3068338639-

3284271446-2009814230-2411358368-814686995) (ENABLED)

internetClient (S-1-15-3-1) (ENABLED)

internetClientServer (S-1-15-3-2) (ENABLED)

privateNetworkClientServer (S-1-15-3-3) (ENABLED)

enterpriseAuthentication (S-1-15-3-8) (ENABLED)

musicLibrary (S-1-15-3-6) (ENABLED)

phoneCall (S-1-15-3-1024-383293015-3350740429-1839969850-1819881064-

1569454686-

4198502490-78857879-1413643331) (ENABLED)

picturesLibrary (S-1-15-3-4) (ENABLED)

sharedUserCertificates (S-1-15-3-9) (ENABLED)

locationHistory (S-1-15-3-1024-3029335854-3332959268-2610968494-

1944663922-
1108717379-267808753-1292335239-2860040626) (ENABLED)
userDataSystem (S-1-15-3-1024-3324773698-3647103388-1207114580-
2173246572-
4287945184-2279574858-157813651-603457015) (ENABLED)
contactsSystem (S-1-15-3-1024-2897291008-3029319760-3330334796-
465641623-3782203132-
742823505-3649274736-3650177846) (ENABLED)
phoneCallHistorySystem (S-1-15-3-1024-2442212369-1516598453-
2330995131-3469896071-
605735848-2536580394-3691267241-2105387825) (ENABLED)
appointmentsSystem (S-1-15-3-1024-2643354558-482754284-283940418-
2629559125-
2595130947-547758827-818480453-1102480765) (ENABLED)
chatSystem (S-1-15-3-1024-2210865643-3515987149-1329579022-
3761842879-3142652231-
371911945-4180581417-4284864962) (ENABLED)
smsSend (S-1-15-3-1024-128185722-850430189-1529384825-139260854-
329499951-
1660931883-3499805589-3019957964) (ENABLED)
emailSystem (S-1-15-3-1024-2357373614-1717914693-1151184220-
2820539834-3900626439-
4045196508-2174624583-3459390060) (ENABLED)
packageQuery (S-1-15-3-1024-1962849891-688487262-3571417821-
3628679630-802580238-
1922556387-206211640-3335523193) (ENABLED)
slapiQueryLicenseValue (S-1-15-3-1024-3578703928-3742718786-7859573-
1930844942-
2949799617-2910175080-1780299064-4145191454) (ENABLED)
S-1-15-3-1861897761-1695161497-2927542615-642690995-327840285-
2659745135-2630312742
(ENABLED)
S-1-15-3-787448254-1207972858-3558633622-1059886964 (ENABLED)
S-1-15-3-3215430884-1339816292-89257616-1145831019 (ENABLED)
S-1-15-3-3071617654-1314403908-1117750160-3581451107 (ENABLED)
S-1-15-3-593192589-1214558892-284007604-3553228420 (ENABLED)
S-1-15-3-3870101518-1154309966-1696731070-4111764952 (ENABLED)
S-1-15-3-2105443330-1210154068-4021178019-2481794518 (ENABLED)
S-1-15-3-2345035983-1170044712-735049875-2883010875 (ENABLED)
S-1-15-3-3633849274-1266774400-1199443125-2736873758 (ENABLED)
S-1-15-3-2569730672-1095266119-53537203-1209375796 (ENABLED)
S-1-15-3-2452736844-1257488215-2818397580-3305426111 (ENABLED)

The output shows the package full name, executable directory, AppContainer SID, publisher information, version, and list of capabilities. Also shown is the LowBox number, which is just a local index of the app.

Lastly, you can inspect these properties in the kernel debugger with the `!token` command.

Some UWP apps are called *trusted*, and although they use the Windows Runtime platform like other UWP apps, they do not run inside an AppContainer, and have an integrity level higher than Low. The canonical example is the System Settings app (`%SystemRoot%\ImmersiveControlPanel\SystemSettings.exe`); this seems reasonable, as the Settings app must be able to make changes to the system that would be impossible to do from an AppContainer-hosted process. If you look at its token, you will see the same three attributes—`PKG`, `SYSAPPID`, and `PKGHOSTID`—which confirm that it's still a packaged application, even without the AppContainer token present.

AppContainer and object namespace

Desktop applications can easily share kernel objects by name. For example, suppose process A creates an event object by calling `CreateEvent(Ex)` with the name `MyEvent`. It gets back a handle it can later use to manipulate the event. Process B running in the same session can call `CreateEvent(Ex)` or `OpenEvent` with the same name, `MyEvent`, and (assuming it has appropriate permissions, which is usually the case if running under the same session) get back another handle to the same underlying event object. Now if process A calls `SetEvent` on the event object while process B was blocked in a call to `WaitForSingleObject` on its event handle, process B's waiting thread would be released because it's the same event object. This sharing works because named objects are created in the object manager directory `\Sessions\x\BaseNamedObjects`, as shown in [Figure 7-18](#) with the WinObj Sysinternals tool.

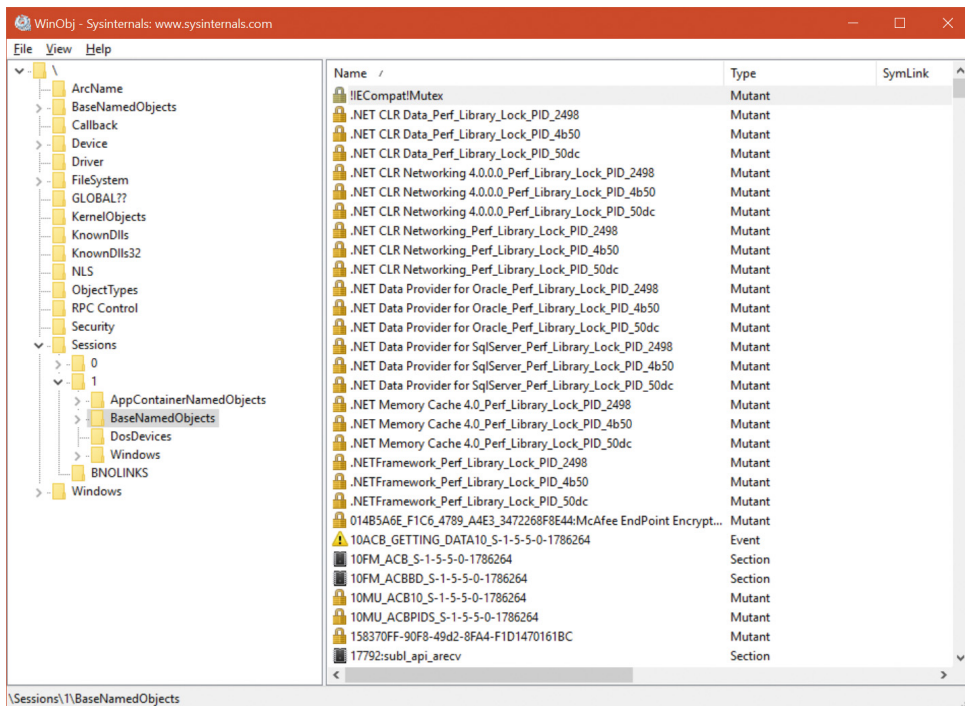


FIGURE 7-18 Object manager directory for named objects.

Furthermore, desktop apps can share objects between sessions by using a name prefixed with Global\. This creates the object in the session 0 object directory located in \BaseNamedObjects (refer to [Figure 7-18](#)).

AppContainer-based processes have their root object namespace under \Sessions\x\AppContainerNamedObjects\<AppContainerSID>. Since every AppContainer has a different AppContainer SID, there is no way two UWP apps can share kernel objects. The ability to create a named kernel object in the session 0 object namespace is not allowed for AppContainer processes. [Figure 7-19](#) shows the object manager's directory for the Windows UWP Calculator app.

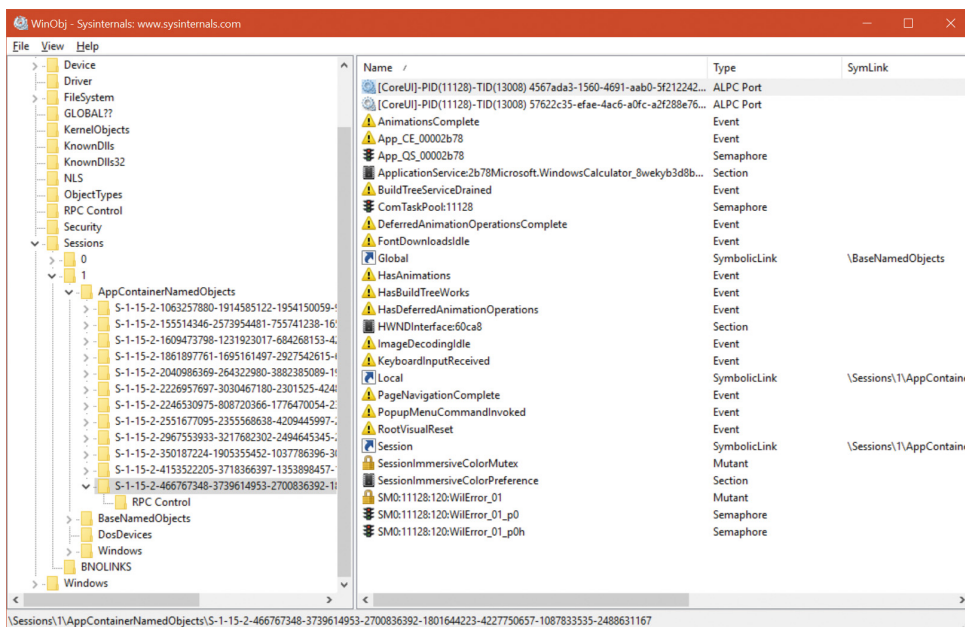


FIGURE 7-19 Object manager directory for Calculator.

UWP apps that want to share data can do so using well-defined contracts, managed by the Windows Runtime. (See the MSDN documentation for more information.)

Sharing kernel objects between desktop apps and UWP apps is possible, and often done by broker services. For example, when requesting access to a file in the Documents folder (and getting the right capability validated) from the file picker broker, the UWP app will receive a file handle that it can use for reads and writes directly, without the cost of marshalling requests back and forth. This is achieved by having the broker duplicate the file handle it obtained directly in the handle table of the UWP application. (More information on handle duplication appears in Chapter 8 in Part 2.) To simplify things even further, the ALPC subsystem (also described in Chapter 8) allows the automatic transfer of handles in this way through ALPC handle attributes. and the Remote Procedure Call (RPC) services that use ALPC as their underlying protocol can use this functionality as part of their interfaces. Marshallable handles in the IDL file will automatically be transferred in this way through the ALPC subsystem.

Outside of official broker RPC services, a desktop app can create a named (or even unnamed) object normally, and then use the `DuplicateHandle` function to inject a handle to the same object into the UWP process manually. This works because desktop apps typically run with medium integrity level and there's nothing preventing them from duplicating handles into UWP processes—only the other way around.



Communication between a desktop app and a UWP is not usually required because a store app cannot have a desktop app companion, and cannot rely on such an app to exist on the device. The capability to inject handles into a UWP app may be needed in specialized cases such as using the desktop bridge (Centennial) to convert a desktop app to a UWP app and communicate with another desktop app that is known to exist.

AppContainer handles

In a typical Win32 application, the presence of the session-local and global BaseNamedObjects directory is guaranteed by the Windows subsystem, as it creates this on boot and session creation. Unfortunately, the AppContainerBaseNamedObjects directory is actually created by the launch application itself. In the case of UWP activation, this is the trusted `DComLaunch` service, but recall that not all AppContainers are necessarily tied to UWP. They can also be manually created through the right process-creation attributes. (See [Chapter 3](#) for more information on which ones to use.) In this case, it's possible for an untrusted application to have created the object directory (and required symbolic links within it), which would result in the ability for this application to close the handles from underneath the AppContainer application. Even without malicious intent, the original launching application might exit, cleaning up its handles and destroying the AppContainer-specific object directory. To avoid this situation, AppContainer tokens have the ability to store an array of handles that are guaranteed to exist throughout the lifetime of any application using the token. These handles are initially passed in when the AppContainer token is being created (through `NtCreateLowBoxToken`) and are duplicated as kernel handles.

Similar to the per-AppContainer atom table, a special `SEP_CACHED_HANDLES_ENTRY` structure is used, this time based on a hash table that's stored in the logon session structure for this user. (See the [“Logon”](#) section later in this chapter for more information on logon sessions.) This structure contains an array of kernel handles that have been duplicated during the creation of the AppContainer token. They will be closed either when this token is destroyed (because the application is exit-

ing) or when the user logs off (which will result in tearing down the logon session).

To view token stored handles, follow these steps:

1. Run Calculator and launch local kernel debugging.
2. Search for the calculator process:

[Click here to view code image](#)

```
lkd> !process 0 1 calculator.exe
PROCESS ffff828cc9ed1080
  SessionId: 1 Cid: 4bd8 Peb: d040bbc000 ParentCid: 03a4
  DeepFreeze
  DirBase: 5fccaa000 ObjectTable: ffff950ad9fa2800 HandleCount:
<Data Not Accessible>
  Image: Calculator.exe
  VadRoot ffff828cd2c9b6a0 Vads 168 Clone 0 Private 2938. Modified
3332.
  Locked 0.
  DeviceMap ffff950aad2cd2f0
  Token                ffff950adb313060
  ElapsedTime           1 Day 08:01:47.018
  UserTime               00:00:00.015
  KernelTime             00:00:00.031
  QuotaPoolUsage[PagedPool] 465880
  QuotaPoolUsage[NonPagedPool] 23288
  Working Set Sizes (now,min,max) (7434, 50, 345) (29736KB, 200KB,
1380KB)
  PeakWorkingSetSize      11097
  VirtualSize             303 Mb
  PeakVirtualSize         314 Mb
  PageFaultCount          21281
  MemoryPriority           BACKGROUND
  BasePriority             8
  CommitCharge            4925
  Job                     ffff828cd4914060
```

3. Dump the token using the `dt` command. (Remember to mask the lower 3 or 4 bits if they are not zero.)

[Click here to view code image](#)

```
lkd> dt nt!_token ffff950adb313060
+0x000 TokenSource      : _TOKEN_SOURCE
+0x010 TokenId          : _LUID
+0x018 AuthenticationId : _LUID
+0x020 ParentTokenId    : _LUID
...
+0x0c8 TokenFlags       : 0x4a00
+0x0cc TokenInUse       : 0x1 "
+0x0d0 IntegrityLevelIndex : 1
+0x0d4 MandatoryPolicy  : 1
+0x0d8 LogonSession     : 0xffff950a'b4bb35c0
_SEP_LOGON_SESSION_REFERENCES
+0x0e0 OriginatingLogonSession : _LUID
+0x0e8 SidHash          : _SID_AND_ATTRIBUTES_HASH
+0x1f8 RestrictedSidHash : _SID_AND_ATTRIBUTES_HASH
+0x308 pSecurityAttributes : 0xffff950a'e4ff57f0
_AUTHZBASEP_SECURITY_ATTRIBUTES_INFORMATION
+0x310 Package         : 0xffff950a'e00ed6d0 Void
+0x318 Capabilities    : 0xffff950a'e8e8fbc0 _SID_AND_ATTRIBUTES
+0x320 CapabilityCount : 1
+0x328 CapabilitiesHash : _SID_AND_ATTRIBUTES_HASH
+0x438 LowboxNumberEntry : 0xffff950a'b3fd55d0
_SEP_LOWBOX_NUMBER_ENTRY
+0x440 LowboxHandlesEntry : 0xffff950a'e6ff91d0
_SEP_LOWBOX_HANDLES_ENTRY
+0x448 pClaimAttributes : (null)
...
```

4. Dump the `LowboxHandlesEntry` member:

[Click here to view code image](#)

```
lkd> dt nt!_sep_lowbox_handles_entry 0xffff950a'e6ff91d0
+0x000 HashEntry        : _RTL_DYNAMIC_HASH_TABLE_ENTRY
+0x018 ReferenceCount    : 0n10
+0x020 PackageSid       : 0xffff950a'e6ff9208 Void
+0x028 HandleCount      : 6
+0x030 Handles          : 0xffff950a'e91d8490 ->
0xffffffff'800023cc Void
```

5. There are six handles. Let's dump their values:

[Click here to view code image](#)

```
lkd> dq 0xffff950ae91d8490 L6
ffff950a'e91d8490 ffffffff'800023cc ffffffff'80001e80
ffff950a'e91d84a0 ffffffff'80004214 ffffffff'8000425c
ffff950a'e91d84b0 ffffffff'800028c8 ffffffff'80001834
```

6. You can see that these handles are kernel handles—that is, handle values starting with `0xffffffff` (64 bit). Now you can use the `!handle` command to look at individual handles. Here are two examples from the six handles above:

[Click here to view code image](#)

```
lkd> !handle ffffffff'80001e80
```

```
PROCESS ffff828cd71b3600
```

```
  SessionId: 1 Cid: 27c4 Peb: 3fdbf2f000 ParentCid: 2324
```

```
  DirBase: 80bb85000 ObjectTable: ffff950addabf7c0 HandleCount:
```

```
<Data Not Accessible>
```

```
  Image: windbg.exe
```

Kernel handle Error reading handle count.

```
80001e80: Object: ffff950ada206ea0 GrantedAccess: 0000000f (Protected)
```

```
(Inherit) (Audit) Entry: ffff950ab5406a00
```

```
Object: ffff950ada206ea0 Type: (ffff828cb66b33b0) Directory
```

```
  ObjectHeader: ffff950ada206e70 (new version)
```

```
  HandleCount: 1 PointerCount: 32770
```

```
  Directory Object: ffff950ad9a62950 Name: RPC Control
```

Hash	Address	Type	Name
------	---------	------	------

----	-----	----	----
------	-------	------	------

23	ffff828cb6ce6950	ALPC Port	
----	------------------	-----------	--

```
OLE376512B99BCCA5DE4208534E7732
```

```
lkd> !handle ffffffff'800028c8
```

```
PROCESS ffff828cd71b3600
```

```
  SessionId: 1 Cid: 27c4 Peb: 3fdbf2f000 ParentCid: 2324
```

```
  DirBase: 80bb85000 ObjectTable: ffff950addabf7c0 HandleCount:
```

```
<Data
```

Not Accessible>

Image: windbg.exe

Kernel handle Error reading handle count.

800028c8: Object: ffff950ae7a8fa70 GrantedAccess: 000f0001 (Audit)

Entry:

ffff950acc426320

Object: ffff950ae7a8fa70 Type: (ffff828cb66296f0) SymbolicLink

ObjectHeader: ffff950ae7a8fa40 (new version)

HandleCount: 1 PointerCount: 32769

Directory Object: ffff950ad9a62950 Name: Session

Flags: 00000000 (Local)

Target String is '\Sessions\1\AppDataContainerNamedObjects
\S-1-15-2-466767348-3739614953-2700836392-1801644223-4227750657
-1087833535-2488631167'

Finally, because the ability to restrict named objects to a particular object directory namespace is a valuable security tool for sandboxing named object access, the upcoming (at the time of this writing) Windows 10 Creators Update includes an additional token capability called *BNO isolation* (where *BNO* refers to `BaseNamedObjects`). Using the same `SEP_CACHE_HANDLES_ENTRY` structure, a new field, `BnoIsolationHandlesEntry`, is added to the `TOKEN` structure, with the type set to `SepCachedHandlesEntryBnoIsolation` instead of `SepCachedHandlesEntryLowbox`. To use this feature, a special process attribute must be used (see [Chapter 3](#) for more information), which contains an isolation prefix and a list of handles. At this point, the same `LowBox` mechanism is used, but instead of an `AppContainer` SID object directory, a directory with the prefix indicated in the attribute is used.

Brokers

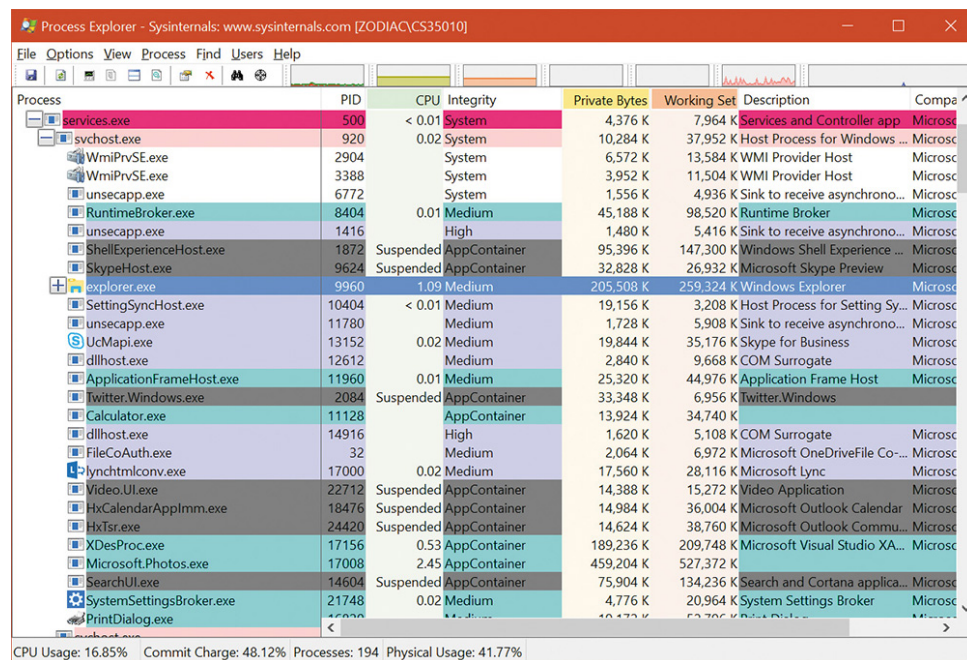
Because AppContainer processes have almost no permissions except for those implicitly granted with capabilities, some common operations cannot be performed directly by the AppContainer and require help. (There are no capabilities for these, as these are too low level to be visible to users in the store, and difficult to manage.) Some examples include selecting files using the common File Open dialog box or printing with a Print dialog box. For these and other similar operations, Windows provides helper processes, called *brokers*, managed by the system broker process, RuntimeBroker.exe.

An AppContainer process that requires any of these services communicates with the Runtime Broker through a secure ALPC channel and Runtime Broker initiates the creation of the requested broker process. Examples are %SystemRoot%\PrintDialog\PrintDialog.exe and %SystemRoot%\System32\PickerHost.exe.

EXPERIMENT: BROKERS

The following steps show how broker processes are launched and terminated:

1. Click the **Start** button, type **Photos**, and select the **Photos** option to run the built-in Windows 10 Photos application.
2. Open Process Explorer, switch the process list to a tree view, and locate the Microsoft.Photos.exe process. Place both windows side by side.
3. In the Photos app, select a picture file, and click **Print** in the top ellipsis menu or right-click the picture and choose **Print** from the menu that appears. The Print dialog box should open, and Process Explorer should show the newly created broker (PrintDialog.exe). Notice they are all children of the same Svchost process. (All UWP processes are launched by the DCOMLaunch service hosted inside that process.)



4. Close the Print dialog box. The PrintDialog.exe process should exit.

Logon

Interactive logon (as opposed to network logon) occurs through the interaction of the following:

- The logon process (Winlogon.exe)
- The logon user interface process (LogonUI.exe) and its credential providers

- Lsass.exe

- One or more authentication packages

- SAM or Active Directory

Authentication packages are DLLs that perform authentication checks. Kerberos is the Windows authentication package for interactive logon to a domain. MSV1_0 is the Windows authentication package for interactive logon to a local computer, for domain logons to trusted pre-Windows 2000 domains, and for times when no domain controller is accessible.

Winlogon is a trusted process responsible for managing security-related user interactions. It coordinates logon, starts the user's first process at logon, and handles logoff. It also manages various other operations relevant to security, including launching LogonUI for entering passwords at logon, changing passwords, and locking and unlocking the workstation. The Winlogon process must ensure that operations relevant to security aren't visible to any other active processes. For example, Winlogon guarantees that an untrusted process can't get control of the desktop during one of these operations and thus gain access to the password.

Winlogon relies on the credential providers installed on the system to obtain a user's account name or password. Credential providers are COM objects located inside DLLs. The default providers are `authui.dll`, `SmartcardCredentialProvider.dll`, and `FaceCredentialProvider.dll`, which support password, smartcard PIN, and face-recognition authentication, respectively. Allowing other credential providers to be installed enables Windows to use different user-identification mechanisms. For example, a third party might supply a credential provider that uses a thumbprint-recognition device to identify users and extract their passwords from an encrypted database. Credential providers are listed in `HKLM\SOFTWARE\Microsoft\Windows\CurrentVersion\Authentication\Credential Providers`, where each subkey identifies a credential provider class by its COM CLSID. (The CLSID itself must be registered at `HKCR\CLSID` like any other COM class.) You can use the `Clist.exe` tool provided with the downloadable resources for this book to list the credential providers with their CLSID, friendly name, and implementation DLL.

To protect Winlogon's address space from bugs in credential providers that might cause the Winlogon process to crash (which, in turn, will result in a system crash, because Winlogon is considered a critical system

process), a separate process, LogonUI.exe, is used to actually load the credential providers and display the Windows logon interface to users. This process is started on demand whenever Winlogon needs to present a user interface to the user, and it exits after the action has finished. It also allows Winlogon to simply restart a new LogonUI process should it crash for any reason.

Winlogon is the only process that intercepts logon requests from the keyboard. These are sent through an RPC message from Win32k.sys. Winlogon immediately launches the LogonUI application to display the user interface for logon. After obtaining a user name and password from credential providers, Winlogon calls Lsass to authenticate the user attempting to log on. If the user is authenticated, the logon process activates a logon shell on behalf of that user. The interaction between the components involved in logon is illustrated in **Figure 7-20**.

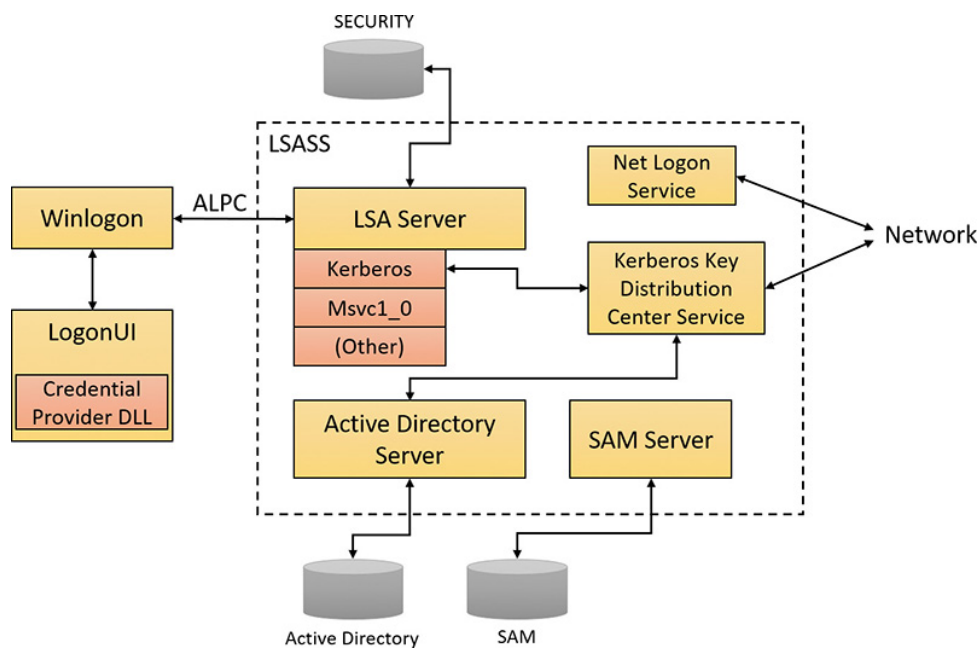


FIGURE 7-20 Components involved in logon.

In addition to supporting alternative credential providers, LogonUI can load additional network provider DLLs that need to perform secondary authentication. This capability allows multiple network providers to gather identification and authentication information all at one time during normal logon. A user logging on to a Windows system might simultaneously be authenticated on a Linux server. That user would then be able to access resources of the UNIX server from the Windows machine without requiring additional authentication. Such a capability is known as one form of single sign-on.

Winlogon initialization

During system initialization, before any user applications are active, Winlogon performs the following steps to ensure that it controls the workstation once the system is ready for user interaction:

1. It creates and opens an interactive window station (for example, `\Sessions\1\Windows\WindowStations\WinSta0` in the object manager namespace) to represent the keyboard, mouse, and monitor. Winlogon creates a security descriptor for the station that has one and only one ACE containing only the system SID. This unique security descriptor ensures that no other process can access the workstation unless explicitly allowed by Winlogon.
2. It creates and opens two desktops: an application desktop (`\Sessions\1\Windows\WinSta0\Default`, also known as the interactive desktop) and a Winlogon desktop (`\Sessions\1\Windows\WinSta0\Winlogon`, also known as the Secure Desktop). The security on the Winlogon desktop is created so that only Winlogon can access that desktop. The other desktop allows both Winlogon and users to access it. This arrangement means that any time the Winlogon desktop is active, no other process has access to any active code or data associated with the desktop. Windows uses this feature to protect the secure operations that involve passwords and locking and unlocking the desktop.
3. Before anyone logs on to a computer, the visible desktop is Winlogon's. After a user logs on, pressing the SAS sequence (by default, Ctrl+Alt+Del) switches the desktop from Default to Winlogon and launches LogonUI. (This explains why all the windows on your interactive desktop seem to disappear when you press Ctrl+Alt+Del, and then return when you dismiss the Windows Security dialog box.) Thus, the SAS always brings up a Secure Desktop controlled by Winlogon.
4. It establishes an ALPC connection with Lsass. This connection will be used for exchanging information during logon, logoff, and password operations, and is made by calling `LsaRegisterLogonProcess`.
5. It registers the Winlogon RPC message server, which listens for SAS, logoff, and workstation lock notifications from Win32k. This measure prevents Trojan horse programs from gaining control of the screen when the SAS is entered.



The Wininit process performs steps similar to steps 1 and 2 to allow legacy interactive services running on session 0 to display windows, but it does not perform any other steps because session 0 is not available for user logon.

HOW SAS IS IMPLEMENTED

The SAS is secure because no application can intercept the Ctrl+Alt+Del keystroke combination or prevent Winlogon from receiving it. Win32k.sys reserves the Ctrl+Alt+Del key combination so that whenever the Windows input system (implemented in the raw input thread in Win32k) sees the combination, it sends an RPC message to Winlogon's message server, which listens for such notifications. The keystrokes that map to a registered hot key are not sent to any process other than the one that registered it, and only the thread that registered a hot key can unregister it, so a Trojan horse application cannot deregister Winlogon's ownership of the SAS.

A Windows function, `SetWindowsHookEx`, enables an application to install a hook procedure that's invoked every time a keystroke is pressed, even before hot keys are processed, and allows the hook to squash keystrokes. However, the Windows hot key processing code contains a special case for Ctrl+Alt+Del that disables hooks so that the keystroke sequence can't be intercepted. In addition, if the interactive desktop is locked, only hot keys owned by Winlogon are processed.

Once the Winlogon desktop is created during initialization, it becomes the active desktop. When the Winlogon desktop is active, it is always locked. Winlogon unlocks its desktop only to switch to the application desktop or the screen-saver desktop. (Only the Winlogon process can lock or unlock a desktop.)

User logon steps

Logon begins when a user presses the SAS (Ctrl+Alt+Del). After the SAS is pressed, Winlogon starts LogonUI, which calls the credential providers to obtain a user name and password. Winlogon also creates a unique local logon SID for this user, which it assigns to this instance of the desktop

(keyboard, screen, and mouse). Winlogon passes this SID to Lsass as part of the `LsaLogonUser` call. If the user is successfully logged on, this SID will be included in the logon process token—a step that protects access to the desktop. For example, another logon to the same account but on a different system will be unable to write to the first machine's desktop because this second logon won't be in the first logon's desktop token.

When the user name and password have been entered, Winlogon retrieves a handle to a package by calling the Lsass function

`LsaLookupAuthenticationPackage`. Authentication packages are listed in the registry under `HKLM\SYSTEM\CurrentControlSet\Control\Lsa`.

Winlogon passes logon information to the authentication package via `LsaLogonUser`. Once a package authenticates a user, Winlogon continues the logon process for that user. If none of the authentication packages indicates a successful logon, the logon process is aborted.

Windows uses two standard authentication packages for interactive username/password-based logons:

■ **MSV1_0** The default authentication package on a stand-alone Windows system is MSV1_0 (`Msv1_0.dll`), an authentication package that implements LAN Manager 2 protocol. Lsass also uses MSV1_0 on domain-member computers to authenticate to pre-Windows 2000 domains and computers that can't locate a domain controller for authentication. (Computers that are disconnected from the network fall into this latter category.)

■ **Kerberos** The Kerberos authentication package, `Kerberos.dll`, is used on computers that are members of Windows domains. The Windows Kerberos package, with the cooperation of Kerberos services running on a domain controller, supports the Kerberos protocol. This protocol is based on Internet RFC 1510. (Visit the Internet Engineering Task Force [IETF] website at <http://www.ietf.org> for detailed information on the Kerberos standard.)

MSV1_0

The MSV1_0 authentication package takes the user name and a hashed version of the password and sends a request to the local SAM to retrieve the account information, which includes the hashed password, the groups to which the user belongs, and any account restrictions. MSV1_0 first checks the account restrictions, such as hours or type of accesses allowed. If the user can't log on because of the restrictions in the SAM database, the logon call fails and MSV1_0 returns a failure status to the LSA.

MSV1_0 then compares the hashed password and user name to that obtained from the SAM. In the case of a cached domain logon, MSV1_0 accesses the cached information by using Lsass functions that store and retrieve “secrets” from the LSA database (the SECURITY hive of the registry). If the information matches, MSV1_0 generates a LUID for the logon session and creates the logon session by calling Lsass, associating this unique identifier with the session and passing the information needed to ultimately create an access token for the user. (Recall that an access token includes the user's SID, group SIDs, and assigned privileges.)



NOTE

MSV1_0 does not cache a user's entire password hash in the registry because that would enable someone with physical access to the system to easily compromise a user's domain account and gain access to encrypted files and to network resources the user is authorized to access. Instead, it caches half of the hash. The cached half-hash is sufficient to verify that a user's password is correct, but it isn't sufficient to gain access to EFS keys and to authenticate as the user on a domain because these actions require the full hash.

If MSV1_0 needs to authenticate using a remote system, as when a user logs on to a trusted pre-Windows 2000 domain, MSV1_0 uses the Netlogon service to communicate with an instance of Netlogon on the remote system. Netlogon on the remote system interacts with the MSV1_0 authentication package on that system, passing back authentication results to the system on which the logon is being performed.

Kerberos

The basic control flow for Kerberos authentication is the same as the flow for MSV1_0. However, in most cases, domain logons are performed from member workstations or servers rather than on a domain controller, so the authentication package must communicate across the network as part of the authentication process. The package does so by communicating via the Kerberos TCP/IP port (port 88) with the Kerberos service on a domain controller. The Kerberos Key Distribution Center service (Kdcsvc.dll), which implements the Kerberos authentication protocol, runs in the Lsass process on domain controllers.

After validating hashed user-name and password information with Active Directory's user account objects (using the Active Directory server Ntdsa.dll), Kdcsvc returns domain credentials to Lsass, which returns the result of the authentication and the user's domain logon credentials (if the logon was successful) across the network to the system where the logon is taking place.



This description of Kerberos authentication is highly simplified, but it highlights the roles of the various components involved. Although the Kerberos authentication protocol plays a key role in distributed domain security in Windows, its details are outside the scope of this book.

After a logon has been authenticated, Lsass looks in the local policy database for the user's allowed access, including interactive, network, batch, or service process. If the requested logon doesn't match the allowed access, the logon attempt will be terminated. Lsass deletes the newly created logon session by cleaning up any of its data structures and then returns failure to Winlogon, which in turn displays an appropriate message to the user. If the requested access is allowed, Lsass adds the appropriate additional security IDs (such as Everyone, Interactive, and the like). It then checks its policy database for any granted privileges for all the SIDs for this user and adds these privileges to the user's access token.

When Lsass has accumulated all the necessary information, it calls the executive to create the access token. The executive creates a primary access token for an interactive or service logon and an impersonation token for a network logon. After the access token is successfully created, Lsass

duplicates the token, creating a handle that can be passed to Winlogon, and closes its own handle. If necessary, the logon operation is audited. At this point, Lsass returns success to Winlogon along with a handle to the access token, the LUID for the logon session, and the profile information, if any, that the authentication package returned.

EXPERIMENT: LISTING ACTIVE LOGON SESSIONS

As long as at least one token exists with a given logon session LUID, Windows considers the logon session to be active. You can use the LogonSessions tool from Sysinternals, which uses the `LsaEnumerateLogonSessions` function (documented in the Windows SDK) to list the active logon sessions:

[Click here to view code image](#)

```
C:\WINDOWS\system32>logonsessions
```

LogonSessions v1.4 - Lists logon session information

Copyright (C) 2004-2016 Mark Russinovich

Sysinternals - www.sysinternals.com

[0] Logon session 00000000:000003e7:

User name: WORKGROUP\ZODIAC\$

Auth package: NTLM

Logon type: (none)

Session: 0

Sid: S-1-5-18

Logon time: 09-Dec-16 15:22:31

Logon server:

DNS Domain:

UPN:

[1] Logon session 00000000:0000cdce:

User name:

Auth package: NTLM

Logon type: (none)

Session: 0

Sid: (none)

Logon time: 09-Dec-16 15:22:31

Logon server:

DNS Domain:

UPN:

[2] Logon session 00000000:000003e4:

User name: WORKGROUP\ZODIAC\$

Auth package: Negotiate

Logon type: Service

Session: 0

Sid: S-1-5-20
Logon time: 09-Dec-16 15:22:31
Logon server:
DNS Domain:
UPN:

[3] Logon session 00000000:00016239:

User name: Window Manager\DWM-1
Auth package: Negotiate
Logon type: Interactive
Session: 1
Sid: S-1-5-90-0-1
Logon time: 09-Dec-16 15:22:32
Logon server:
DNS Domain:
UPN:

[4] Logon session 00000000:00016265:

User name: Window Manager\DWM-1
Auth package: Negotiate
Logon type: Interactive
Session: 1
Sid: S-1-5-90-0-1
Logon time: 09-Dec-16 15:22:32
Logon server:
DNS Domain:
UPN:

[5] Logon session 00000000:000003e5:

User name: NT AUTHORITY\LOCAL SERVICE
Auth package: Negotiate
Logon type: Service
Session: 0
Sid: S-1-5-19
Logon time: 09-Dec-16 15:22:32
Logon server:
DNS Domain:
UPN:

...

[8] Logon session 00000000:0005c203:

User name: NT VIRTUAL MACHINE\AC9081B6-1E96-4BC8-8B3B-

C609D4F85F7D

Auth package: Negotiate

Logon type: Service

Session: 0

Sid: S-1-5-83-1-2895151542-1271406230-163986315-2103441620

Logon time: 09-Dec-16 15:22:35

Logon server:

DNS Domain:

UPN:

[9] Logon session 00000000:0005d524:

User name: NT VIRTUAL MACHINE\B37F4A3A-21EF-422D-8B37-AB6B0A016ED8

Auth package: Negotiate

Logon type: Service

Session: 0

Sid: S-1-5-83-1-3011463738-1110254063-1806382987-3631087882

Logon time: 09-Dec-16 15:22:35

Logon server:

DNS Domain:

UPN:

...

[12] Logon session 00000000:0429ab2c:

User name: IIS APPPOOL\DefaultAppPool

Auth package: Negotiate

Logon type: Service

Session: 0

Sid: S-1-5-82-3006700770-424185619-1745488364-794895919-4004696415

Logon time: 09-Dec-16 22:33:03

Logon server:

DNS Domain:

UPN:

Information reported for a session includes the SID and name of the user associated with the session, as well as the session's authentication package and logon time. Note that the Negotiate authentication package, seen in logon sessions 2 and 9 in the preceding output, will attempt to authenticate via Kerberos or NTLM, depending on which is most appropriate for the authentication request.

The LUID for a session is displayed on the `Logon Session` line of each session block. Using the `Handle.exe` utility (also from Sysinternals), you can find the tokens that represent a particular logon session. For example, to find the tokens for logon session 8 in the output just shown, you could enter this command:

[Click here to view code image](#)

```
C:\WINDOWS\system32>handle -a 5c203
```

Nthandle v4.1 - Handle viewer

Copyright (C) 1997-2016 Mark Russinovich

Sysinternals - www.sysinternals.com

```
System      pid: 4   type: Directory  1274: \Sessions\0\
DosDevices\00000000-0005c203
lsass.exe   pid: 496 type: Token    D7C: NT VIRTUAL MACHINE\
AC9081B6-1E96-4BC8-8B3B-C609D4F85F7D:5c203
lsass.exe   pid: 496 type: Token    2350: NT VIRTUAL MACHINE\
AC9081B6-1E96-4BC8-8B3B-C609D4F85F7D:5c203
lsass.exe   pid: 496 type: Token    2390: NT VIRTUAL MACHINE\
AC9081B6-1E96-4BC8-8B3B-C609D4F85F7D:5c203
svchost.exe pid: 900 type: Token    804: NT VIRTUAL MACHINE\
AC9081B6-1E96-4BC8-8B3B-C609D4F85F7D:5c203
svchost.exe pid: 1468 type: Token  10EC: NT VIRTUAL MACHINE\
AC9081B6-1E96-4BC8-8B3B-C609D4F85F7D:5c203
vmms.exe    pid: 4380 type: Token    A34: NT VIRTUAL MACHINE\
AC9081B6-1E96-4BC8-8B3B-C609D4F85F7D:5c203
vmcompute.exe pid: 6592 type: Token    200: NT VIRTUAL
MACHINE\
AC9081B6-1E96-4BC8-8B3B-C609D4F85F7D:5c203
vmwp.exe    pid: 7136 type: WindowStation 168:
\Windows\WindowStations\
Service-0x0-5c203$
vmwp.exe    pid: 7136 type: WindowStation 170:
\Windows\WindowStations\
Service-0x0-5c203$
```

Winlogon then looks in the registry at the value
HKLM\SOFTWARE\Microsoft\Windows NT\Current
Version\Winlogon\Userinit and creates a process to run whatever the

value of that string is. (This value can be several EXEs separated by commas.) The default value is Userinit.exe, which loads the user profile and then creates a process to run whatever the value of HKCU\SOFTWARE\Microsoft\Windows NT\CurrentVersion\Winlogon\Shell is, if that value exists. That value does not exist by default, however. If it doesn't exist, Userinit.exe does the same for HKLM\SOFTWARE\Microsoft\Windows NT\CurrentVersion\Winlogon\Shell, which defaults to Explorer.exe. Userinit then exits (which is why Explorer.exe shows up as having no parent when examined in Process Explorer). For more information on the steps followed during the user logon process, see Chapter 11 in Part 2.

Assured authentication

A fundamental problem with password-based authentication is that passwords can be revealed or stolen and used by malicious third parties. Windows includes a mechanism that tracks the authentication strength of how a user authenticated with the system, which allows objects to be protected from access if a user did not authenticate securely. (Smartcard authentication is considered to be a stronger form of authentication than password authentication.)

On systems that are joined to a domain, the domain administrator can specify a mapping between an object identifier (OID) (a unique numeric string representing a specific object type) on a certificate used for authenticating a user (such as on a smartcard or hardware security token) and a SID that is placed into the user's access token when the user successfully authenticates with the system. An ACE in a DACL on an object can specify such a SID be part of a user's token in order for the user to gain access to the object. Technically, this is known as a *group claim*. In other words, the user is claiming membership in a particular group, which is allowed certain access rights on specific objects, with the claim based upon the authentication mechanism. This feature is not enabled by default, and it must be configured by the domain administrator in a domain with certificate-based authentication.

Assured authentication builds on existing Windows security features in a way that provides a great deal of flexibility to IT administrators and anyone concerned with enterprise IT security. The enterprise decides which OIDs to embed in the certificates it uses for authenticating users and the mapping of particular OIDs to Active Directory universal groups (SIDs). A user's group membership can be used to identify whether a certificate

was used during the logon operation. Different certificates can have different issuance policies and, thus, different levels of security, which can be used to protect highly sensitive objects (such as files or anything else that might have a security descriptor).

Authentication protocols (APs) retrieve OIDs from certificates during certificate-based authentication. These OIDs must be mapped to SIDs, which are in turn processed during group membership expansion, and placed in the access token. The mapping of OID to universal group is specified in Active Directory.

As an example, an organization might have several certificate-issuance policies named Contractor, Full Time Employee, and Senior Management, which map to the universal groups Contractor-Users, FTE-Users, and SM-Users, respectively. A user named Abby has a smartcard with a certificate issued using the Senior Management issuance policy. When she logs in using her smartcard, she receives an additional group membership (which is represented by a SID in her access token) indicating that she is a member of the SM-Users group. Permissions can be set on objects (using an ACL) such that only members of the FTE-Users or SM-Users group (identified by their SIDs within an ACE) are granted access. If Abby logs in using her smartcard, she can access those objects, but if she logs in with just her user name and password (without the smartcard), she cannot access those objects because she will not have either the FTE-Users or SM-Users group in her access token. A user named Toby who logs in with a smartcard that has a certificate issued using the Contractor issuance policy would not be able to access an object that has an ACE requiring FTE-Users or SM-Users group membership.

Windows Biometric Framework

Windows provides a standardized mechanism for supporting certain types of biometric devices, such as fingerprint scanners, used to enable user identification via a fingerprint swipe: the Windows Biometric Framework (WBF). Like many other such frameworks, the WBF was developed to isolate the various functions involved in supporting such devices, so as to minimize the code required to implement a new device.

The primary components of the WBF are shown in [Figure 7-21](#). Except as noted in the following list, all of these components are supplied by Windows:

■ The Windows Biometric Service

(%SystemRoot%\System32\Wbiosrv.dll This provides the process-execution environment in which one or more biometric service providers can execute.

■ **The Windows Biometric Driver Interface (WBDI)** This is a set of interface definitions (IRP major function codes, DeviceIoControl codes, and so forth) to which any driver for a biometric scanner device must conform if it is to be compatible with the Windows Biometric Service. WBDI drivers can be developed using any of the standard driver frameworks (UMDF, KMDF and WDM). However, UMDF is recommended to reduce code size and increase reliability. WBDI is described in the Windows Driver Kit documentation.

■ **The Windows Biometric API** This allows existing Windows components such as Winlogon and LogonUI to access the biometric service. Third-party applications have access to the Windows Biometric API and can use the biometric scanner for functions other than logging in to Windows. An example of a function in this API is WinBioEnumServiceProviders . The Biometric API is exposed by %SystemRoot%\System32\Winbio.dll.

■ **The fingerprint biometric service provider** This wraps the functions of biometric-type-specific adapters to present a common interface, independent of the type of biometric, to the Windows Biometric Service. In the future, additional types of biometrics, such as retinal scans or voice-print analyzers, might be supported by additional biometric service providers. The biometric service provider in turn uses three adapters, which are user-mode DLLs:

- **The sensor adapter** This exposes the data-capture functionality of the scanner. The sensor adapter usually uses Windows I/O calls to access the scanner hardware. Windows provides a sensor adapter that can be used with simple sensors, those for which a WBDI driver exists. For more complex sensors, the sensor adapter is written by the sensor vendor.

- **The engine adapter** This exposes processing and comparison functionality specific to the scanner's raw data format and other features. The actual processing and comparison might be performed within the engine adapter DLL, or the DLL might communicate with some other module. The engine adapter is always provided by the sensor vendor.

- **The storage adapter** This exposes a set of secure storage functions. These are used to store and retrieve templates against which scanned biometric data is matched by the engine adapter. Windows provides a storage adapter using Windows cryptography services and standard disk file storage. A sensor vendor might provide a different storage adapter.

■ **The functional device driver for the actual biometric scanner device** This exposes the WBDI at its upper edge. It usually uses the services of a lower-level bus driver, such as the USB bus driver, to access the scanner device. This driver is always provided by the sensor vendor.

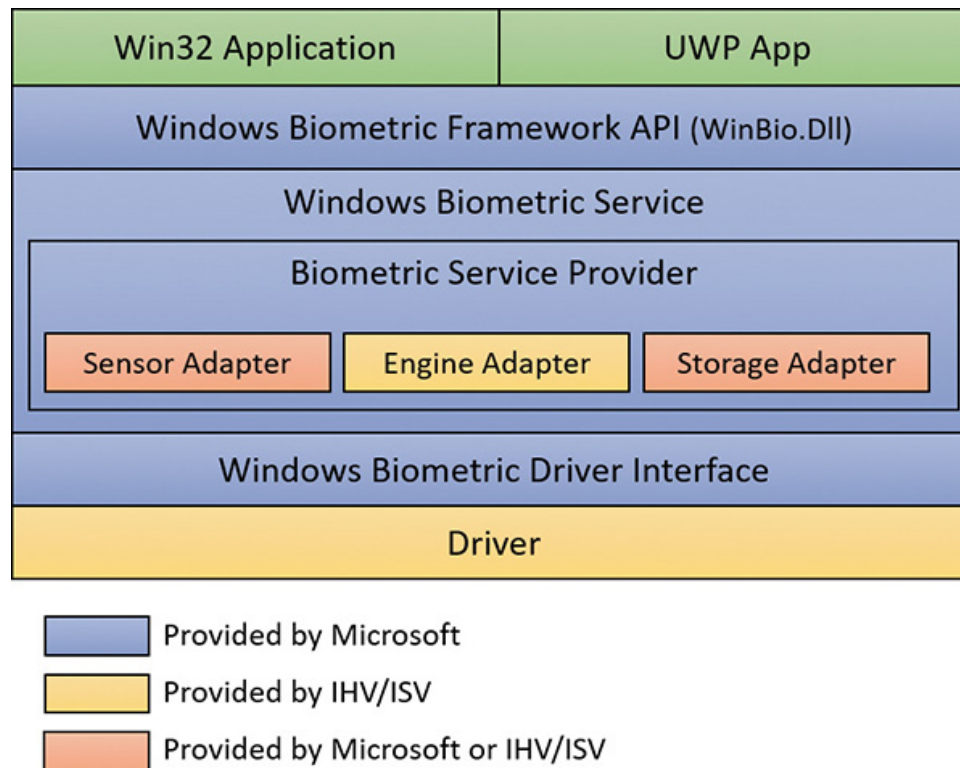


FIGURE 7-21 Windows Biometric Framework components and architecture.

A typical sequence of operations to support logging in via a fingerprint scan might be as follows:

1. After initialization, the sensor adapter receives from the service provider a request for capture data. The sensor adapter in turn sends a `DeviceIoControl` request with the `IOCTL_BIOMETRIC_CAPTURE_DATA` control code to the WBDI driver for the fingerprint scanner device.
2. The WBDI driver puts the scanner into capture mode and queues the `IOCTL_BIOMETRIC_CAPTURE_DATA` request until a fingerprint scan occurs.
3. A prospective user swipes a finger across the scanner. The WBDI driver receives notification of this, obtains the raw scan data from the sensor,

and returns this data to the sensor driver in a buffer associated with the `IOCTL_BIOMETRIC_CAPTURE_DATA` request.

4. The sensor adapter provides the data to the fingerprint biometric service provider, which in turn passes the data to the engine adapter.

5. The engine adapter processes the raw data into a form compatible with its template storage.

6. The fingerprint biometric service provider uses the storage adapter to obtain templates and corresponding security IDs from secure storage. It invokes the engine adapter to compare each template to the processed scan data. The engine adapter returns a status indicating whether it's a match or not a match.

7. If a match is found, the Windows Biometric Service notifies Winlogon, via a credential provider DLL, of a successful login and passes it the security ID of the identified user. This notification is sent via an ALPC message, providing a path that cannot be spoofed.

Windows Hello

Windows Hello, introduced in Windows 10, provides new ways to authenticate users based on biometric information. With this technology, users can log in effortlessly just by showing themselves to the device's camera or swiping their finger.

At the time of this writing, Windows Hello supports three types of biometric identification:

■ Fingerprint

■ Face

■ Iris

The security aspect of biometrics needs to be considered first. What is the likelihood of someone being identified as you? What is the likelihood of you *not* being identified as you? These questions are parameterized by two factors:

■ **False accept rate (uniqueness)** This is the probability of another user having the same biometric data as you. Microsoft's algorithms make sure

the likelihood is 1 in 100,000.

■ **False reject rate (reliability)** This is the probability of you not being correctly recognized as you (for example, in abnormal lighting conditions for face or iris recognition). Microsoft's implementation makes sure there is less than 1 percent chance of this happening. If it does happen, the user can try again or use a PIN code instead.

Using a PIN code may seem less secure than using a full-blown password (the PIN can be as simple as a four-digit number). However, a PIN is more secure than a password for two main reasons:

■ The PIN code is local to the device and is never transmitted across the network. This means that even if someone gets a hold of the PIN, they cannot use it to log in as the user from any other device. Passwords, on the other hand, travel to the domain controller. If someone gets hold of the password, they can log in from another machine into the domain.

■ The PIN code is stored in the Trusted Platform Module (TPM)—a piece of hardware that also plays a part in Secure Boot (discussed in detail in Chapter 11 in Part 2)—so is difficult to access. In any case, it requires physical access to the device, raising the bar considerably for a potential security compromise.

Windows Hello is built upon the Windows Biometric Framework (WBF) (described in the previous section). Current laptop devices support fingerprint and face biometrics, while iris is only supported on the Microsoft Lumia 950 and 950 XL phones. (This will likely change and expand in future devices.) Note that face recognition requires an infrared (IR) camera as well as a normal (RGB) one, and is supported on devices such as the Microsoft Surface Pro 4 and the Surface Book.

User Account Control and virtualization

User Account Control (UAC) is meant to enable users to run with standard user rights as opposed to administrative rights. Without administrative rights, users cannot accidentally (or deliberately) modify system settings, malware can't normally alter system security settings or disable antivirus software, and users can't compromise the sensitive information of other users on shared computers. Running with standard user rights can thus mitigate the impact of malware and protect sensitive data on shared computers.

UAC had to address a couple of problems to make it practical for a user to run with a standard user account. First, because the Windows usage model has been one of assumed administrative rights, software developers assumed their programs would run with those rights and could therefore access and modify any file, registry key, or operating system setting. Second, users sometimes need administrative rights to perform such operations as installing software, changing the system time, and opening ports in the firewall.

The UAC solution to these problems is to run most applications with standard user rights, even though the user is logged in to an account with administrative rights. At the same time, UAC makes it possible for standard users to access administrative rights when they need them—whether for legacy applications that require them or for changing certain system settings. As described, UAC accomplishes this by creating a filtered admin token as well as the normal admin token when a user logs in to an administrative account. All processes created under the user's session will normally have the filtered admin token in effect so that applications that can run with standard user rights will do so. However, the administrative user can run a program or perform other functions that require full Administrator rights through UAC elevation.

Windows also allows certain tasks that were previously reserved for administrators to be performed by standard users, enhancing the usability of the standard user environment. For example, Group Policy settings exist that can enable standard users to install printers and other device drivers approved by IT administrators and to install ActiveX controls from administrator-approved sites.

Finally, when software developers test in the UAC environment, they are encouraged to develop applications that can run without administrative rights. Fundamentally, non-administrative programs should not need to run with administrator privileges; programs that often require administrator privileges are typically legacy programs using old APIs or techniques, and they should be updated.

Together, these changes obviate the need for users to run with administrative rights all the time.

File system and registry virtualization

Although some software legitimately requires administrative rights, many programs needlessly store user data in system-global locations. When an application executes, it can be running in different user accounts, and it should therefore store user-specific data in the per-user %AppData% directory and save per-user settings in the user's registry profile under HKEY_CURRENT_USER\Software. Standard user accounts don't have write access to the %ProgramFiles% directory or HKEY_LOCAL_MACHINE\Software, but because most Windows systems are single-user and most users have been administrators until UAC was implemented, applications that incorrectly saved user data and settings to these locations worked anyway.

Windows enables these legacy applications to run in standard user accounts through the help of file system and registry namespace virtualization. When an application modifies a system-global location in the file system or registry and that operation fails because access is denied, Windows redirects the operation to a per-user area. When the application reads from a system-global location, Windows first checks for data in the per-user area and, if none is found, permits the read attempt from the global location.

Windows will always enable this type of virtualization unless:

- **The application is 64-bit** Because virtualization is purely an application-compatibility technology meant to help legacy applications, it is enabled only for 32-bit applications. The world of 64-bit applications is relatively new and developers should follow the development guidelines for creating standard user-compatible applications.

- **The application is already running with administrative rights** In this case, there is no need for any virtualization.

- **The operation came from a kernel-mode caller**

- **The operation is being performed while the caller is impersonating** For example, any operations not originating from a process classified as legacy according to this definition, including network file-sharing accesses, are not virtualized.

- **The executable image for the process has a UAC-compatible manifest** Specifying a `requestedExecutionLevel` setting, described in the

next section.

■ **The administrator does not have write access to the file or registry**

key This exception exists to enforce backward compatibility because the legacy application would have failed before UAC was implemented even if the application was run with administrative rights.

■ **Services are never virtualized**

You can see the virtualization status (the process virtualization status is stored as a flag in its token) of a process by adding the UAC Virtualization column to Task Manager's Details page, as shown in [Figure 7-22](#). Most Windows components—including the Desktop Window Manager (Dwm.exe), the Client Server Run-Time Subsystem (Csrss.exe), and Explorer—have virtualization disabled because they have a UAC-compatible manifest or are running with administrative rights and so do not allow virtualization. However, 32-bit Internet Explorer (iexplore.exe) has virtualization enabled because it can host multiple ActiveX controls and scripts and must assume that they were not written to operate correctly with standard user rights. Note that, if required, virtualization can be completely disabled for a system using a Local Security Policy setting.

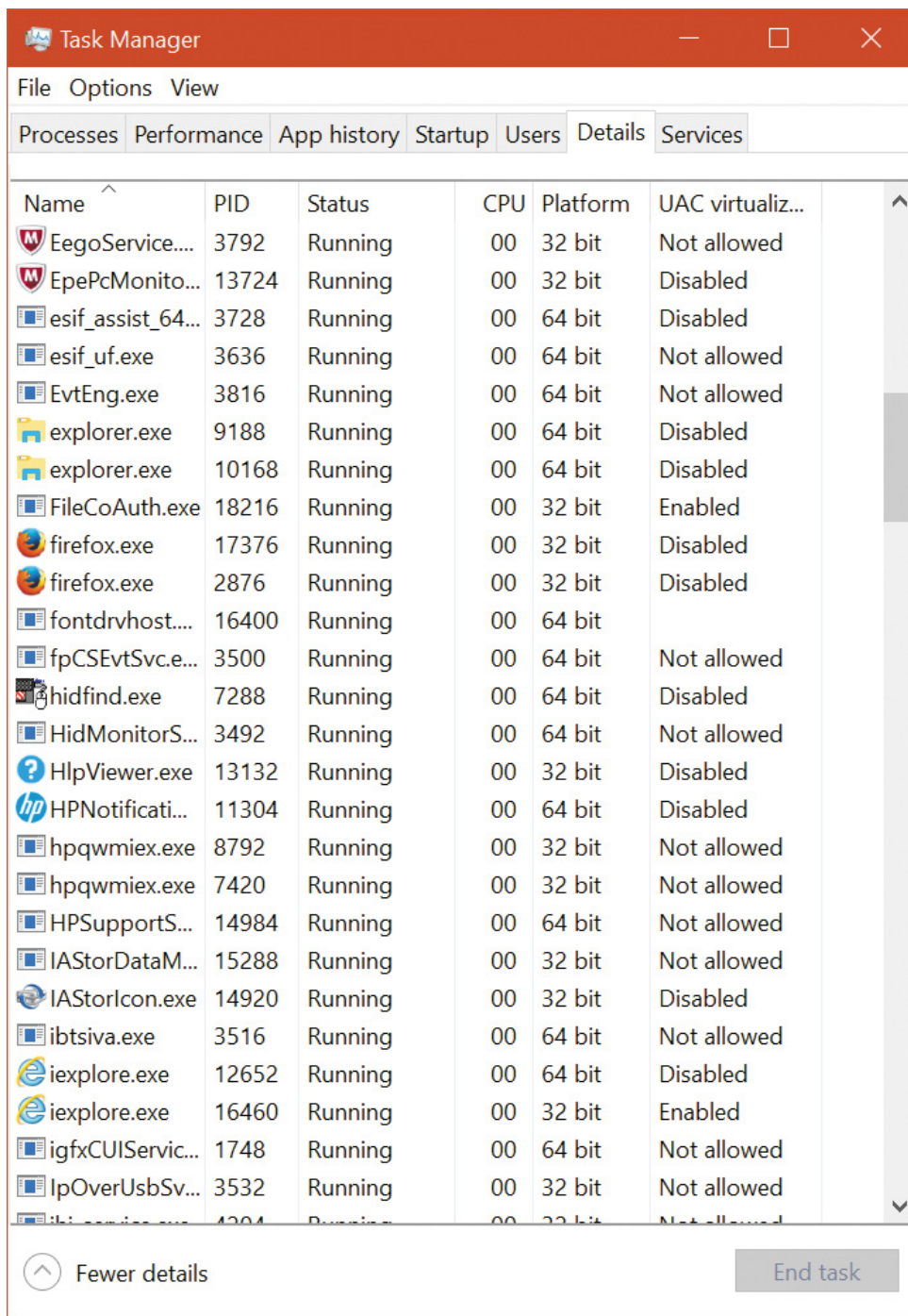


FIGURE 7-22 Using Task Manager to view virtualization status.

In addition to file system and registry virtualization, some applications require additional help to run correctly with standard user rights. For example, an application that tests the account in which it's running for membership in the Administrators group might otherwise work, but it won't run if it's not in that group. Windows defines a number of application-compatibility shims to enable such applications to work anyway. The shims most commonly applied to legacy applications for operation with standard user rights are shown in [Table 7-15](#).

Flag	Meaning
ElevateCreateProcess	This changes CreateProcess to handle ERROR_ELEVATION_REQUIRED errors by calling the application information service to prompt for elevation.
ForceAdminAccess	This spoofs queries of Administrator group membership.
VirtualizeDeleteFile	This spoofs successful deletion of global files and directories.
LocalMappedObject	This forces global section objects into the user's namespace.
VirtualizeHKCR Lite	This redirects global registration of COM objects to a per-user location.
VirtualizeRegisterTypeLib	This converts per-machine type lib registrations to per-user registrations.

TABLE 7-15 UAC virtualization shims

File virtualization

The file system locations that are virtualized for legacy processes are %ProgramFiles%, %ProgramData%, and %SystemRoot%, excluding some specific subdirectories. However, any file with an executable extension—including .exe, .bat, .scr, .vbs, and others—is excluded from virtualization. This means that programs that update themselves from a standard user account fail instead of creating private versions of their executables that aren't visible to an administrator running a global updater.



NOTE

To add extensions to the exception list, enter them in the HKLM\System\Current-ControlSet\Services\Luafv\Parameters\ExcludedExtensionsAdd registry key and reboot. Use a multistring type to delimit multiple extensions, and do not include a leading dot in the extension name.

Modifications to virtualized directories by legacy processes are redirected to the user's virtual root directory, %LocalAppData%\VirtualStore. The Local component of the path highlights the fact that virtualized files don't roam with the rest of the profile when the account has a roaming profile.

The UAC File Virtualization filter driver

(%SystemRoot%\System32\Drivers\Luafv.sys) implements file system virtualization. Because this is a file system filter driver, it sees all local file system operations, but it implements functionality only for operations from legacy processes. As shown in **Figure 7-23**, the filter driver changes the target file path for a legacy process that creates a file in a system-global location but does not for a non-virtualized process with standard user rights. Default permissions on the \Windows directory deny access to the application written with UAC support, but the legacy process acts as

though the operation succeeds when it really created the file in a location fully accessible by the user.

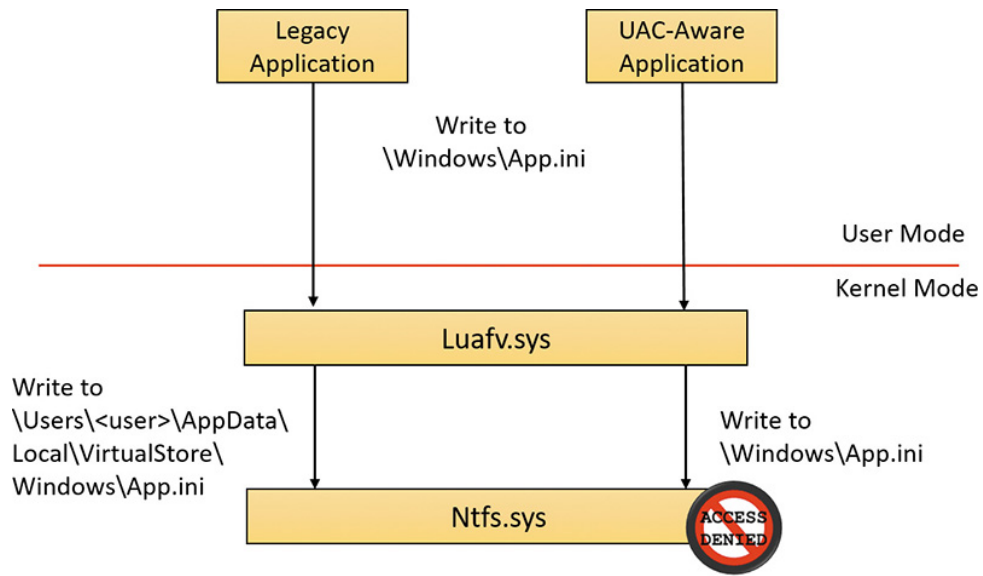


FIGURE 7-23 UAC File Virtualization filter driver operation.

EXPERIMENT: FILE VIRTUALIZATION BEHAVIOR

In this experiment, you will enable and disable virtualization on the command prompt and see several behaviors to demonstrate UAC file virtualization:

1. Open a non-elevated command prompt (you must have UAC enabled for this to work) and enable virtualization for it. You can change the virtualization status of a process by right-clicking the process in the Task Manager Details tab and selecting **UAC Virtualization** from the shortcut menu that appears.

2. Navigate to the C:\Windows directory and use the following command to write a file:

```
echo hello-1 > test.txt
```

3. List the contents of the directory. You'll see that the file appears.

```
dir test.txt
```

4. Disable virtualization by right-clicking the process in the Task Manager Details tab and deselecting **UAC Virtualization**. Then list the directory as in step 3. Notice that the file is gone. However, a directory listing of the VirtualStore directory will reveal the file:

[Click here to view code image](#)

```
dir %LOCALAPPDATA%\VirtualStore\Windows\test.txt
```

5. Enable virtualization again for this process.

6. To look at a more complex scenario, create a new command prompt window, but elevated this time. Then repeat steps 2 and 3 using the string `hello-2`.

7. Examine the text inside these files by using the following command in both command prompts. The screenshots that follow show the expected output.

```
type test.txt
```

```
C:\WINDOWS\system32\cmd.exe
C:\Windows>echo hello-1 > test.txt
C:\Windows>dir test.txt
Volume in drive C is Windows
Volume Serial Number is 1C8D-7A57

Directory of C:\Windows

26-Feb-17  23:17                10 test.txt
             1 File(s)                10 bytes
             0 Dir(s) 91,545,452,544 bytes free

C:\Windows>dir test.txt
Volume in drive C is Windows
Volume Serial Number is 1C8D-7A57

Directory of C:\Windows

File Not Found

C:\Windows>type test.txt
hello-1
C:\Windows>

Administrator: Command Prompt
Microsoft Windows [Version 10.0.14393]
(c) 2016 Microsoft Corporation. All rights reserved.

C:\WINDOWS\system32>cd\windows
C:\Windows>echo hello-2 > test.txt
C:\Windows>type test.txt
hello-2
C:\Windows>
```

8. From your elevated command prompt, delete the test.txt file:

```
del test.txt
```

9. Repeat step 3 of the experiment in both windows. Notice that the elevated command prompt cannot find the file anymore, while the standard user command prompt shows the old contents of the file again. This demonstrates the failover mechanism described earlier: Read operations look in the per-user virtual store location first, but if the file doesn't exist, read access to the system location will be granted.

Registry virtualization

Registry virtualization is implemented slightly differently from file system virtualization. Virtualized registry keys include most of the HKEY_LOCAL_MACHINE\Software branch, but there are numerous exceptions, such as the following:

- HKLM\Software\Microsoft\Windows
- HKLM\Software\Microsoft\Windows NT

Only keys that are commonly modified by legacy applications, but that don't introduce compatibility or interoperability problems, are virtualized. Windows redirects modifications of virtualized keys by a legacy application to a user's registry virtual root at `HKEY_CURRENT_USER\Software\Classes\VirtualStore`. The key is located in the user's Classes hive, `%LocalAppData%\Microsoft\Windows\UsrClass.dat`, which, like any other virtualized file data, does not roam with a roaming user profile. Instead of maintaining a fixed list of virtualized locations as Windows does for the file system, the virtualization status of a key is stored as a combination of flags, shown in [Table 7-16](#).

Flag	Meaning
<code>REG_KEY_DONT_VIRTUALIZE</code>	This specifies whether virtualization is enabled for this key. If the flag is set, virtualization is disabled.
<code>REG_KEY_DONT_SILENT_FAIL</code>	If the <code>REG_KEY_DONT_VIRTUALIZE</code> flag is set (virtualization is disabled), this key specifies that a legacy application that would be denied access performing an operation on the key is instead granted <code>MAXIMUM_ALLOWED</code> rights to the key (any access the account is granted) instead of the rights the application requested. If this flag is set, it implicitly disables virtualization as well.
<code>REG_KEY_RECURSE_FLAG</code>	This determines whether the virtualization flags will propagate to the child keys (subkeys) of this key.

TABLE 7-16 Registry virtualization flags

You can use the `Reg.exe` utility included in Windows, with the `flags` option, to display the current virtualization state for a key or to set it. In [Figure 7-24](#), note that the `HKLM\Software` key is fully virtualized, but the `Windows` subkey (and all its children) have only silent failure enabled.

```

Administrator: Command Prompt

C:\Windows>reg flags hklm\software

HKEY_LOCAL_MACHINE\software
    REG_KEY_DONT_VIRTUALIZE: CLEAR
    REG_KEY_DONT_SILENT_FAIL: CLEAR
    REG_KEY_RECURSE_FLAG: CLEAR

The operation completed successfully.

C:\Windows>reg flags hklm\software\microsoft\windows

HKEY_LOCAL_MACHINE\software\microsoft\windows
    REG_KEY_DONT_VIRTUALIZE: SET
    REG_KEY_DONT_SILENT_FAIL: CLEAR
    REG_KEY_RECURSE_FLAG: SET

The operation completed successfully.

C:\Windows>
  
```

FIGURE 7-24 UAC registry virtualization flags on the `Software` and `Windows` keys.

Unlike file virtualization, which uses a filter driver, registry virtualization is implemented in the configuration manager. (See Chapter 9 in Part 2 for more information on the registry and the configuration manager.) As with file system virtualization, a legacy process creating a subkey of a virtualized key is redirected to the user's registry virtual root, but a UAC-compatible process is denied access by default permissions. This is shown in [Figure 7-25](#).

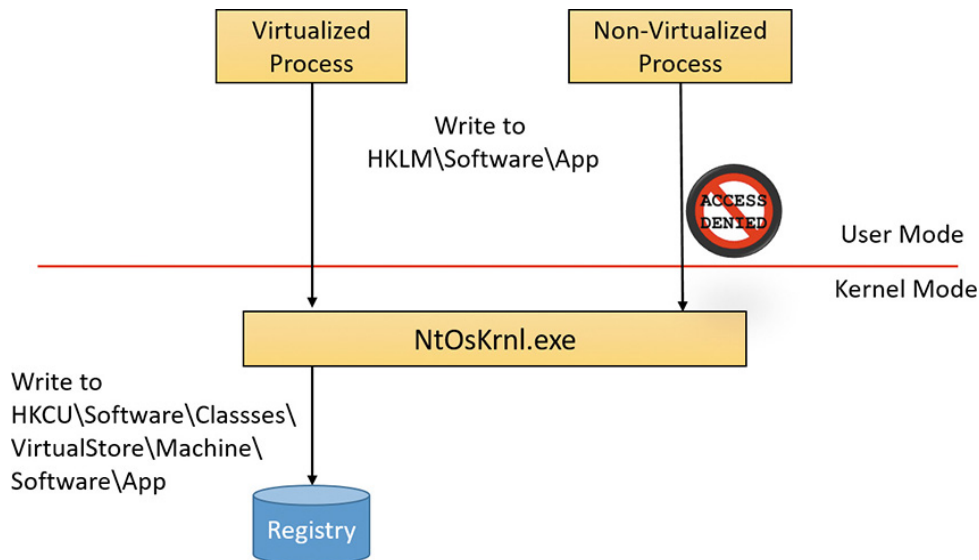


FIGURE 7-25 UAC registry virtualization operation.

Elevation

Even if users run only programs that are compatible with standard user rights, some operations still require administrative rights. For example, the vast majority of software installations require administrative rights to create directories and registry keys in system-global locations or to install services or device drivers. Modifying system-global Windows and application settings also requires administrative rights, as does the parental controls feature. It would be possible to perform most of these operations by switching to a dedicated administrator account, but the inconvenience of doing so would likely result in most users remaining in the administrator account to perform their daily tasks, most of which do not require administrative rights.

It's important to be aware that UAC elevations are conveniences and not security boundaries. A security boundary requires that security policy dictate what can pass through the boundary. User accounts are an example of a security boundary in Windows because one user can't access the data belonging to another user without having that user's permission.

Because elevations aren't security boundaries, there's no guarantee that malware running on a system with standard user rights can't compromise an elevated process to gain administrative rights. For example, elevation dialog boxes only identify the executable that will be elevated; they say nothing about what it will do when it executes.

Running with administrative rights

Windows includes enhanced "run as" functionality so that standard users can conveniently launch processes with administrative rights. This functionality requires giving applications a way to identify operations for which the system can obtain administrative rights on behalf of the application, as necessary (we'll say more on this topic shortly).

To enable users acting as system administrators to run with standard user rights but not have to enter user names and passwords every time they want to access administrative rights, Windows makes use of a mechanism called *Admin Approval Mode (AAM)*. This feature creates two identities for the user at logon: one with standard user rights and another with administrative rights. Since every user on a Windows system is either a standard user or acting for the most part as a standard user in AAM, developers must assume that all Windows users are standard users, which will result in more programs working with standard user rights without virtualization or shims.

Granting administrative rights to a process is called *elevation*. When elevation is performed by a standard user account (or by a user who is part of an administrative group but not the actual Administrators group), it's referred to as an *over-the-shoulder (OTS) elevation* because it requires the entry of credentials for an account that's a member of the Administrators group, something that's usually completed by a privileged user typing over the shoulder of a standard user. An elevation performed by an AAM user is called a *consent elevation* because the user simply has to approve the assignment of his administrative rights.

Stand-alone systems, which are typically home computers, and domain-joined systems treat AAM access by remote users differently because domain-connected computers can use domain administrative groups in their resource permissions. When a user accesses a stand-alone computer's file share, Windows requests the remote user's standard user identity. But on domain-joined systems, Windows honors all the user's domain group memberships by requesting the user's administrative identity. Executing an image that requests administrative rights causes the ap-

plication information service (AIS, contained in %SystemRoot%\System32\Appinfo.dll), which runs inside a standard service host process (SvcHost.exe), to launch %SystemRoot%\System32\Consent.exe. Consent captures a bitmap of the screen, applies a fade effect to it, switches to a desktop that's accessible only to the local system account (the Secure Desktop), paints the bitmap as the background, and displays an elevation dialog box that contains information about the executable. Displaying this dialog box on a separate desktop prevents any application present in the user's account from modifying the appearance of the dialog box.

If an image is a Windows component digitally signed (by Microsoft or another entity), the dialog box displays a light blue stripe across the top, as shown at the left of [Figure 7-26](#) (the distinction between Microsoft signed images and other signers has been removed in Windows 10). If the image is unsigned, the stripe becomes yellow, and the prompt stresses the unknown origin of the image (see the right of [Figure 7-26](#)). The elevation dialog box shows the image's icon, description, and publisher for digitally signed images, but it shows only the file name and "Publisher: Unknown" for unsigned images. This difference makes it harder for malware to mimic the appearance of legitimate software. The Show More Details link at the bottom of the dialog box expands it to show the command line that will be passed to the executable if it launches.

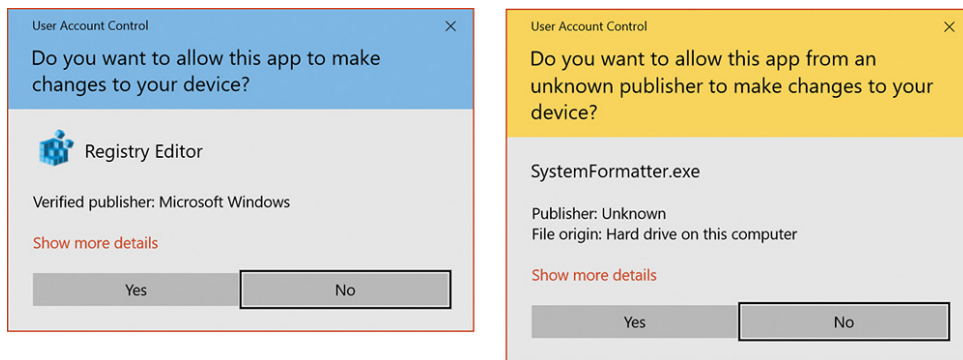


FIGURE 7-26 AAC UAC elevation dialog boxes based on image signature.

The OTS consent dialog box, shown in [Figure 7-27](#), is similar, but prompts for administrator credentials. It will list any accounts with administrator rights.

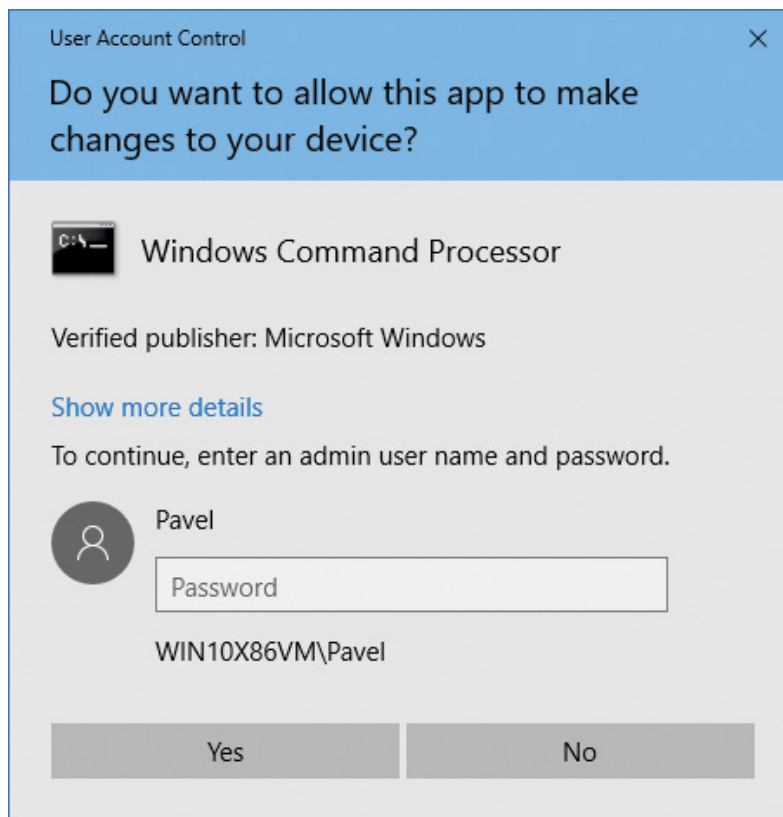


FIGURE 7-27 OTS consent dialog box.

If a user declines an elevation, Windows returns an access-denied error to the process that initiated the launch. When a user agrees to an elevation by either entering administrator credentials or clicking Yes, AIS calls `CreateProcessAsUser` to launch the process with the appropriate administrative identity. Although AIS is technically the parent of the elevated process, AIS uses new support in the `CreateProcessAsUser` API that sets the process's parent process ID to that of the process that originally launched it. That's why elevated processes don't appear as children of the AIS service-hosting process in tools such as Process Explorer that show process trees. **Figure 7-28** shows the operations involved in launching an elevated process from a standard user account.

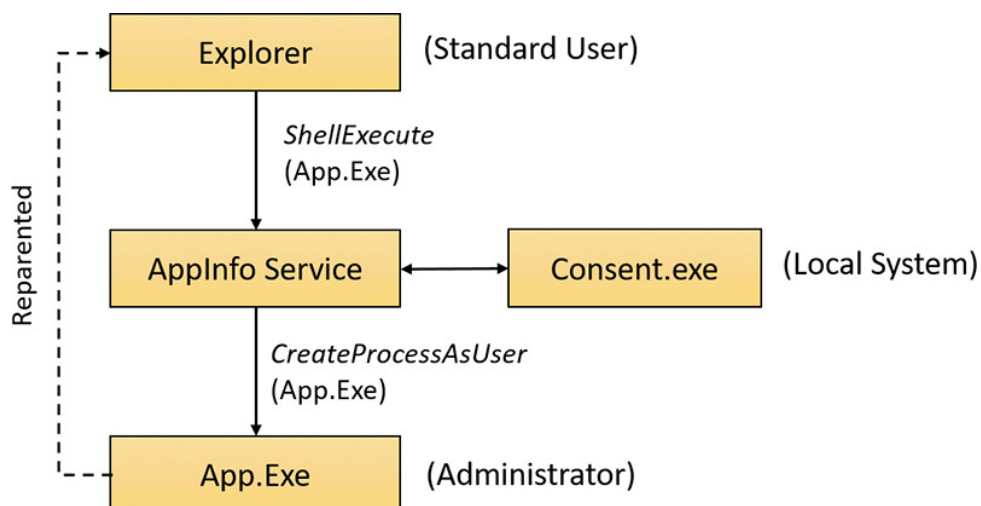


FIGURE 7-28 Launching an administrative application as a standard user.

Requesting administrative rights

There are a number of ways the system and applications identify a need for administrative rights. One that shows up in the Explorer user interface is the Run as Administrator context menu command and shortcut option. These items also include a blue and gold shield icon that should be placed next to any button or menu item that will result in an elevation of rights when it is selected. Choosing the Run as Administrator command causes Explorer to call the `ShellExecute` API with the `runas` verb.

The vast majority of installation programs require administrative rights, so the image loader, which initiates the launch of an executable, includes installer-detection code to identify likely legacy installers. Some of the heuristics it uses are as simple as detecting internal version information or whether the image has the words *setup*, *install*, or *update* in its file name. More sophisticated means of detection involve scanning for byte sequences in the executable that are common to third-party installation wrapper utilities. The image loader also calls the application-compatibility library to see if the target executable requires administrator rights. The library looks in the application-compatibility database to see whether the executable has the `RequireAdministrator` or `RunAsInvoker` compatibility flag associated with it.

The most common way for an executable to request administrative rights is for it to include a `requestedExecutionLevel` tag in its application manifest file. The element's level attribute can have one of the three values shown in **Table 7-17**.

Elevation Level	Meaning	Usage
As invoker	No need for administrative rights; never ask for elevation.	Typical user applications that don't need administrative privileges—for example, Notepad.
Highest available	Request approval for highest rights available. If the user is logged on as a standard user, the process will be launched as invoker; otherwise, an AAM elevation prompt will appear, and the process will run with full administrative rights.	Applications that can function without full administrative rights but expect users to want full access if it's easily accessible. For example, the Registry Editor, Microsoft Management Console, and the Event Viewer use this level.
Require administrator	Always request administrative rights. An OTS elevation dialog box prompt will be shown for standard users; otherwise, AAM.	Applications that require administrative rights to work, such as the Firewall Settings Editor, which affects system-wide security.

TABLE 7-17 Requested elevation levels

The presence of the `trustInfo` element in a manifest (which you can see in the manifest dump of `eventvwr.exe`) denotes an executable that was written with support for UAC and the `requestedExecutionLevel` element nests within it. The `uiAccess` attribute is where accessibility applications can use the UIPI bypass functionality mentioned earlier.

[Click here to view code image](#)

```
C:\>sigcheck -m c:\Windows\System32\eventvwr.exe
...
<trustInfo xmlns="urn:schemas-microsoft-com:asm.v3">
  <security>
    <requestedPrivileges>
      <requestedExecutionLevel
        level="highestAvailable"
        uiAccess="false"
      />
    </requestedPrivileges>
  </security>
</trustInfo>
<asmv3:application>
  <asmv3:windowsSettings xmlns="http://schemas.microsoft.com/SMI/2005/WindowsSetti
    <autoElevate>true</autoElevate>
  </asmv3:windowsSettings>
</asmv3:application>
...
```

Auto-elevation

In the default configuration (see the next section for information on changing this), most Windows executables and control panel applets do not result in elevation prompts for administrative users, even if they need administrative rights to run. This is because of a mechanism called *auto-elevation*. Auto-elevation is intended to preclude administrative users from seeing elevation prompts for most of their work; the programs will automatically run under the user's full administrative token.

Auto-elevation has several requirements. One is that the executable in question must be considered as a Windows executable. This means it must be signed by the Windows publisher (not just by Microsoft; oddly, they are not the same—Windows-signed is considered more privileged than Microsoft-signed). It must also be in one of several directories considered secure: %SystemRoot%\System32 and most of its subdirectories, %Systemroot%\Ehome, and a small number of directories under %ProgramFiles% (for example, those containing Windows Defender and Windows Journal).

There are additional requirements, depending on the type of executable. EXE files other than Mmc.exe auto-elevate if they are requested via an `autoElevate` element in their manifest. The manifest shown earlier of eventvwr.exe in the previous section illustrates this.

Mmc.exe is treated as a special case because whether it should auto-elevate or not depends on which system management snap-ins it is to load. Mmc.exe is normally invoked with a command line specifying an MSC file, which in turn specifies which snap-ins are to be loaded. When Mmc.exe is run from a protected administrator account (one running with the limited administrator token), it asks Windows for administrative rights. Windows validates that Mmc.exe is a Windows executable and then checks the MSC. The MSC must also pass the tests for a Windows executable, and furthermore must be on an internal list of auto-elevate MSCs. This list includes nearly all MSC files in Windows.

Finally, COM (out-of-process server) classes can request administrative rights within their registry key. To do so requires a subkey named `Elevation` with a DWORD value named `Enabled`, having a value of `1`. Both the COM class and its instantiating executable must meet the Windows executable requirements, although the executable need not have requested auto-elevation.

Controlling UAC behavior

UAC can be modified via the dialog box shown in [Figure 7-29](#). This dialog box is available under Change User Account Control Settings. [Figure 7-29](#) shows the control in its default position.

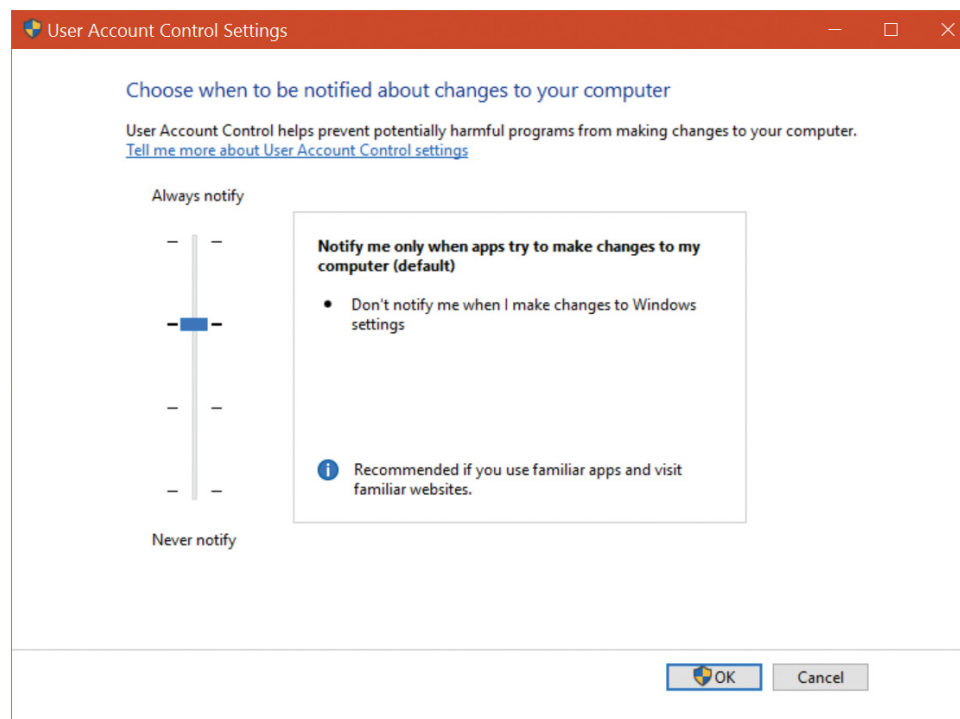


FIGURE 7-29 User Account Control Settings dialog box.

The four possible settings have the effects described in [Table 7-18](#).

Slider Position	When administrative user not running with administrative rights...		Remarks
	...attempts to change Windows settings (for example, use certain Control Panel applets)	...attempts to install software or run a program whose manifest calls for elevation, or uses Run as Administrator	
Highest position (Always Notify)	A UAC elevation prompt appears on the Secure Desktop.	A UAC elevation prompt appears on the Secure Desktop.	This was the Windows Vista behavior.
Second position	UAC elevation occurs automatically with no prompt or notification.	A UAC elevation prompt appears on the Secure Desktop.	Windows default setting.
Third position	UAC elevation occurs automatically with no prompt or notification.	A UAC elevation prompt appears on the user's normal desktop.	Not recommended.
Lowest position (Never Notify)	UAC is turned off for administrative users.	UAC is turned off for administrative users.	Not recommended.

TABLE 7-18 UAC options

The third position is not recommended because the UAC elevation prompt appears not on the Secure Desktop but on the normal user's desktop. This could allow a malicious program running in the same session to change the appearance of the prompt. It is intended for use only in systems where the video subsystem takes a long time to dim the desktop or is otherwise unsuitable for the usual UAC display.

The lowest position is strongly discouraged because it turns UAC off completely as far as administrative accounts are concerned. Prior to Windows 8, all processes run by a user with an administrative account are run with the user's full administrative rights in effect; there is no filtered admin token. Starting with Windows 8, UAC cannot be turned off completely, because of the AppContainer model. Admin users won't be prompted for elevation, but processes will not elevate unless required to do so by the manifest or launched from an elevated process.

The UAC setting is stored in four values in the registry under `HKLM\SOFTWARE\Microsoft\Windows\CurrentVersion\Policies\System`, as shown in [Table 7-19](#). `ConsentPromptBehaviorAdmin` controls the UAC elevation prompt for administrators running with a filtered admin token, and `ConsentPromptBehaviorUser` controls the UAC prompt for users other than administrators.

Slider Position	ConsentPromptBehaviorAdmin	ConsentPromptBehaviorUser	EnableLUA	PromptOnSecureDesktop
Highest position (Always Notify)	2 (display AAC UAC elevation prompt)	3 (display OTS UAC elevation prompt)	1 (enabled)	1 (enabled)
Second position	5 (display AAC UAC elevation prompt, except for changes to Windows settings)	3	1	1
Third position	5	3	1	0 (disabled; UAC prompt appears on user's normal desktop)
Lowest position (Never Notify)	0	3	0 (disabled; logins to administrative accounts do not create a restricted admin access token)	0

TABLE 7-19 UAC registry values

Exploit mitigations

Throughout this chapter, we've seen a number of technologies that help protect the user, guarantee the code-signing properties of executable code, and lock down access to resources through sandboxing. At the end of the day, however, all secure systems have failure points, all code has bugs, and attackers leverage increasingly complex attacks to exploit them. A security model in which all code is assumed to be bug-free, or in which a software developer assumes all bugs will eventually be found and fixed, is destined to fail. Additionally, many security features that provide code-execution “guarantees” do so at a cost of performance or compatibility, which may be unacceptable in such scenarios.

A much more successful approach is to identify the most common techniques used by attackers, as well as employ an internal “red team” (that is, an internal team attacking its own software) to discover new techniques before attackers do and to implement mitigations against such techniques. (These mitigations can be as simple as moving some data around or as complex as employing Control Flow Integrity [CFI] techniques.) Because vulnerabilities can number in the thousands in a complex code base such as Windows, but exploit techniques are limited, the idea is to make large classes of bugs very difficult (or in some cases, impossible) to exploit, without worrying about finding all the bugs.

Process-mitigation policies

While individual applications can implement various exploit mitigations on their own (such as Microsoft Edge, which leverages a mitigation called *MemGC* to avoid many classes of memory-corruption attacks), this section will cover mitigations that are provided by the operating system to all applications or to the system itself to reduce exploitable bug classes. [Table 7-20](#) describes all mitigations in the latest version of Windows 10 Creators Update, the type of bug class they mitigate against, and mechanisms to activate them.

Mitigation Name	Use Case	Enabling Mechanism
ASLR Bottom Up Randomization	This makes calls to <code>VirtualAlloc</code> subject to ASLR with 8-bit entropy, including stack-base randomization.	This is set with the <code>PROCESS_CREATION_MITIGATION_POLICY_BOTTOM_UP_ASLR_ALWAYS_ON</code> process-creation attribute flag.
ASLR Force Relocate Images	This forces ASLR even on binaries that do not have the <code>/DYNAMICBASE</code> linker flag.	This is set with <code>SetProcessMitigationPolicy</code> or the <code>PROCESS_CREATION_MITIGATION_POLICY_FORCE_RELOCATE_IMAGES_ALWAYS_ON</code> process-creation flag.
High Entropy ASLR (HEASLR)	This significantly increases entropy of ASLR on 64-bit images, increasing bottom-up randomization to up to 1 TB of variance (that is, bottom-up allocations may start anywhere between 64 KB and 1 TB into the address space, giving 24 bits of entropy).	Must be set through <code>/HIGHENTROPYVA</code> at link time or the <code>PROCESS_CREATION_MITIGATION_POLICY_HIGH_ENTROPY_ASLR_ALWAYS_ON</code> process-creation attribute flag.
ASLR Disallow Stripped Images	This blocks the load of any library without relocations (linked with the <code>/FIXED</code> flag) when combined with ASLR Force Relocate Images.	This is set with <code>SetProcessMitigationPolicy</code> or with the <code>PROCESS_CREATION_MITIGATION_POLICY_FORCE_RELOCATE_IMAGES_ALWAYS_ON_REQ_RELOCS</code> process-creation flag.
DEP: Permanent	This prevents the process from disabling DEP on itself. Only relevant on x86. Only relevant on 32-bit applications (and/or under WoW64)	This is set with the <code>SetProcessMitigationPolicy</code> , process-creation attribute or with <code>SetProcessDEPPolicy</code> .
DEP: Disable ATL Thunk Emulation	This prevents legacy ATL library code from executing ATL thunks in the heap, even if a known compatibility issue. Only relevant on 32-bit applications (and/or under WoW64)	This is set with the <code>SetProcessMitigationPolicy</code> , process-creation attribute or with <code>SetProcessDEPPolicy</code> .
SEH Overwrite Protection (SEHOP)	This prevents structure exception handlers from being overwritten with incorrect ones, even if the image was not linked with Safe SEH (<code>/SAFESEH</code>). Only relevant on 32-bit applications (and/or under WoW64).	This can be set with <code>SetProcessDEPPolicy</code> or with the <code>PROCESS_CREATION_MITIGATION_POLICY_SEHOP_ENABLE</code> process-creation flag.
Raise Exception on Invalid Handle	This helps catch handle reuse (use-after-handle-close) attacks in which a process uses a handle that is no longer the handle it expected (for example: <code>SetEvent</code> on a mutex) by crashing the process instead of returning a failure that the process might ignore.	This is set with <code>SetProcessMitigationPolicy</code> or the <code>PROCESS_CREATION_MITIGATION_POLICY_STRICT_HANDLE_CHECKS_ALWAYS_ON</code> process-creation attribute flag.
Raise Exception on Invalid Handle Close	This helps catch handle reuse (double-handle-close) attacks in which a process is attempting to close a handle that has already been closed, suggesting that a different handle may potentially be used in other scenarios, in which an exploit would be successful, ultimately limiting its universal effectiveness.	Undocumented, and can only be set through an undocumented API.

Disallow Win32k System Calls	This disables all access to the Win32 kernel-mode subsystem driver, which implements the Window Manager (GUI) and Graphics Device Interface (GDI) and DirectX. No system calls to this component will be possible.	This is set with <code>SetProcessMitigationPolicy</code> or the <code>PROCESS_CREATION_MITIGATION_POLICY_WIN32K_SYSTEM_CALL_DISABLE_ALWAYS_ON</code> process-creation attribute flag.
Filter Win32k System Calls	This filters access to the Win32k kernel-mode subsystem driver only to certain APIs allowing simple GUI and Direct X access, mitigating many of the possible attacks, without completely disabling availability of the GUI/GDI services.	This is set through an internal process-creation attribute flag, which can define one out of three possible sets of Win32k filters that are enabled. However, because the filter sets are hard-coded, this mitigation is reserved for Microsoft internal usage.
Disable Extension Points	This prevents a process from loading an input method editor (IME), a Windows hook DLL (<code>SetWindowsHookEx</code>), an app-initialization DLL (<code>AppInitD11s</code> value in the registry), or a Winsock layered service provider (LSP).	This is set with <code>SetProcessMitigationPolicy</code> or the <code>PROCESS_CREATION_MITIGATION_POLICY_EXTENSION_POINT_DISABLE_ALWAYS_ON</code> process-creation attribute flag.
Arbitrary Code Guard (CFG)	This prevents a process from allocating executable code or from changing the permission of existing executable code to make it writeable. It can be configured to allow a particular thread inside the process to request this capability or to allow a remote process from disabling this mitigation, which are not supported from a security point of view.	This is set with <code>SetProcessMitigationPolicy</code> or the <code>PROCESS_CREATION_MITIGATION_POLICY_PROHIBIT_DYNAMIC_CODE_ALWAYS_ON</code> and <code>PROCESS_CREATION_MITIGATION_POLICY_PROHIBIT_DYNAMIC_CODE_ALWAYS_ON_ALLOW_OPT_OUT</code> process-creation attribute flags.
Control Flow Guard (CFG)	This helps prevent memory corruption vulnerabilities from being used to hijack control flow by validating the target of any indirect CALL or JMP instruction against a list of valid expected target functions. Part of Control Flow Integrity (CFI) mechanisms described in the next section.	The image must be compiled with the <code>/guard:cf</code> option, and linked with the <code>/guard:cf</code> option. It can be set with the <code>PROCESS_CREATION_MITIGATION_POLICY_CONTROL_FLOW_GUARD_ALWAYS_ON</code> process-creation attribute flag in case the image does not support it, but CFG enforcement is still desirable for other images loading in the process.
CFG Export Suppression	This strengthens CFG by suppressing indirect calls to the exported API table of the image.	The image must be compiled with <code>/guard: exportsuppress</code> , and can also be configured through <code>SetProcessMitigationPolicy</code> or with the <code>PROCESS_CREATION_MITIGATION_POLICY_CONTROL_FLOW_GUARD_EXPORT_SUPPRESSION</code> process-creation attribute flag.
CFG Strict Mode	This prevents the loading of any image library within the current process that was not linked with the <code>/guard:cf</code> option.	This is set through <code>SetProcessMitigationPolicy</code> or with the <code>PROCESS_CREATION_MITIGATION_POLICY2_STRICT_CONTROL_FLOW_GUARD_ALWAYS_ON</code> process-creation attribute flag.
Disable Non System Fonts	This prevents the loading of any font files that have not been registered by Winlogon at user logon time, after being installed in the <code>C:\windows\fonts</code> directory.	This is set through <code>SetProcessMitigationPolicy</code> or the <code>PROCESS_CREATION_MITIGATION_POLICY_FONT_DISABLE_ALWAYS_ON</code> process-creation attribute flag.
Microsoft-Signed Binaries Only	This prevents the loading of any image library within the current process that has not been signed by a Microsoft CA—issued certificate.	This is set through the <code>PROCESS_CREATION_MITIGATION_POLICY_BLOCK_NON_MICROSOFT_BINARIES_ALWAYS_ON</code> process-attribute flag at startup time.
Store-Signed Binaries Only	This prevents the loading of any image library within the current process that has not been signed by the Microsoft Store CA.	This is set through the <code>PROCESS_CREATION_MITIGATION_POLICY_BLOCK_NON_MICROSOFT_BINARIES_ALLOW_STORE</code> process attribute flag at startup time.

No Remote Images	This prevents the loading of any image library within the current process that is present on a non-local (UNC or WebDAV) path.	This is set through SetProcessMitigationPolicy or the PROCESS_CREATION_MITIGATION_POLICY_IMAGE_LOAD_NO_REMOTE_ALWAYS_ON process-creation attribute flag.
No Low IL Images	This prevents the loading of any image library within the current process that has a mandatory label below medium (0x2000).	This is set through SetProcessMitigationPolicy or the PROCESS_CREATION_MITIGATION_POLICY_IMAGE_LOAD_NO_LOW_LABEL_ALWAYS_ON process-creation flag. It can also be set through a resource claim ACE called IMAGELOAD on the file of the process being loaded.
Prefer System32 Images	This modifies the loader's search path to always look for the given image library being loaded (through a relative name) in the %SystemRoot%\System32 directory, regardless of the current search path.	This is set through SetProcessMitigationPolicy or the PROCESS_CREATION_MITIGATION_POLICY_IMAGE_LOAD_PREFER_SYSTEM32_ALWAYS_ON process-creation attribute flag.
Return Flow Guard (RFG)	This helps prevent additional classes of memory-corruption vulnerabilities that affect control flow by validating, before the execution of a RET instruction, that the function was not called through a return-oriented programming (ROP) exploit by not having begun its execution correctly or by executing on an invalid stack. This is part of the Control Flow Integrity (CFI) mechanisms.	Currently still being implemented in a robust and performant way, this mitigation is not yet available, but is included here for completeness.
Restrict Set Thread Context	This restricts the modification of the current thread's context.	Currently disabled pending the availability of RFG, which makes the mitigation more robust, this mitigation may appear in a future version of Windows. It is included here for completeness.
Loader Continuity	This prohibits the process from dynamically loading any DLLs that do not have the same integrity level as the process, in cases where a signature policy mitigation above could not be enabled at startup time due to compatibility concerns. This specifically targets cases of DLL planting attacks.	This is set through SetProcessMitigationPolicy or the PROCESS_CREATION_MITIGATION_POLICY2_LOADER_INTEGRITY_CONTINUITY_ALWAYS_ON process-creation attribute flag.
Heap Terminate On Corruption	This disables the Fault Tolerant Heap (FTH) and the raising of a continuable exception in the case of heap corruption by terminating the process instead. This prevents the use of heap corruption as a way to force an attacker-controlled exception handler from executing, or in cases where the program ignores the heap exception, or cases where the exploit only sometimes causes heap corruption (limiting its universal effectiveness or reliability).	This is set through HeapSetInformation or by using the PROCESS_CREATION_MITIGATION_POLICY_HEAP_TERMINATE_ALWAYS_ON process-creation attribute flag.
Disable Child Process Creation	This prohibits the creation of child processes by marking the token with a special restriction, which should stop any other component from creating a process while impersonating the token of this process (for example, WMI process creation, or a kernel component creating the process).	This is set through the PROCESS_CREATION_CHILD_PROCESS_RESTRICTED process-creation attribute flag. It can be overridden to allow packaged (UWP) applications with the PROCESS_CREATION_DESKTOP_APPX_OVERRIDE flag.
All Application Packages Policy	This makes an application running under an AppContainer unable to access resources that have an ALL APPLICATION PACKAGES SID present, as was explained earlier in the "AppContainers" section. The presence of an ALL RESTRICTED APPLICATION PACKAGES SID will be required instead. Sometimes referred to as Less Privileged App Container (LPAC).	This is set through the PROC_THREAD_ATTRIBUTE_ALL_APPLICATION_PACKAGES_POLICY process-creation attribute.

TABLE 7-20 Process mitigation options

Note that it is also possible to some of these mitigations on a per-application or per-system basis without the cooperation of the application developer. To do so, open the Local Group Policy Editor. Then expand **Computer Configuration**, then **Administrative Templates**, then **System**, and finally **Mitigation Options** (see [Figure 7-30](#)). In the Process Mitigation Options dialog box, enter the appropriate bit-number value

that corresponds to the mitigations being enabled, using **1** to enable a mitigation, **0** to disable it, or **?** to leave it to its default or process-requested value (again, see [Figure 7-30](#)). The bit numbers are taken from the `PROCESS_MITIGATION_POLICY` enumeration found in the `Winnt.h` header file. This will result in the appropriate registry value being written in the Image File Execution Options (IFEO) key for the entered image name. Unfortunately the current version of Windows 10 Creators Update and earlier will strip out many of the newer mitigations. You can avoid this by manually setting the `REG_DWORD MitigationOptions` registry value.

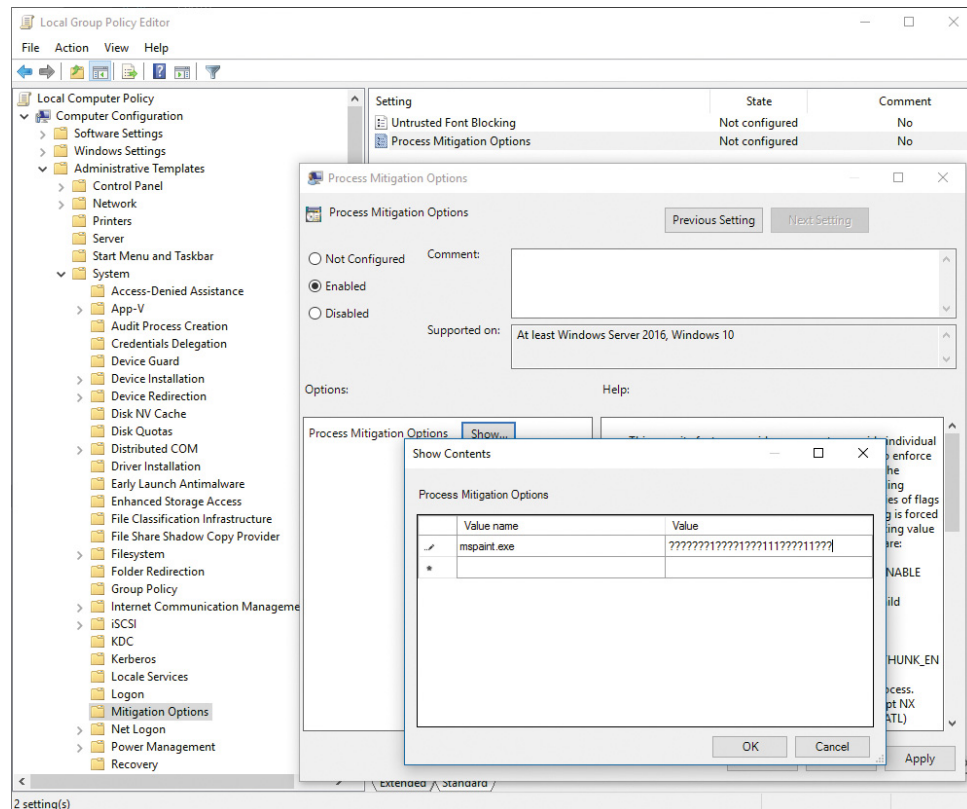


FIGURE 7-30 Customizing process-mitigation options.

Control Flow Integrity

Data Execution Prevention (DEP) and Arbitrary Code Guard (ACG) make it hard for exploits to place executable code on the heap or stack, to allocate new executable code, or to change existing executable code. As a result, memory/data-only attacks have become more interesting. Such attacks allow the modification of portions of memory to redirect control flow, such as modifying return addresses on the stack or indirect function pointers stored in memory. Techniques such as return-oriented-programming (ROP) and jump-oriented-programming (JOP) are often used to violate the regular code flow of the program and redirect it to known locations of interesting code snippets (“gadgets”).

Because such snippets are often present in the middle or end of various functions, when control flow is redirected in this way, it must be redirected into the middle or end of a legitimate function. By employing Control Flow Integrity (CFI) technologies—which can, for example, validate that the target of an indirect JMP or CALL instruction is the beginning of a real function, or that a RET instruction is pointing to an expected location, or that a RET instruction is issued after the function was entered through its beginning—the operating system and compiler can detect and prevent most classes of such exploits.

Control Flow Guard

Control Flow Guard (CFG) is an exploit-mitigation mechanism first introduced in Windows 8.1 Update 3 that exists in enhanced version in Windows 10 and Server 2016, with further improvements released on various updates (up to and including the latest Creators Update). Originally implemented only for user-mode code, CFG now also exists as Kernel CFG (KCFG) on the Creators Update. CFG addresses the indirect CALL/JMP part of CFI by verifying that the target of an indirect call is at the start of a known function (more on that momentarily). If the target is not at the start of a known function, the process is simply terminated.

Figure 7-31 shows the conceptual operation of CFG.

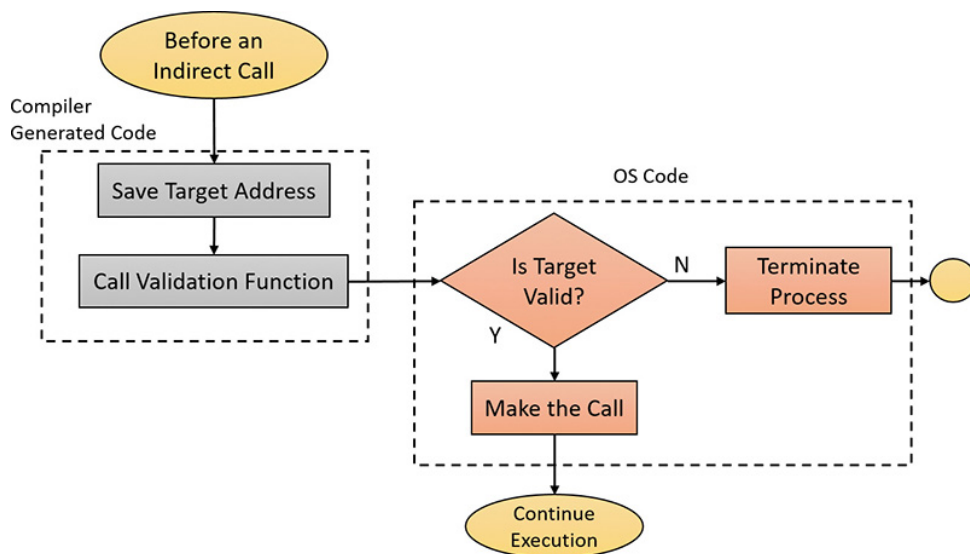


FIGURE 7-31 Conceptual view of Control Flow Guard.

CFG requires the cooperation of a supported compiler that will add the call to the validation code before indirect changes in control flow. The Visual C++ compiler has an option, `/guard:cf`, that must be set for images (or even on a C/C++ source file level) to be built with CFG support (this option is also available in Visual Studio's GUI in the C/C++/Code Generation/Control Flow Guard setting in the project's properties). This

setting should also be set in the linker settings, as both components of Visual Studio are required to collaborate to support CFG.

Once those settings are present, images (EXEs and DLLs) that are compiled with CFG-enabled indicate this in their PE header. In addition, they contain a list of functions that are the valid indirect control flow targets in a .gfids PE section (by default merged by the linker with the .rdata section). This list is built by the linker and contains the relative virtual address (RVA) of all functions in the image. This includes those that might not be called by an indirect call by the code present in the image because there's no way of knowing if outside code does not somehow legitimately know the address of a function and is attempting to call it. This can be especially true of exported functions, which can be called after obtaining their pointer through `GetProcAddress`.

That being said, programmers can use a technique called *CFG suppression*, which is supported through the `DECLSPEC_GUARD_SUPPRESS` annotation, and which marks the function in the table of valid functions with a special flag indicating that the programmer never expects such a function to be the target of any indirect call or jump.

Now that a table of valid function targets exists, all that a simple validation function would need to do is to compare the target of the `CALL` or `JMP` instruction with one of the functions in the table. Algorithmically, this would result in an $O(n)$ algorithm, where the number of functions needed to check would be equivalent, in the worst case, to the number of functions in the table. Clearly, linearly scanning an entire array during every single indirect change in control flow would bring a program to its knees, so operating system support is needed to perform CFG checks efficiently. We'll see in the next section how Windows can achieve this.

The DumpBin Visual Studio tool can show some basic CFG information.
The following dumps header and loader configuration information for
Smss:

[Click here to view code image](#)

```
c:\> dumpbin /headers /loadconfig c:\windows\system32\smss.exe
```

```
Microsoft (R) COFF/PE Dumper Version 14.00.24215.1
```

```
Copyright (C) Microsoft Corporation. All rights reserved.
```

```
Dump of file c:\windows\system32\smss.exe
```

```
PE signature found
```

```
File Type: EXECUTABLE IMAGE
```

```
FILE HEADER VALUES
```

```
    8664 machine (x64)
```

```
    6 number of sections
```

```
57899A7D time date stamp Sat Jul 16 05:22:53 2016
```

```
    0 file pointer to symbol table
```

```
    0 number of symbols
```

```
    F0 size of optional header
```

```
    22 characteristics
```

```
        Executable
```

```
        Application can handle large (>2GB) addresses
```

```
OPTIONAL HEADER VALUES
```

```
    20B magic # (PE32+)
```

```
    14.00 linker version
```

```
    12800 size of code
```

```
    EC00 size of initialized data
```

```
    0 size of uninitialized data
```

```
    1080 entry point (00000000140001080) NtProcessStartupW
```

```
    1000 base of code
```

```
140000000 image base (00000000140000000 to 00000000140024FFF)
```

```
    1000 section alignment
```

```
    200 file alignment
```

```
    10.00 operating system version
```

```
    10.00 image version
```

```
    10.00 subsystem version
```

```
    0 Win32 version
```

```
    25000 size of image
```

```
    400 size of headers
```

```
    270FD checksum
```

1 subsystem (Native)
4160 DLL characteristics
 High Entropy Virtual Addresses
 Dynamic base
 NX compatible
 Control Flow Guard

...

Section contains the following load config:

000000D0 size
 0 time date stamp
0.00 Version
 0 GlobalFlags Clear
 0 GlobalFlags Set
 0 Critical Section Default Timeout
 0 Decommit Free Block Threshold
 0 Decommit Total Free Threshold
0000000000000000 Lock Prefix Table
 0 Maximum Allocation Size
 0 Virtual Memory Threshold
 0 Process Heap Flags
 0 Process Affinity Mask
 0 CSD Version
0800 Dependent Load Flag
0000000000000000 Edit List
0000000140020660 Security Cookie
00000001400151C0 Guard CF address of check-function pointer
00000001400151C8 Guard CF address of dispatch-function pointer
00000001400151D0 **Guard CF function table**
 2A Guard CF function count
00010500 **Guard Flags**
 CF Instrumented
 FID table present
 Long jump target table present
0000 Code Integrity Flags
0000 Code Integrity Catalog
00000000 Code Integrity Catalog Offset
00000000 Code Integrity Reserved
0000000000000000 **Guard CF address taken IAT entry table**
 0 Guard CF address taken IAT entry count
0000000000000000 **Guard CF long jump target table**

0 Guard CF long jump target count

0000000000000000 Dynamic value relocation table

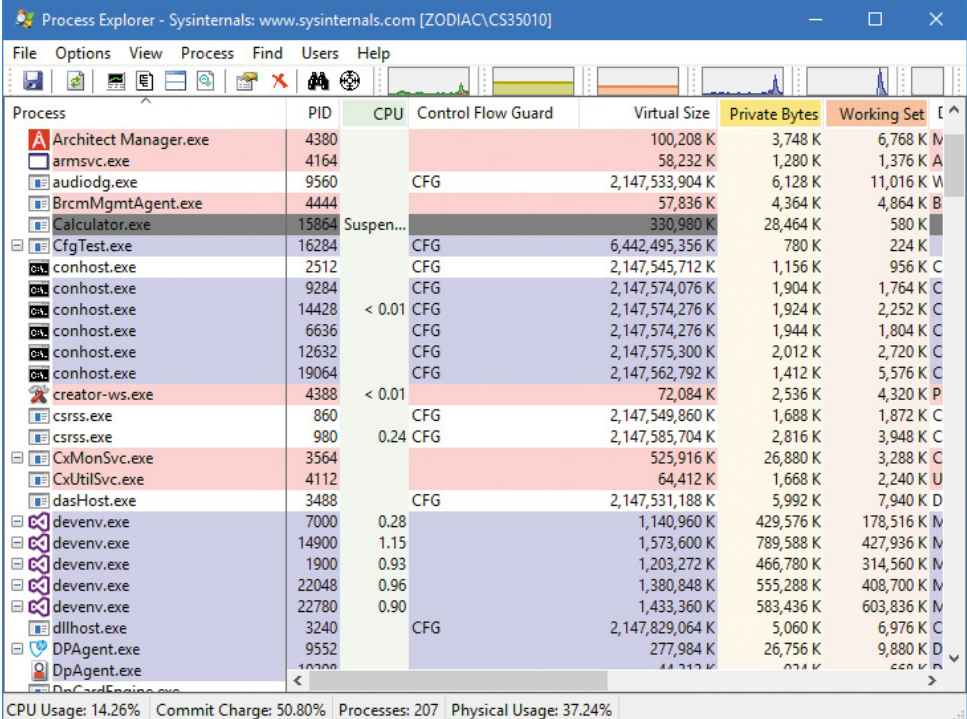
Guard CF Function Table

Address

0000000140001010 _TlgEnableCallback
0000000140001070 SmpSessionComplete
0000000140001080 NtProcessStartupW
0000000140001B30 SmscpLoadSubSystemsForMuSession
0000000140001D10 SmscpExecuteInitialCommand
0000000140002FB0 SmpExecPgm
0000000140003620 SmpStartCsr
00000001400039F0 SmpApiCallback
0000000140004E90 SmpStopCsr

...

The CFG-related information is marked in bold in the preceding output. We will discuss that shortly. For now, open Process Explorer, right-click the process column header, choose **Select Columns**. Then, in the Process Image tab, select the **Control Flow Guard** check box. Also select **Virtual Size** in the Process Memory tab. You should see something like this:



Process	PID	CPU	Control Flow Guard	Virtual Size	Private Bytes	Working Set
Architect Manager.exe	4380			100,208 K	3,748 K	6,768 K
armsvc.exe	4164			58,232 K	1,280 K	1,376 K
audiodg.exe	9560		CFG	2,147,533,904 K	6,128 K	11,016 K
BrcmMgmtAgent.exe	4444			57,836 K	4,364 K	4,864 K
Calculator.exe	15864	Suspen...		330,980 K	28,464 K	580 K
CfgTest.exe	16284		CFG	6,442,495,356 K	780 K	224 K
conhost.exe	2512		CFG	2,147,545,712 K	1,156 K	956 K
conhost.exe	9284		CFG	2,147,574,076 K	1,904 K	1,764 K
conhost.exe	14428	< 0.01	CFG	2,147,574,276 K	1,924 K	2,252 K
conhost.exe	6636		CFG	2,147,574,276 K	1,944 K	1,804 K
conhost.exe	12632		CFG	2,147,575,300 K	2,012 K	2,720 K
conhost.exe	19064		CFG	2,147,562,792 K	1,412 K	5,576 K
creator-ws.exe	4388	< 0.01		72,084 K	2,536 K	4,320 K
csrss.exe	860		CFG	2,147,549,860 K	1,688 K	1,872 K
csrss.exe	980	0.24	CFG	2,147,585,704 K	2,816 K	3,948 K
CxMonSvc.exe	3564			525,916 K	26,880 K	3,288 K
CxUtilSvc.exe	4112			64,412 K	1,668 K	2,240 K
dasHost.exe	3488		CFG	2,147,531,188 K	5,992 K	7,940 K
devenv.exe	7000	0.28		1,140,960 K	429,576 K	178,516 K
devenv.exe	14900	1.15		1,573,600 K	789,588 K	427,936 K
devenv.exe	1900	0.93		1,203,272 K	466,780 K	314,560 K
devenv.exe	22048	0.96		1,380,848 K	555,288 K	408,700 K
devenv.exe	22780	0.90		1,433,360 K	583,436 K	603,836 K
dllhost.exe	3240		CFG	2,147,829,064 K	5,060 K	6,976 K
DPAgent.exe	9552			277,984 K	26,756 K	9,880 K
DpAgent.exe	10700			44,312 K	0 K	560 K

CPU Usage: 14.26% Commit Charge: 50.80% Processes: 207 Physical Usage: 37.24%

You should see most Microsoft-provided processes were built with CFG (including Smss, Csrss, Audiodg, Notepad, and many others). The virtual size for CFG-built processes is surprisingly high. Recall that the virtual

size indicates the total address space used in the process, whether that memory is committed or reserved. In contrast, the Private Bytes column shows the private committed memory and is not even remotely close to the virtual size (although the virtual size includes non-private memory as well). For 64-bit processes, the virtual size is at least 2 TB, which we will shortly be able to rationalize.

The CFG bitmap

As you saw earlier, forcing the program to iterate through a list of function calls every few instructions would not be practical. Therefore, instead of an algorithm that requires linear time $O(n)$, performance requirements dictate that an $O(1)$ algorithm be used instead—one where a constant lookup time is used, regardless of how many functions are present in the table. This constant lookup time should be as small as possible. A clear winner of such a requirement would be an array that is indexable by the target function's address, which is an indication if this address is valid or not (such as a simple `BOOL`). With a 128 TB of possible addresses, though, such an array would itself have to be `128 TB * sizeof(BOOL)`, which is an unacceptable storage size—bigger than the address space itself. Can we do better?

First, we can leverage the fact that compilers ought to generate x64 function code on 16-byte boundaries. This reduces the size to the required array to only `8 TB * sizeof(BOOL)`. But using an entire `BOOL` (which is 4 bytes in the worst case or 1 byte in the best) is extremely wasteful. We only need one state, valid or invalid, which only needs to use 1 bit. This makes the calculation `8 TB / 8`, or simply 1 TB. Unfortunately, however, there's a snag. There's no guarantee that the compiler will generate *all* functions on a 16-byte binary. Hand-crafted assembly code and certain optimizations might violate this rule. As such, we'll have to figure out a solution. One possible option is to simply use another bit to indicate if the function begins *somewhere* on the next 15 bytes instead of on the 16-byte boundary itself. Thus, we have the following possibilities:

- `{0, 0}` No valid function begins inside this 16-byte boundary.
- `{1, 0}` A valid function begins exactly on this aligned 16-byte address.
- `{1, 1}` A valid function begins somewhere inside of this 16-byte address.

Thanks to this setup, if the attacker attempts to call inside a function that was marked as 16-byte aligned by the linker, the 2-bit state will be {1, 0}, while the required bits (that is, bits 3 and 4) in the address will be {1, 1} as the address won't be 16-byte aligned. Therefore, an attacker will only be able to call an arbitrary instruction in the first 16 bytes of the function if the linker did not generate the function aligned in the first place (bits would then be {1, 1}, as shown above). Even then, this instruction must somehow be useful to the attacker without crashing the function (typically some sort of stack pivot or gadget that ends in a `ret` instruction).

With this understanding in mind, we can apply the following formulas to compute the size of the CFG bitmap:

■ **32-bit application on x86 or x64** $2\text{ GB} / 16 * 2 = 32\text{ MB}$

■ **32-bit application with `/LARGEADDRESSAWARE`, booted in 3 GB mode on x86** $3\text{ GB} / 16 * 2 = 48\text{ MB}$

■ **64-bit application** $128\text{ TB} / 16 * 2 = 2\text{ TB}$

■ **32-bit application with `/LARGEADDRESSAWARE`, on x64** $4\text{ GB} / 16 * 2 = 64\text{ MB}$, plus the size of the 64-bit bitmap, which is needed to protect 64-bit Ntdll.dll and WoW64 components, so $2\text{ TB} + 64\text{ MB}$

Allocating and filling out 2 TB of bits on every single process execution is still a tough performance overhead to swallow. Even though we have fixed the execution cost of the indirect call itself, process startup cannot be allowed to take so long, and 2 TB of committed memory would exhaust the commit limit instantly. Therefore, two memory-saving and performance-helping tricks are used.

First, the memory manager will only reserve the bitmap, basing itself on the assumption that the CFG validation function will treat an exception during CFG bitmap access as an indication that the bit state is {0,0}. As such, as long as the region contains 4 KB of bit states that are all {0, 0}, it can be left as reserved, and only pages with at least one bit set {1, X} need to be committed.

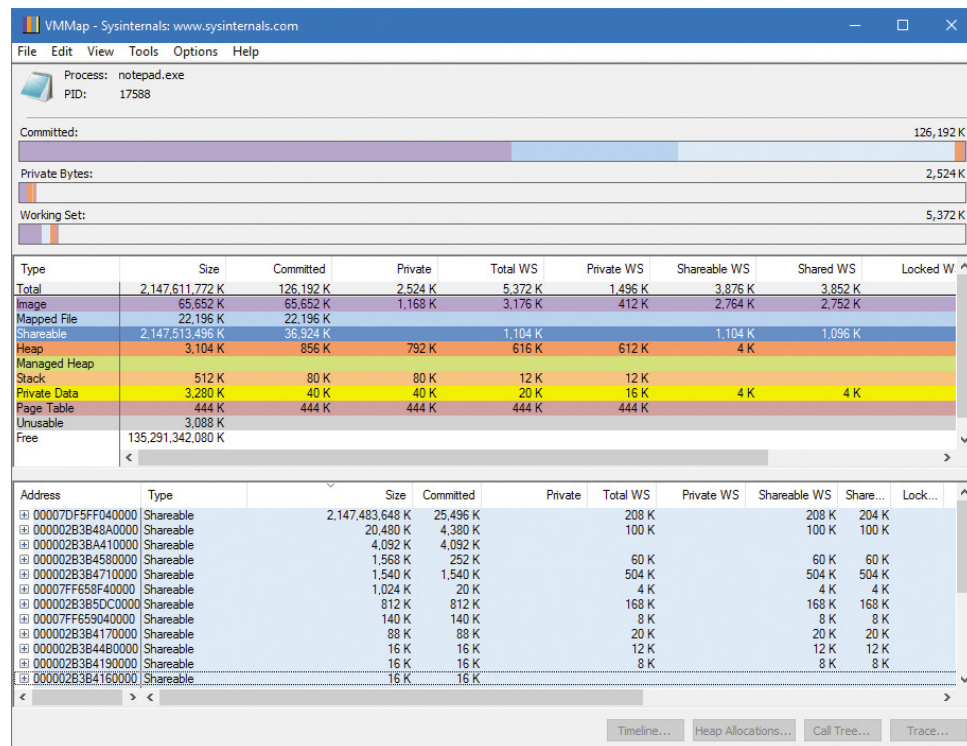
Next, as described in the ASLR section of [Chapter 5, “Memory management,”](#) the system performs the randomization/relocation of libraries typically only once at boot, as a performance-saving measure to avoid repeated relocations. As such, after a library that supports ASLR has been loaded once at a given address, it will always be loaded at that same ad-

dress. This also therefore means that once the relevant bitmap states have been calculated for the functions in that library, they will be identical in all other processes that also load the same binary. As such, the memory manager treats the CFG bitmap as a region of pagefile-backed shareable memory, and the physical pages that correspond to the shared bits only exist in RAM once.

This reduces the cost of the committed pages in RAM and means that only the bits corresponding to private memory need to be calculated. In regular applications, private memory is not executable except in the copy-on-write case where someone has patched a library (but this will not happen at image load), so the cost of loading an application, if it shares the same libraries as other previously launched applications, is almost nil. The next experiment demonstrates this.

EXPERIMENT: CONTROL FLOW GUARD BITMAP

Open the VMMap tool and select a Notepad process. You should see a large reserved block in the Sharable section like so:



You can sort the lower pane by size and quickly locate the large chunk used for the CFGBitmap, as shown. Additionally, if you attach to the process and use the `!address` command on the process, you will see WinDBG identifying the CFG bitmap for you:

[Click here to view code image](#)

```
+ 7df5'ff530000 7df6'0118a000 0'01c5a000 MEM_MAPPED
MEM_RESERVE
```

Other [CFG Bitmap]

```
7df6'0118a000 7df6'011fb000 0'00071000 MEM_MAPPED
MEM_COMMIT PAGE_NOACCESS
```

Other [CFG Bitmap]

```
7df6'011fb000 7ff5'df530000 1ff'de335000 MEM_MAPPED
MEM_RESERVE
```

Other [CFG Bitmap]

```
7ff5'df530000 7ff5'df532000 0'00002000 MEM_MAPPED
MEM_COMMIT PAGE_READONLY
```

Other [CFG Bitmap]

Note how large regions are marked as `MEM_RESERVE`, in between regions that are `MEM_COMMIT`, representing that at least one valid bit state {1, X} is

set. Also, all (or almost all) the regions will be `MEM_MAPPED`, since they belong to the shared bitmap.

CFG bitmap construction

Upon system initialization, the `MiInitializeCfg` function is called to initialize support for CFG. The function creates one or two section objects (`MmCreateSection`) as reserved memory with size appropriate for the platform, as shown earlier. For 32-bit platforms, one bitmap is enough. For x64 platforms, two bitmaps are required—one for 64-bit processes and the other for Wow64 processes (32-bit applications). The section objects' pointers are stored in a substructure within the `MiState` global variable.

After a process is created, the appropriate section is securely mapped into the process's address space. *Securely* here means that the section cannot be unmapped by code running within the process or have its protection changed. (Otherwise, malicious code could just unmap the memory, re-allocate, and fill everything with 1 bits, effectively disabling CFG, or simply modify any bits by marking the region read/write.)

The user mode CFG bitmap(s) are populated in the following scenarios:

- During image mapping, images that have been dynamically relocated due to ASLR (see [Chapter 5](#), for more on ASLR) will have their indirect call target metadata extracted. If an image does not have indirect call target metadata, meaning it was not compiled with CFG, it is assumed that every address within the image can be called indirectly. As explained, because dynamically relocated images are expected to load at the same address in every process, their metadata is used to populate the shared section that is used for the CFG bitmap.
- During image mapping, special care is needed for non-dynamically relocated images and images not being mapped at their preferred base. For these image mappings, the relevant pages of the CFG bitmap are made private and are populated using the CFG metadata from the image. For images whose CFG bits are present in the shared CFG bitmap, a check is made to ensure that all the relevant CFG bitmap pages are still shared. If this is not the case, the bits of the private CFG bitmap pages are populated using the CFG metadata from the image.

- When virtual memory is allocated or re-protected as executable, the relevant pages of the CFG bitmap are made private and initialized to all 1s by default. This is needed for cases such as just-in-time (JIT) compilation, where code is generated on the fly and then executed (for example, .NET or Java).

Strengthening CFG protection

Although CFG does an adequate job to prevent types of exploits that leverage indirect calls or jumps, it could be bypassed through the following ways:

- If the process can be tricked or an existing JIT engine abused to allocate executable memory, all the corresponding bits will be set to {1, 1}, meaning that all memory is considered a valid call target.

- For 32-bit applications, if the expected call target is `__stdcall` (standard calling convention), but an attacker is able to change the indirect call target to `__cdecl` (C calling convention), the stack will become corrupt, as the C call function will not perform cleanup of the caller's arguments, unlike a standard call function. Because CFG cannot differentiate between the different calling conventions, this results in a corrupt stack, potentially with an attacker-controlled return address, bypassing the CFG mitigation.

- Similarly, compiler-generated `setjmp` / `longjmp` targets behave differently from true indirect calls. CFG cannot differentiate between the two.

- Certain indirect calls are harder to protect, such as the Import Address Table (IAT) or Delay-Load Address Table, which is typically in a read-only section of the executable.

- Exported functions may not be desirable indirect function calls.

Windows 10 introduces advancements to CFG that address all these issues. The first is to introduce a new flag to the `VirtualAlloc` function called `PAGE_TARGETS_INVALID` and one to `VirtualProtect` called `PAGE_TARGETS_NO_UPDATE`. With these flags set, JIT engines that allocate executable memory will not see all their allocations' bits set to the {1, 1} state. Instead, they must manually call the `SetProcess-ValidCallTargets` function (which calls the native `NtSetInformationVirtualMemory` function), which will allow them to specify the actual function start addresses of their JITed code.

Additionally, this function is marked as a suppressed call with `DECLSPEC_GUARD_SUPPRESS`, making sure that attackers cannot use an indirect CALL or JMP to redirect into it, even at its function start. (Because it's an inherently dangerous function, calling it with a controlled stack or registers could result in the bypassing of CFG.)

Next, improved CFG changes the default flow you saw in the beginning of this section with a more refined flow. In this flow, the loader does not implement a simple “verify target, return” function, but rather a “verify target, call target, check stack, return” function, which is used in a subset of places on 32-bit applications (and/or running under WoW64). This improved execution flow is shown in **Figure 7-32**.

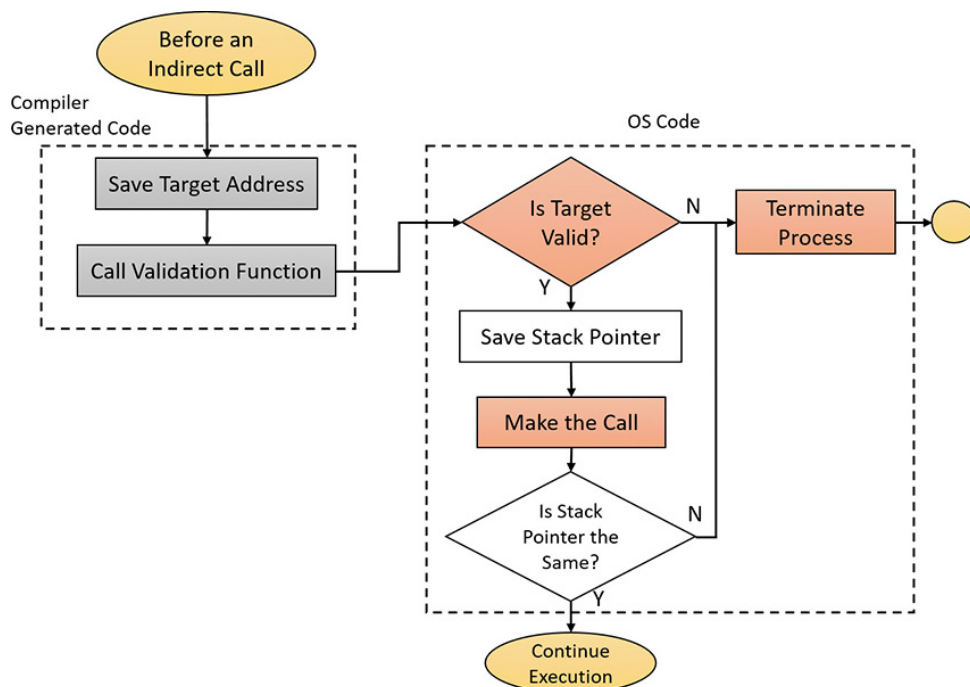


FIGURE 7-32 Improved CFG.

Next, improved CFG adds additional tables inside of the executable, such as the Address Taken IAT table and the Long Jump Address table. When `longjmp` and IAT CFG protection are enabled in the compiler, these tables are used to store destination addresses for these specific types of indirect calls, and the relevant functions are *not* placed in the regular function table, therefore not figuring in the bitmap. This means that if code is attempting to indirect jump/call to one of these functions, it will be treated as an illegal transition. Instead, the C Runtime and linker will validate the targets of, say, the `longjmp` function, by manually checking this table. Although it's more inefficient than a bitmap, there should be little to no functions in these tables, making the cost bearable.

Finally, improved CFG implements a feature called *export suppression*, which must be supported by the compiler and enabled by process-mitigation policy. (See the section “**Process-mitigation policies**” for more on process level mitigations.) With this feature enabled, a new bit state is implemented (recall that bulleted list had {0, 1} as an undefined state). This state indicates that the function is valid but export-suppressed, and it will be treated differently by the loader.

You can determine which features are present in a given binary by looking at the guard flags in the Image Load Configuration Directory, which the DumpBin application used earlier can decode. For reference, they are listed in **Table 7-21**.

TABLE 7-21 Control Flow Guard flags

Loader interaction with CFG

Although it is the memory manager that builds the CFG bitmap, the user-mode loader (see **Chapter 3** for more information) serves two purposes. The first is to dynamically enable CFG support only if the feature is enabled (for example, the caller may have requested no CFG for the child process, or the process itself might not have CFG support). This is done by the `LdrpCfgProcessLoadConfig` loader function, which is called to initialize CFG for each loaded module. If the module `DllCharacteristics` flags in the optional header of the PE does not have the CFG flag set (`IMAGE_DLLCHARACTERISTICS_GUARD_CF`), the `GuardFlags` member of `IMAGE_LOAD_CONFIG_DIRECTORY` structure does not have the `IMAGE_GUARD_CF_INSTRUMENTED` flag set, or the kernel has forcibly turned off CFG for this module, then there is nothing to do.

Second, if the module is indeed using CFG, `LdrpCfgProcessLoadConfig` gets the indirect checking function pointer retrieved from the image (the `GuardCFCheckFunctionPointer` member of `IMAGE_LOAD_CONFIG_DIRECTORY` structure) and sets it to either `LdrpValidateUserCallTarget` or `LdrpValidateUserCallTargetES` in `Ntdll`, depending on whether export suppression is enabled. Additionally, the function first makes sure the indirect pointer has not been somehow modified to point outside the module itself.

Furthermore, if improved CFG was used to compile this binary, a second indirect routine is available, called the *dispatch CFG routine*. It is used to implement the enhanced execution flow described earlier. If the image includes such a function pointer (in the `GuardCFDispatchFunctionPointer` member of the abovementioned structure), it is initialized to `LdrpDispatchUserCallTarget`, or `LdrpDispatchUserCallTargetES` if export suppression is enabled.



NOTE

In some cases, the kernel itself can emulate or perform indirect jumps or calls on behalf of user mode. In situations where this is a possibility, the kernel implements its own `MmValidateUserCallTarget` routine, which performs the same work as `LdrpValidateUserCallTarget`.

The code generated by the compiler when CFG is enabled issues an indirect call that lands in the `LdrpValidateCallTarget(ES)` or `LdrpDispatchUserCallTarget(ES)` functions in `Ntdll`. This function uses the target branch address and checks the bit state value for the function:

- If the bit state is {0, 0}, the dispatch is potentially invalid.
- If the bit state is {1, 0}, and the address is 16-byte aligned, the dispatch is valid. Otherwise, it is potentially invalid.
- If the bit state is {1, 1}, and the address is not 16-byte aligned, the dispatch is valid. Otherwise, it is potentially invalid.
- If the bit state is {0, 1}, the dispatch is potentially invalid.

If the dispatch is potentially invalid, the `RtlpHandleInvalidUserCallTarget` function will execute to determine

the appropriate action. First, it checks if suppressed calls are allowed in the process, which is an unusual application-compatibility option that might be set if Application Verifier is enabled, or through the registry. If so, it will check if the address is suppressed, which is why it was not inserted into the bitmap (recall that a special flag in the guard function table entry indicates this). If this is the case, the call is allowed through. If the function is not valid at all (meaning it's not in the table), then the dispatch is aborted and the process terminated.

Second, a check is made to see if export suppression is enabled. If it is, the target address is checked against the list of export-suppressed addresses, which is once again indicated with another flag that is added in the guard function table entry. If this is the case, the loader validates that the target address is a forwarder reference to the export table of another DLL, which is the only allowed case of an indirect call toward an image with suppressed exports. This is done by a complex check that makes sure the target address is in a different image, that its image load directory has enabled export suppression, and that this address is in the import directory of that image. If these checks match, the kernel is called through the `NtSetInformationVirtualMemory` call described earlier, to change the bit state to {1, 0}. If any of these checks fail, or export suppression is not enabled, then the process is terminated.

For 32-bit applications, an additional check is performed if DEP is enabled for the process. (See [Chapter 5](#) for more on DEP.) Otherwise, because there are no execution guarantees to begin with, the incorrect call is allowed, as it may be an older application calling into the heap or stack for legitimate reasons.

Finally, because large sets of {0, 0} bit states are not committed to save space, if checking the CFG bitmap lands on a reserved page, an access violation exception occurs. On x86, where exception handling setup is expensive, instead of being handled as part of the verification code, it is left to propagate normally. (See Chapter 8 in Part 2 for more on exception dispatching.) The user-mode dispatcher handler, `KiUserExceptionDispatcher`, has specific checks for recognizing CFG bitmap access violation exceptions within the validation function and will automatically resume execution if the exception code was `STATUS_IN_PAGE_ERROR`. This simplifies the code in `LdrpValidateUserCallTarget(ES)` and `LdrpDispatchUserCallTarget(ES)`, which don't have to include exception handling code. On x64, where exception handlers are simply regis-

tered in tables, the `LdrpICallHandler` handler runs instead, with the same logic as above.

Kernel CFG

Although drivers compiled with Visual Studio and `/guard:cf` also ended up with the same binary properties as user-mode images, the first versions of Windows 10 did not do anything with this data. Unlike the user-mode CFG bitmap, which is protected by a higher, more trusted entity (the kernel), there is nothing that can truly “protect” the kernel CFG bitmap if one were to be created. A malicious exploit could simply edit the PTE that corresponded to the page containing the desired bits to modify, mark it as read/write, and proceed with the indirect call or jump. Therefore, the overhead of setting up such a trivially bypassable mitigation was simply not worth it.

With a greater number of users enabling VBS features, once again, the higher security boundary that VTL 1 provides can be leveraged. The SLAT page table entries come to the rescue by providing a second boundary against PTE page protection changes. While the bitmap is readable to VTL 0 because the SLAT entries are marked as read only, if a kernel attacker attempts to change the PTEs to mark them read/write, they cannot do the same to the SLAT entries. As such, this will be detected as an invalid KCFG bitmap access which HyperGuard can act on (for telemetry reasons alone—since the bits can’t be changed anyway).

KCFG is implemented almost identically to regular CFG, except that export suppression is not enabled, nor is `longjmp` support, nor is the ability to dynamically request additional bits for JIT purposes. Kernel drivers should not be doing any of these things. Instead, the bits are set in the bitmap based on the “address taken IAT table” entries, if any are set; by the usual function entries in the guard table each time a driver image is loaded; and for the HAL and kernel during boot by `MiInitializeKernelCfg`. If the hypervisor is not enabled, and SLAT support is not present, then none of this will be initialized, and Kernel CFG will be kept disabled.

Just like in the user-mode case, a dynamic pointer in the load configuration data directory is updated, which in the enabled case will point to `__guard_check_icall` for the check function and `__guard_dispatch_icall` for the dispatch function in enhanced CFG mode. Additionally, a variable named `guard_icall_bitmap` will hold the virtual address of the bitmap.

One last detail on Kernel CFG is that unfortunately, dynamic Driver Verifier settings will not be configurable (for more information on Driver Verifier, see [Chapter 6, “I/O system”](#)), as this would require adding dynamic kernel hooks and redirecting execution to functions that may not be in the bitmap. In this case, `STATUS_VRF_CFG_ENABLED (0xC000049F)` will be returned, and a reboot is required (at which time the bitmap can be built with the Verifier Driver hooks in place).

Security assertions

Earlier, we described how Control Flow Guard will terminate the process. We also explained how certain other mitigations or security features will raise an exception to kill the process. It is important to be accurate with what *exactly* happens during these security violations because both these descriptions hide important details about the mechanism.

In fact, when a security-related breach occurs, such as when CFG detects an incorrect indirect call or jump, terminating the process through the standard `TerminateProcess` mechanism would not be an adequate path. There would be no crash generated, and no telemetry sent to Microsoft. These are both important tools for the administrator to understand that a potential exploit has executed or that an application compatibility issue exists, as well as for Microsoft to track zero-day exploitation in the wild. On the flip side, while raising an exception would achieve the desired result, exceptions are callbacks, which can be:

- Potentially hooked by attackers if `/SAFESEH` and `SEHOP` mitigations are not enabled, causing the security check to be the one that gives control to an attacker in the first place—or an attacker can simply “swallow” the exception.
- Potentially hooked by legitimate parts of the software through an unhandled exception filter or vectored exception handler, both of which might accidentally swallow the exception.
- Same as above, but intercepted by a third-party product that has injected its own library into the process. Common to many security tools, this can also lead in the exception not being correctly delivered to Windows Error Reporting (WER).
- A process might have an application recovery callback registered with WER. This might then display a less clear UI to the user, and might restart

the process in its current exploited state, leading anywhere from a recursive crash/start loop to the exception being swallowed whole.

■ Likely in a C++-based product, caught by an outer exception handler as if “thrown” by the program itself, which, once again, might swallow the exception or continue execution in an unsafe manner.

Solving these issues requires a mechanism that can raise an exception that cannot be intercepted by any of the process’s components outside of the WER service, which must itself be guaranteed to receive the exception. This is where security assertions come into play.

Compiler and OS support

When Microsoft libraries, programs, or kernel components encounter unusual security situations, or when mitigations recognize dangerous violations of security state, they now use a special compiler intrinsic supported by Visual Studio, called `__fastfail`, which takes one parameter as input. Alternatively, they can call a runtime library (`Rtl`) function in `Ntdll` called `RtlFailFast2`, which itself contains a `__fastfail` intrinsic. In some cases, the WDK or SDK contain inline functions that call this intrinsic, such as when using the `LIST_ENTRY` functions `InsertTailList` and `RemoveEntryList`. In other situations, it is the Universal CRT (uCRT) itself that has this intrinsic in its functions. In yet others, APIs will do certain checks when called by applications and may use this intrinsic as well.

Regardless of the situation, when the compiler sees this intrinsic, it generates assembly code that takes the input parameter, moves it into the RCX (x64) or ECX (x86) register, and then issues a software interrupt with the number 0x29. (For more information on interrupts, see Chapter 8 in Part 2.)

In Windows 8 and later, this software interrupt is registered in the Interrupt Dispatch Table (IDT) with the handler `KiRaiseSecurityCheckFailure`, which you can verify on your own by using the `!idt 29` command in the debugger. This will result (for compatibility reasons) in `KiFastFailDispatch` being called with the `STATUS_STACK_BUFFER_OVERRUN` status code (`0xC0000409`). This will then do regular exception dispatching through `KiDispatchException`, but treat this as a second-chance exception, which means that the debugger and process won’t be notified.

This condition will be specifically recognized and an error message will be sent to the WER error ALPC port as usual. WER will claim the exception as non-continuable, which will then cause the kernel to terminate the process with the usual `ZwTerminateProcess` system call. This, therefore, guarantees that once the interrupt is used, no return to user mode will ever be performed within this process again, that WER will be notified, and that the process will be terminated (additionally, the error code will be the exception code). When the exception record is generated, the first exception argument will be the input parameter to `__fastfail`.

Kernel-mode code can also raise exceptions, but in this case `KiBugCheckDispatch` will be called instead, which will result in a special kernel mode crash (bugcheck) with code `0x139` (`KERNEL_SECURITY_CHECK_FAILURE`), where the first argument will be the input parameter to `__fastfail`.

Fast fail/security assertion codes

Because the `__fastfail` intrinsic contains an input argument that is bubbled up to the exception record or crash screen, it allows the failing check to identify what part of the system or process is not working correctly or has encountered a security violation. **Table 7-22** shows the various failure conditions and their meaning or significance.

TABLE 7-22 `__fastfail` failure codes

Application Identification

Historically, security decisions in Windows have been based on a user's identity (in the form of the user's SID and group membership), but a growing number of security components (AppLocker, firewall, antivirus, anti-malware, Rights Management Services, and others) need to make security decisions based on what code is to be run. In the past, each of these security components used their own proprietary method for identifying applications, which led to inconsistent and overly complicated policy authoring. The purpose of Application Identification (AppID) is to bring consistency to how the security components recognize applications by providing a single set of APIs and data structures.



This is not the same as the AppID used by DCOM/COM+ applications, where a GUID represents a process that is shared by multiple CLSIDs, nor is it related to UWP application ID.

Just as a user is identified when she logs in, an application is identified just before it is started by generating the main program's AppID. An AppID can be generated from any of the following attributes of the application:

■ **Fields** Fields within a code-signing certificate embedded within the file allow for different combinations of publisher name, product name, file name, and version. `APPID://FQBN` is a fully qualified binary name, and it is a string in the following form:

`{Publisher\Product\Filename,Version}`. `Publisher` is the Subject field of the x.509 certificate used to sign the code, using the following fields:

- **O** Organization
- **L** Locality
- **S** State or province
- **C** Country

■ **File hash** there are several methods that can be used for hashing. The default is `APPID://SHA256HASH`. However, for backward compatibility

with SRP and most x.509 certificates, SHA-1 (`APPID://SHA1HASH`) is still supported. `APPID://SHA256HASH` specifies the SHA-256 hash of the file.

■ **The partial or complete path to the file** `APPID://Path` specifies a path with optional wildcard characters (`*`).



An AppID does not serve as a means for certifying the quality or security of an application. An AppID is simply a way of identifying an application so that administrators can reference the application in security policy decisions.

The AppID is stored in the process access token, allowing any security component to make authorization decisions based on a single consistent identification. AppLocker uses conditional ACEs (described earlier) for specifying whether a particular program is allowed to be run by the user.

When an AppID is created for a signed file, the certificate from the file is cached and verified to a trusted root certificate. The certificate path is reverified daily to ensure the certificate path remains valid. Certificate caching and verification are recorded in the system event log at Application and Services Logs\Microsoft\Windows\AppID\Operational.

AppLocker

Windows 8.1 and Windows 10 (Enterprise editions) and Windows Server 2012/R2/2016 support a feature known as *AppLocker*, which allows an administrator to lock down a system to prevent unauthorized programs from being run. Windows XP introduced Software Restriction Policies (SRP), which was the first step toward this capability, but SRP was difficult to manage, and it couldn't be applied to specific users or groups. (All users were affected by SRP rules.) AppLocker is a replacement for SRP, and yet coexists alongside SRP, with AppLocker's rules being stored separately from SRP's rules. If both AppLocker and SRP rules are in the same Group Policy object (GPO), only the AppLocker rules will be applied.

Another feature that makes AppLocker superior to SRP is AppLocker's auditing mode, which allows an administrator to create an AppLocker policy and examine the results (stored in the system event log) to determine whether the policy will perform as expected—without actually performing the restrictions. AppLocker auditing mode can be used to monitor which applications are being used by one or more users on a system.

AppLocker allows an administrator to restrict the following types of files from being run:

- Executable images (EXE and COM)
- Dynamic-link libraries (DLL and OCX)
- Microsoft Software Installer (MSI and MSP) for both install and uninstall
- Scripts
- Windows PowerShell (PS1)
- Batch (BAT and CMD)
- VisualBasic Script (VBS)
- Java Script (JS)

AppLocker provides a simple GUI rule-based mechanism, which is very similar to network firewall rules, for determining which applications or scripts are allowed to be run by specific users and groups, using condi-

tional ACEs and AppID attributes. There are two types of rules in AppLocker:

- Allow the specified files to run, denying everything else.
- Deny the specified files from being run, allowing everything else. Deny rules take precedence over allow rules.

Each rule can also have a list of exceptions to exclude files from the rule. Using an exception, you could create a rule to, for example, allow everything in the C:\Windows or C:\Program Files directories to be run except RegEdit.exe.

AppLocker rules can be associated with a specific user or group. This allows an administrator to support compliance requirements by validating and enforcing which users can run specific applications. For example, you can create a rule to allow users in the Finance security group to run the finance line-of-business applications. This blocks everyone who is not in the Finance security group from running finance applications (including administrators) but still provides access for those who have a business need to run the applications. Another useful rule would be to prevent users in the Receptionists group from installing or running unapproved software.

AppLocker rules depend upon conditional ACEs and attributes defined by AppID. Rules can be created using the following criteria:

- **Fields within a code-signing certificate embedded within the file, allowing for different combinations of publisher name, product name, file name, and version** For example, a rule could be created to allow all versions greater than 9.0 of Contoso Reader to run or allow anyone in the Graphics group to run the installer or application from Contoso for GraphicsShop as long as the version is 14.*. For example, the following SDDL string denies execute access to any signed programs published by Contoso for the RestrictedUser user account (identified by the user's SID):

[Click here to view code image](#)

```
D:(XD;;FX;;;S-1-5-21-3392373855-1129761602-2459801163-1028;((Exists
APPID://FQBN)
&& ((APPID://FQBN) >= ({"O=CONTOSO, INCORPORATED, L=REDMOND,
S=CWASHINGTON, C=US\*",0}))))
```

■ **Directory path, allowing only files within a particular directory**

tree to run This can also be used to identify specific files. For example, the following SDDL string denies execute access to the programs in the directory C:\Tools for the RestrictedUser user account (identified by the user's SID):

[Click here to view code image](#)

```
D:(XD;;FX;;;S-1-5-21-3392373855-1129761602-2459801163-1028;  
(APPID://PATH  
Contains "%OSDRIVE%\TOOLS\*"))
```

■ **File hash** Using a hash will also detect if a file has been modified and prevent it from running. This can also be a weakness if files are changed frequently because the hash rule will need to be updated frequently. File hashes are often used for scripts because few scripts are signed. For example, this SDDL string denies execute access to programs with the specified hash values for the RestrictedUser user account (identified by the user's SID):

[Click here to view code image](#)

```
D:(XD;;FX;;;S-1-5-21-3392373855-1129761602-2459801163-1028;  
(APPID://SHA256HASH  
Any_of  
{#7a334d2b99d48448eedd308dfca63b8a3b7b44044496ee2f8e236f5997f1b647,  
#2a782f76cb94ece307dc52c338f02edbbfdca83906674e35c682724a8a92a76b}))
```

AppLocker rules can be defined on the local machine using the Security Policy MMC snap-in (secpol.msc, see [Figure 7-33](#)) or a Windows PowerShell script, or they can be pushed to machines within a domain using Group Policy. AppLocker rules are stored in multiple locations within the registry:

■ **HKLM\Software\Policies\Microsoft\Windows\SrpV2** This key is also mirrored to HKLM\SOFTWARE\Wow6432Node\Policies\Microsoft\Windows\SrpV2. The rules are stored in XML format.

■ **HKLM\SYSTEM\CurrentControlSet\Control\Srp\Gp\Exe** The rules are stored as SDDL and a binary ACE.



**HKEY_CURRENT_USER\Software\Microsoft\Windows\CurrentVersion\Group
Policy Objects**

{GUID}Machine\Software\Policies\Microsoft\Windows\SrpV2

AppLocker policy pushed down from a domain as part of a GPO are stored here in XML format.

FIGURE 7-33 AppLocker configuration page in Local Security Policy.

Certificates for files that have been run are cached in the registry under the key HKLM\SYSTEM\CurrentControlSet\Control\AppID\CertStore.

AppLocker also builds a certificate chain (stored in HKLM\SYSTEM\CurrentControlSet\Control\AppID\CertChainStore) from the certificate found in a file back to a trusted root certificate.

There are also AppLocker-specific PowerShell commands (cmdlets) to enable deployment and testing via scripting. After using the Import-Module AppLocker to get AppLocker cmdlets into PowerShell, several cmdlets are available. These include `Get-AppLockerFileInformation`, `Get-AppLockerPolicy`, `New-AppLockerPolicy`, `Set-AppLockerPolicy`, and `Test-AppLockerPolicy`.

The AppID and SRP services coexist in the same binary (AppIdSvc.dll), which runs within an `SvcHost` process. The service requests a registry change notification to monitor any changes under that key, which is written by either a GPO or the AppLocker UI in the Local Security Policy MMC snap-in. When a change is detected, the AppID service triggers a user-mode task (AppIdPolicyConverter.exe), which reads the new rules (de-

scribed with XML) and translates them into binary format ACEs and SDDL strings, which are understandable by both the user-mode and kernel-mode AppID and AppLocker components. The task stores the translated rules under HKLM\SYSTEM\CurrentControlSet\Control\Srp\Gp. This key is writable only by System and Administrators, and it is marked read-only for authenticated users. Both user-mode and kernel-mode AppID components read the translated rules from the registry directly. The service also monitors the local machine trusted root certificate store, and it invokes a user-mode task (AppIdCertStoreCheck.exe) to reverify the certificates at least once per day and whenever there is a change to the certificate store. The AppID kernel-mode driver (%SystemRoot%\System32\drivers\AppId.sys) is notified about rule changes by the AppID service through an APPID_POLICY_CHANGED DeviceIoControl request.

An administrator can track which applications are being allowed or denied by looking at the system event log using Event Viewer (once AppLocker has been configured and the service started). See [Figure 7-34](#).

FIGURE 7-34 Event Viewer showing AppLocker allowing and denying access to various applications. Event ID 8004 is denied; 8002 is allowed.

The implementations of AppID, AppLocker, and SRP are somewhat blurred and violate strict layering, with various logical components coexisting within the same executables, and the naming is not as consistent as one would like.

The AppID service runs as LocalService so that it has access to the Trusted Root Certificate Store on the system. This also enables it to per-

form certificate verification. The AppID service is responsible for the following:

- Verification of publisher certificates
- Adding new certificates to the cache
- Detecting AppLocker rule updates and notifying the AppID driver

The AppID driver performs the majority of the AppLocker functionality and relies on communication (via `DeviceIoControl` requests) from the AppID service, so its device object is protected by an ACL, granting access only to the NT SERVICE\AppIDSvc, LOCAL SERVICE and BUILTIN\Administrators groups. Thus, the driver cannot be spoofed by malware.

When the AppID driver is first loaded, it requests a process-creation callback by calling `PsSetCreateProcessNotifyRoutineEx`. When the notification routine is called, it is passed a `PPS_CREATE_NOTIFY_INFO` structure (describing the process being created). It then gathers the AppID attributes that identify the executable image and writes them to the process's access token. Then it calls the undocumented routine `SeSrpAccessCheck`, which examines the process token and the conditional ACE AppLocker rules, and determines whether the process should be allowed to run. If the process should not be allowed to run, the driver writes `STATUS_ACCESS_DISABLED_BY_POLICY_OTHER` to the `Status` field of the `PPS_CREATE_NOTIFY_INFO` structure, which causes the process creation to be canceled (and sets the process's final completion status).

To perform DLL restriction, the image loader sends a `DeviceIoControl` request to the AppID driver whenever it loads a DLL into a process. The driver then checks the DLL's identity against the AppLocker conditional ACEs, just like it would for an executable.



Performing these checks for every DLL load is time-consuming and might be noticeable to end users. For this reason, DLL rules are normally disabled, and they must be specifically enabled via the Advanced tab in the AppLocker properties page in the Local Security Policy snap-in.

The scripting engines and the MSI installer have been modified to call the user-mode SRP APIs whenever they open a file, to check whether a file is allowed to be opened. The user-mode SRP APIs call the AuthZ APIs to perform the conditional ACE access check.

Software Restriction Policies

Windows contains a user-mode mechanism called *Software Restriction Policies* (SRP) that enables administrators to control what images and scripts execute on their systems. The Software Restriction Policies node of the Local Security Policy editor, shown in [Figure 7-35](#), serves as the management interface for a machine's code execution policies, although per-user policies are also possible using domain group policies.

FIGURE 7-35 Software Restriction Policy configuration.

Several global policy settings appear beneath the Software Restriction Policies node:

- **Enforcement** This policy configures whether restriction policies apply to libraries, such as DLLs, and whether policies apply to users only or to administrators as well.

- **Designated File Types** This policy records the extensions for files that are considered executable code.

- **Trusted Publishers** This policy controls who can select which certificate publishers are trusted.

When configuring a policy for a particular script or image, an administrator can direct the system to recognize it using its path, its hash, its Internet zone (as defined by Internet Explorer), or its cryptographic certificate, and can specify whether it is associated with the Disallowed or Unrestricted security policy.

Enforcement of SRPs takes place within various components where files are treated as containing executable code. Some of these components are listed here:

- The user-mode Windows `CreateProcess` function in `Kernel32.dll` enforces it for executable images.

- The DLL loading code in `Ntdll` enforces it for DLLs.

- The Windows command prompt (`Cmd.exe`) enforces it for batch file execution.

- Windows Scripting Host components that start scripts—`Cscript.exe` (for command-line scripts), `Wscript.exe` (for UI scripts), and `Scrobj.dll` (for script objects)—enforce it for script execution.

- The PowerShell host (`PowerShell.exe`) enforces it for PowerShell script execution.

Each of these components determines whether the restriction policies are enabled by reading the `TransparentEnabled` registry value in the `HKLM\Software\Policies\Microsoft\Windows\Safer\CodeIdentifiers` key, which if set to `1` indicates that policies are in effect. Then it determines whether the code it's about to execute matches one of the rules specified in a subkey of the `CodeIdentifiers` key and, if so, whether the execution should be allowed. If there is no match, the default policy, as specified in

the `DefaultLevel` value of the `CodeIdentifiers` key, determines whether the execution is allowed.

Software Restriction Policies are a powerful tool for preventing the unauthorized access of code and scripts, but only if properly applied. Unless the default policy is set to disallow execution, a user can make minor changes to an image that's been marked as disallowed so that he can bypass the rule and execute it. For example, a user can change an innocuous byte of a process image so that a hash rule fails to recognize it, or copy a file to a different location to avoid a path-based rule.

You can indirectly see SRPs being enforced by watching accesses to the registry when you attempt to execute an image that you've disallowed.

1. Run `secpol.msc` to open the Local Security Policy editor and navigate to the Software Restriction Policies node.
2. Choose **Create New Policies** from the context menu if no policies are defined.
3. Create a path-based disallow restriction policy (under the Additional Rules node) for `%SystemRoot%\System32\notepad.exe`.
4. Run Process Monitor and set an include a Path filter for Safer.
5. Open a command prompt and run Notepad from the prompt.

Your attempt to run Notepad should result in a message telling you that you cannot execute the specified program, and Process Monitor should show the command prompt (`cmd.exe`) querying the local machine restriction policies.

Kernel Patch Protection

Some device drivers modify the behavior of Windows in unsupported ways. For example, they patch the system call table to intercept system calls or patch the kernel image in memory to add functionality to specific internal functions. Such modifications are inherently dangerous and can reduce system stability and security. Additionally, it is also possible for such modifications to be made with malicious intent, either by rogue drivers or through exploits due to vulnerabilities in Windows drivers.

Without the presence of a more privileged entity than the kernel itself, detecting and protecting against kernel-based exploits or drivers from within the kernel itself is a tricky game. Because both the detection/protection mechanism and the unwanted behavior operate in ring 0, it is not possible to define a security boundary in the true sense of the word, as the unwanted behavior could itself be used to disable, patch, or fool the detection/prevention mechanism. That being said, even in such conditions, a mechanism to react to such unwanted operations can still be useful in the following ways:

■ By crashing the machine with a clearly identifiable kernel-mode crash dump, both users and administrators can easily see that an unwanted behavior has been operating inside of their kernel and they can take action. Additionally, it means that legitimate software vendors will not want to take the risk of crashing their customers' systems and will find supported ways of extending kernel functionality (such as by using the filter manager for file system filters or other callback-based mechanisms).

■ Obfuscation (which is not a security boundary) can make it costly—either in time or in complexity—for the unwanted behavior to disable the detection mechanism. This added cost means that the unwanted behavior is more clearly identified as potentially malicious, and that its complexity results in additional costs to a potential attacker. By shifting the obfuscation techniques, it means that legitimate vendors will be better off taking the time to move away from their legacy extension mechanisms and implement supported techniques instead, without the risk of looking like malware.

■ Randomization and non-documentation of which specific checks the detection/prevention mechanism makes to monitor kernel integrity, and non-determinism of when such checks are executed, cripple the ability of attackers to ensure their exploits are reliable. It forces them to account for every possible non-deterministic variable and state transition that the mechanism has through static analysis, which obfuscation makes nearly impossible within the timeframe required before another obfuscation shift or feature change is implemented in the mechanism.

■ Because kernel mode crash dumps are automatically submitted to Microsoft, it allows the company to receive telemetry of in-the-wild unwanted code, and to either identify software vendors whose code is unsupported and is crashing systems, or to track the progress of malicious drivers in the wild, or even zero-day kernel-mode exploitations, and fix bugs that may not have been reported, but are actively exploited.

PatchGuard

Shortly after the release of 64-bit Windows for x64 and before a rich third-party ecosystem had developed, Microsoft saw an opportunity to preserve the stability of 64-bit Windows, and to add telemetry and exploit-crippling patch detection to the system, through a technology called *Kernel Patch Protection (KPP)*, also referred to as *PatchGuard*. When Windows Mobile was released, which operates on a 32-bit ARM processor core, the feature was ported to such systems, too, and it will be present in 64-bit ARM (AArch64) systems as well. Due to the existence of too many legacy 32-bit drivers that still use unsupported and dangerous hooking techniques, however, this mechanism is not enabled on such systems, even on Windows 10 operating systems. Fortunately, usage of 32-bit systems is almost coming to an end, and server versions of Windows no longer support this architecture at all.

Although both *Guard* and *Protection* imply that this mechanism will protect the system, it is important to realize that the only guard/protection offered is the crashing of the machine, which prevents further execution of the unwanted code. The mechanism does not *prevent* the attack in the first place, nor mitigate against it, nor undo it. Think of KPP as an Internet-connected video security system, or CCTV, with a loud alarm (the crash) inside the vault (the kernel), not as an impenetrable lock on the vault.

KPP has a variety of checks that it makes on protected systems, and documenting them all would both be impractical (due to the difficulty of static analysis) and valuable to potential attackers (reducing their research time). However, Microsoft does document certain checks, which we generalize in [Table 7-23](#). When, where, and how KPP makes these checks, and which specific functions or data structures are affected, is outside of the scope of this analysis.

TABLE 7-23 Generalized description of elements protected by KPP

As mentioned, when KPP detects unwanted code on the system, it crashes the system with an easily identifiable code. This corresponds to bugcheck

code `0x109`, which stands for `CRITICAL_STRUCTURE_CORRUPTION`, and the Windows Debugger can be used to analyze this crash dump. (See Chapter 15, “Crash dump analysis,” in Part 2 for more information.) The dump information will contain *some* information about the corrupted or scrump-tuously modified part of the kernel, but any additional data must be analyzed by Microsoft’s Online Crash Analysis (OCA) and/or Windows Error Reporting (WER) teams and is not exposed to users.

For third-party developers who use techniques that KPP deters, the following supported techniques can be used:

- **File system (mini) filters** Use these to hook all file operations, including loading image files and DLLs, that can be intercepted to purge malicious code on-the-fly or block reading of known bad executables or DLLs. (See Chapter 13, “File systems,” in Part 2 for more information on these.)

- **Registry filter notifications** Use these to hook all registry operations. (See Chapter 9 in Part 2 for more information on these notifications.) Security software can block modification of critical parts of the registry, as well as heuristically determine malicious software by registry access patterns or known bad registry keys.

- **Process notifications** Security software can monitor the execution and termination of all processes and threads on the system, as well as DLLs being loaded or unloaded. With the enhanced notifications added for antivirus and other security vendors, they also can block process launch. (See [Chapter 3](#) for more information on these notifications.)

- **Object manager filtering** Security software can remove certain access rights being granted to processes and/or threads to defend their own utilities against certain operations. (These are discussed in Chapter 8 in Part 2.)

- **NDIS Lightweight Filters (LWF) and Windows Filtering Platform (WFP) filters** Security software can intercept all socket operations (accept, listen, connect, close, and so on) and even the packets themselves. With LWF, security vendors have access to the raw Ethernet frame data that is going from the network card (NIC) to the wire.

- **Event Tracing for Windows (ETW)** Through ETW, many types of operations that have interesting security properties can be consumed by a user-mode component, which can then react to data in near real-time. In

certain cases, special secure ETW notifications are available to anti-malware-protected processes under NDA with Microsoft and participation in various security programs, which give access to a greater set of tracing data. (ETW is discussed in Chapter 8 in Part 2.)

HyperGuard

On systems that run with virtualization-based security (described earlier in this chapter in the section “**Virtualization-based security**”), it is no longer true that attackers with kernel-mode privileges are essentially running at the same security boundary as a detection/prevention mechanism. In fact, such attackers would operate at VTL 0, while a mechanism could be implemented in VTL 1. In the Anniversary Update of Windows 10 (version 1607), such a mechanism does indeed exist, which is appropriately named *HyperGuard*. HyperGuard has a few interesting properties that set it apart from PatchGuard:

- It does not need to rely on obfuscation. The symbol files and function names that implement HyperGuard are available for anyone to see, and the code is not obfuscated. Complete static analysis is possible. This is because HyperGuard is a true security boundary.
- It does not need to operate non-deterministically because this would provide no advantage due to the preceding property. In fact, by operating deterministically, HyperGuard can crash the system at the precise time unwanted behavior is detected. This means crash data will contain clear and actionable data for the administrator (and Microsoft’s analysis teams), such as the kernel stack, which will show the code that performed the undesirable behavior.
- Due to the preceding property, it can detect a wider variety of attacks, because the malicious code does not have the chance to restore a value back to its correct value during a precise time window, which is an unfortunate side-effect of PatchGuard’s non-determinism.

HyperGuard is also used to extend PatchGuard’s capabilities in certain ways, and to strengthen its ability to run undetected by attackers trying to disable it. When HyperGuard detects an inconsistency, it too will crash the system, albeit with a different code: `0x18C (HYPERGUARD_VIOLATION)`. As before, it might be valuable to understand, at a generic level, what kind of things HyperGuard will detect, which you can see in [Table 7-24](#).

TABLE 7-24 Generalized description of elements protected by HyperGuard

On systems with VBS enabled, there is another security-related feature that is worth describing, which is implemented in the hypervisor itself: Non-Privileged Instruction Execution Prevention (NPIEP). This mitigation targets specific x64 instructions that can be used to leak the kernel-mode addresses of the GDT, IDT, and LDT, which are SGDT, SIDT, and SLDT. With NPIEP, these instructions are still allowed to execute (due to compatibility concerns), but will return a per-processor unique number that is not actually the kernel address of these structures. This serves as a mitigation against Kernel ASLR (KASLR) information leaks from local attackers.

Finally, note that there is no way to disable PatchGuard or HyperGuard once they are enabled. However, because device-driver developers might need to make changes to a running system as part of debugging, PatchGuard is not enabled when the system boots in debugging mode with an active remote kernel-debugging connection. Similarly, HyperGuard is disabled if the hypervisor boots in debugging mode with a remote debugger attached.

Conclusion

Windows provides an extensive array of security functions that meet the key requirements of both government agencies and commercial installations. In this chapter, we've taken a brief tour of the internal components that are the basis of these security features. In Chapter 8 of Part 2, we'll look at various mechanisms that are spread out throughout the Windows system.